Dual-Stage Grid Power Quality System

on the STM32F401RE

Complete Study Guide: Concepts and Line-by-Line Firmware Walkthrough

Prepared for the project presentation and technical panel review

Firmware: `final/main.c` (1500 lines) Dashboard: `final/dashboard.py`

Repository: `github.com/dharineeshmv3/embed-project`

How to use this guide

Part I teaches every concept the project uses, from power-quality theory to the STM32 peripherals, with the actual numbers of this design worked out. **Part II** walks through `main.c` from the first line to the last. Every block of code is reproduced with its *real line numbers* (they match the file in the repository), followed by an explanation of what each line does and why. **Part III** explains the Python dashboard. **Part IV** is panel preparation: the numbers to remember, likely questions with answers, and known limitations.

Read Part I first, at least chapters 1–10. When you then read a block of code in Part II, the concept behind it will already be familiar. Boxes mark three kinds of material:

Key idea

Core concept to remember.

Calculation

A worked calculation with this project's real values.

Panel tip

How to say it to the panel, or what they are likely to ask.

Contents

I Concepts **7**

1 The project in one picture **8**

1.1 What the system does . . . 8

1.2 The data path in one paragraph . . . 8

1.3 The three time scales . . . 9

2 Power quality fundamentals **10**

2.1 RMS value . . . 10

2.2 Active, reactive, apparent and distortion power . . . 10

2.3 Power factor (PF) and displacement power factor (DPF) . . . 10

2.4 Harmonics and THD . . . 11

2.5 Capacitor power factor correction . . . 11

2.6 Active harmonic filtering . . . 11

3 Sensors and signal conditioning **13**

3.1 ZMPT101B voltage sensor . . . 13

3.2 ACS712 Hall-effect current sensor . . . 13

3.3 The voltage divider on sensor I . . . 13

3.4 From ADC counts to physical units . . . 13

3.5 Sensor noise . . . 14

4 Sampling theory and timing calculations **15**

4.1 Why sample at 10 kHz? . . . 15

4.2 Coherent (synchronous) sampling . . . 15

4.3 Windows and averaging . . . 15

4.4 ADC conversion time: does the scan fit? . . . 16

4.5 Inter-channel skew . . . 16

5 STM32F401RE architecture **17**

5.1 The core . . . 17

5.2 Buses and peripherals . . . 17

5.3 Register-level programming with CMSIS . . . 17

5.4 Memory use . . . 18

6 The clock system **19**

6.1 Clock tree . . . 19

6.2 Flash wait states and voltage scaling . . . 19

7 GPIO: digital and analog pins **20**

7.1 Pin modes . . . 20

7.2 Open-drain drive of 5 V relay modules . . . 20

7.3 Safe start-up . . . 20

8 Timers: sample clock and PWM **21**

8.1 How a timer counts . . . 21

8.2 Preload registers (ARPE, OCxPE) . . . 21

8.3 Trigger output (TRGO) and master/slave timers . . . 21

8.4 PWM mode 1 and the “always high” trick . . . 21

8.5 TB6612FNG sign-magnitude drive . . . 22

8.6 Fractional period: how the timer follows the grid . . . 22

9 ADC and DMA **23**

9.1 The successive-approximation ADC . . . 23

9.2 Direct memory access . . . 23

9.3 DMA for UART transmission . . . 23

10 Interrupts, priorities and shared data **24**

10.1 The NVIC . . . 24

10.2 Rules for data shared with interrupts . . . 24

10.3 Sleeping . . . 24

11 Serial communication and protocol **25**

11.1 UART basics . . . 25

11.2 The text protocol . . . 25

12 Signal processing: DFT, phasors and noise **26**

12.1 The discrete Fourier transform used here . . . 26

12.2 Phasors . . . 26

12.3 Fundamental power from the DFT . . . 26

12.4 Separating real current from sensor noise . . . 26

12.5 Deadbands . . . 27

13 Digital filters **28**

13.1 Butterworth low-pass . . . 28

13.2 Filtering without spoiling the measurement . . . 28

14 Software phase-locked loop (grid lock) **30**

14.1 Why lock to the grid? . . . 30

14.2 How the lock works . . . 30

15 Capacitor bank control (APFC) 31

15.1 Decision algorithm (every 0.5 s) . . . 31

15.2 Relay safety: why the timing matters . . . 31

16 Active harmonic filter control 33

16.1 The problem with a simple copy . . . 33

16.2 Selective harmonic closed loop . . . 33

16.3 From phasors to PWM . . . 33

16.4 Measured results . . . 33

17 Execution timeline from reset 35

II Line-by-line firmware walkthrough 36

18 File header, includes and constants 37

18.1 Header comment and includes (L1–58) . . . 37

18.2 Clock and sampling constants (L60–89) . . . 38

18.3 Grid lock constants (L91–96) . . . 39

18.4 Sensor calibration and deadbands (L98–130) . . . 39

18.5 APFC and AHF constants (L132–181) . . . 40

18.6 Relay constants and point-on-wave indices (L183–221) . . . 42

19 The DSP core 43

19.1 Complex-number helpers (L222–251) . . . 43

19.2 Tables and state of the DSP core (L253–272) . . . 43

19.3 Initial AHF plant model (L274–282) . . . 44

19.4 Butterworth design (L284–296) . . . 44

19.5 Zero-phase filter (L298–323) . . . 45

19.6 Filter set-up and response table (L325–339) . . . 46

19.7 Start-up tables (L341–352) . . . 46

19.8 DFT and THD (L354–375) . . . 46

19.9 AHF control step (L377–457) . . . 47

19.10 PLL update and period calculation (L459–493) . . . 49

20 Global variables 51

21 Hardware initialisation 54

21.1 `clock_init` (L560–590) . . . 54

21.2 Relay output functions (L592–623) . . . 55

21.3 `gpio_init` (L625–665) . . . 56
21.4 Optional relay self-test (L667–686) . . . 57

22 Acquisition: TIM2, ADC, DMA **58**
22.1 `tim2_init` (L688–703) . . . 58
22.2 `adc_dma_init` (L705–730) . . . 58
22.3 Window hand-off and error interrupts (L732–758) . . . 59
22.4 `tim4_init` (L760–779) . . . 60
22.5 The 10 kHz interrupt: `TIM2_IRQHandler` (L781–815) . . . 61
22.6 Starting acquisition: `sampling_sync_start` (L817–852) . . . 62
22.7 Applying the PLL result: `pll_apply` (L854–868) . . . 63

23 UART: telemetry and commands **64**
23.1 UART set-up, DMA transmit and its interrupt (L870–912) . . . 64
23.2 Command reception and parsing (L914–981) . . . 65

24 Signal processing: `process_window` **67**
24.1 Part 1: folding and the sensor-A filter (L983–1018) . . . 67
24.2 Part 2: offsets, rms, noise separation, power (L1019–1067) . . . 68
24.3 Part 3: averaged waveforms, DFT, sensor-A correction (L1069–1120) . . . 69
24.4 Part 4: fundamental powers and the compensated spectrum (L1122–1150) . . . 70
24.5 Part 5: display smoothing, target and compensated waveform (L1152–1166) . . . 71

25 Control: AHF step and APFC **72**
25.1 `ahf_step` (L1168–1188) . . . 72
25.2 APFC helpers (L1190–1237) . . . 72
25.3 `apfc_step` (L1239–1282) . . . 73
25.4 `relay_service` (L1284–1333) . . . 74

26 Telemetry, start-up and the main loop **77**
26.1 Text formatting helpers (L1335–1372) . . . 77
26.2 `telemetry_send` (L1374–1419) . . . 78
26.3 `nvic_init` (L1421–1436) . . . 79
26.4 `on_window` (L1438–1458) . . . 79
26.5 `main` (L1460–1501) . . . 80

III The PC dashboard **82**

27 Dashboard architecture **83**
27.1 What the dashboard is for . . . 83
27.2 Libraries . . . 83

27.3 Threads and data flow . . . 83

28 Dashboard code walkthrough **85**

28.1 Protocol constants (L46–60) . . . 85
28.2 The frame format, side by side . . . 85
28.3 `Telemetry` (L64–151) . . . 86
28.4 `find_port` and `SerialLink` (L157–222) . . . 86
28.5 `DemoLink`: the built-in simulator (L228–394) . . . 86
28.6 Helper functions (L397–425) . . . 87
28.7 Widgets (L430–775) . . . 87
28.8 `Dashboard` window (L777–1284) . . . 87
28.8.1 Construction . . . 87
28.8.2 Commands . . . 88
28.8.3 Refresh . . . 88

IV Panel preparation **89**

29 Numbers to remember **90**

29.1 Measured results . . . 91

30 Likely panel questions with answers **92**

30.1 System and concept . . . 92
30.2 Sampling and timing . . . 92
30.3 Timers and PWM . . . 93
30.4 ADC, DMA and interrupts . . . 93
30.5 DSP . . . 94
30.6 PLL and grid lock . . . 95
30.7 APFC . . . 95
30.8 AHF . . . 95
30.9 Communication . . . 96

31 Limitations and honest answers **97**

31.1 Future scope . . . 97

32 Glossary **98**

Part I

Concepts

1 The project in one picture

1.1 What the system does

Electrical loads such as phone chargers, laptop adapters, LED drivers, fans and motors do not all draw a clean sinusoidal current in phase with the voltage. Two different problems result:

1. **Displacement (phase shift).** Inductive loads such as motors and fans draw a sinusoidal current that *lags* the voltage. This needs reactive power Q and lowers the power factor. The classic remedy is **capacitors**, which supply leading reactive power.
2. **Distortion (harmonics).** Rectifier loads such as chargers draw current in short pulses near the voltage peak. That current contains **harmonics**, multiples of 50 Hz. A capacitor cannot cancel them; an **active harmonic filter** (AHF) can, by injecting an equal and opposite harmonic current.

The project therefore has two stages, controlled by one STM32F401RE microcontroller:

- **Stage 1, APFC (Automatic Power Factor Correction):** three capacitors (2.5, 10 and 20 μF), switched by two relay modules and one solid-state relay (SSR). The firmware calculates how much capacitance the load needs and switches in the best combination.
- **Stage 2, AHF (Active Harmonic Filter):** a TB6612FNG H-bridge driven by 80 kHz PWM. It injects a current shaped so that harmonics 2 to 15 of the load current are cancelled.

A Python dashboard on the PC receives measurements twice per second over USB and lets the operator lock capacitors or switch the AHF on and off.

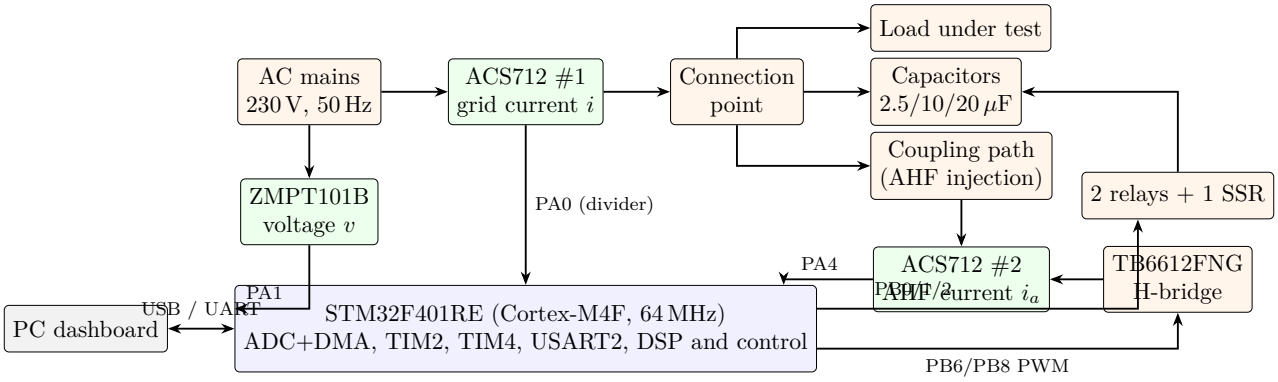


Figure 1.1: System overview. Three analog signals go into the ADC; GPIOs drive the capacitor switches; two PWM outputs drive the H-bridge; USART2 talks to the PC.

1.2 The data path in one paragraph

Timer **TIM2** ticks 10 000 times per second, exactly 200 times per grid cycle, because the firmware locks it to the grid. Each tick triggers the **ADC** to convert the three analog inputs (grid current, voltage, AHF current) one after another. The **DMA** controller copies each result into a RAM buffer without the CPU. When 1000 sample sets (100 ms, i.e. five grid cycles) have been collected, the DMA raises an interrupt. The **main loop** then copies that block, averages the five cycles into one clean cycle, and runs a **DFT** to obtain harmonics 1 to 15. From these it computes rms values, active, reactive and distortion power, power factor and THD. It updates the **grid lock** (software PLL), runs the **AHF control loop** to produce the next cycle of PWM duty values, and every 0.5 s it runs

the **capacitor decision** and sends a **telemetry frame** to the PC. In parallel, the same TIM2 tick interrupt outputs the next PWM duty value to **TIM4**, which drives the H-bridge at 80 kHz, and switches relay coils at exactly the planned instant of the mains cycle.

1.3 The three time scales

Period	Triggered by	What happens
100 μs (10 kHz)	TIM2 update interrupt	ADC scan of 3 channels (hardware); DMA stores results; ISR adjusts the timer period (grid lock), outputs one AHF duty value to TIM4, switches scheduled relay coils
12.5 μs (80 kHz)	TIM4 counter	One PWM period of the H-bridge (8 per sample)
100 ms	DMA half/full-transfer interrupt	One *window*: 1000 samples per channel = 5 grid cycles. Main loop: measurements, DFT, PLL update, AHF control step
0.5 s	every 5th window	Averaged measurements, capacitor decision (APFC), telemetry frame to PC, LED toggle

Table 1.1: Timing hierarchy of the firmware.

Panel tip

A one-sentence summary for the panel: *"A timer-triggered, DMA-driven ADC samples voltage and two currents at exactly 200 samples per grid cycle; every 100 ms the firmware computes a DFT-based power analysis, closes a per-harmonic control loop that drives an 80 kHz H-bridge, and every half second decides the capacitor combination and streams the results to a PC dashboard. Everything is written at register level with CMSIS, without HAL."*

2 Power quality fundamentals

2.1 RMS value

The **root-mean-square** (rms) value of a periodic signal is the DC value that would deliver the same heating power into a resistor:

$$X_{\text{rms}} = \sqrt{\frac{1}{T}\int_0^T x^2(t)\,dt} \quad \xrightarrow{\text{sampled}} \quad X_{\text{rms}} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} x_n^2}.$$

For a sine wave of peak X_{pk}, $X_{\text{rms}} = X_{\text{pk}}/\sqrt{2}$. The 230 V mains therefore peaks at $230\sqrt{2} = 325$ V. The firmware uses the sampled form directly: it sums the squared samples over 1000 samples (5 complete cycles) and takes the square root. Averaging over *whole* cycles matters, because the average of a sinusoid over a partial cycle is not zero.

2.2 Active, reactive, apparent and distortion power

- **Active power** P [W], the energy actually converted per second, is the average of instantaneous power:
$$P = \frac{1}{N}\sum_n v_n i_n.$$
- **Apparent power** S [VA] is the product of the rms values: $S = V_{\text{rms}} I_{\text{rms}}$. It determines the current rating of cables and transformers.
- **Fundamental reactive power** Q_1 [var] comes from the 50 Hz components only: $Q_1 = V_1 I_1 \sin\varphi_1$, where φ_1 is the phase angle of the fundamental current behind the voltage. A positive Q_1 means the current lags (inductive load); a negative Q_1 means it leads (capacitive).
- **Distortion power** D [VA] is everything in S that is neither P nor Q_1. It comes from harmonic currents (IEEE 1459). With a nearly sinusoidal voltage:
$$S^2 = P^2 + Q_1^2 + D^2 \quad \Rightarrow \quad D = \sqrt{S^2 - P^2 - Q_1^2}.$$

2.3 Power factor (PF) and displacement power factor (DPF)

$$\text{PF} = \frac{P}{S} \qquad \text{DPF} = \cos\varphi_1 = \frac{P_1}{\sqrt{P_1^2 + Q_1^2}}.$$

PF is the *true* power factor and includes the effect of harmonics. DPF only measures the phase shift of the fundamental. For a sinusoidal voltage they are related through the current THD:

$$\boxed{\text{PF} \approx \frac{\text{DPF}}{\sqrt{1 + \text{THD}_I^2}}}$$

Your laptop charger (live measurement)

Measured: $P = 97.5\,\mathrm{W}$, $Q_1 = +15.3\,\mathrm{var}$, $S = 105.7\,\mathrm{VA}$, $D = 37.7\,\mathrm{VA}$, DPF $= 0.988$, $\mathrm{THD}_I = 34\,\%$.
Check 1: $\sqrt{97.5^2 + 15.3^2 + 37.7^2} = 105.6\,\mathrm{VA} \approx S$ ✓
Check 2: $0.988/\sqrt{1 + 0.34^2} = 0.935$, measured PF $= 0.923$ ✓
Conclusion: the PF of 0.92 is caused almost entirely by harmonics (D), not by phase shift (DPF ≈ 0.99). Capacitors cannot help here; the AHF can.

2.4 Harmonics and THD

Any periodic waveform can be written as a sum of sinusoids at integer multiples h of the fundamental frequency (Fourier series):

$$x(t) = \sum_{h=1}^{\infty} \left(a_h \cos(h\omega t) + b_h \sin(h\omega t)\right).$$

Harmonic h has frequency $h \times 50\,\mathrm{Hz}$: the 3rd harmonic is 150 Hz and the 15th is 750 Hz. Rectifier loads mainly produce odd harmonics (3, 5, 7, ...). **Total harmonic distortion** compares the rms of all harmonics with the fundamental:

$$\mathrm{THD} = \frac{\sqrt{\sum_{h=2}^{15} X_h^2}}{X_1} \times 100\,\%.$$

The firmware computes harmonics up to $h = 15$, which is the band the AHF corrects. Standards: IEEE 519 limits voltage THD at the point of common coupling to 8 % for systems up to 1 kV, and IEC 61000-3-2 limits the harmonic currents of equipment. That standard is why laptop adapters above 75 W contain active PFC.

2.5 Capacitor power factor correction

A capacitor C on voltage V draws a leading current and supplies reactive power

$$Q_C = \omega C V^2 = 2\pi f C V^2.$$

Reactive power per microfarad at 230 V, 50 Hz

$k = 2\pi \cdot 50 \cdot 230^2 \cdot 10^{-6} = 16.62\,\mathrm{var}$ per μF. So the 2.5 μF step gives 41.5 var (0.18 A), the 10 μF step 166 var (0.72 A) and the 20 μF step 332 var (1.45 A).

To raise the displacement factor of a load with active power P and reactive power Q_{load} to a target PF_t, the remaining reactive power may be at most $Q_{\mathrm{tgt}} = P \tan(\arccos \mathrm{PF}_t)$. The capacitance needed is therefore

$$C_{\mathrm{need}} = \frac{Q_{\mathrm{load}} - Q_{\mathrm{tgt}}}{k}.$$

The firmware uses $\mathrm{PF}_t = 0.99$, so $\tan(\arccos 0.99) = 0.1425$.

2.6 Active harmonic filtering

A **shunt active filter** is a controlled current source connected in parallel with the load. If the load current is $i_L = i_1 + i_h$ (fundamental plus harmonics) and the filter injects $i_a = -i_h$, the grid supplies $i_L + i_a = i_1$, a clean sine. In this project the current source is an H-bridge (TB6612FNG) whose

output voltage is set by PWM. The bridge pushes current through a coupling path, whose details are not documented, so the firmware measures its behaviour. Because the bridge is a *voltage* source, the current it produces at each harmonic depends on the impedance of that path at that frequency. That is why the controller works harmonic by harmonic (Chapter 16).

Key idea

Capacitors correct **phase shift** (Q_1, DPF). The AHF corrects **distortion** (harmonics, D). The true PF needs both. This is why the system has two stages and why the firmware measures DPF and THD separately.

3 Sensors and signal conditioning

3.1 ZMPT101B voltage sensor

The ZMPT101B module contains a small voltage transformer and an op-amp stage. It produces a low-voltage replica of the mains voltage centred on a DC offset (about half of its supply). The ratio between mains volts and ADC volts was calibrated as `ZMPT_V_PER_V` = 1014: 1 V at the ADC pin represents 1014 V of mains. A 230 V mains (325 V peak) therefore appears as 325/1014 = 0.32 V peak around the offset, which is about ±398 ADC counts. The voltage channel is connected to **PA1** (ADC channel 1).

3.2 ACS712 Hall-effect current sensor

The ACS712-05B carries the load current through a copper path inside the IC. A Hall sensor converts the magnetic field into a voltage:

$$V_{\text{out}} = \frac{V_{CC}}{2} + 0.185\,\text{V/A} \times i \qquad (V_{CC} = 5\,\text{V} \Rightarrow 2.5\,\text{V at } 0\,\text{A}).$$

The output is *ratiometric*: both the zero point and the sensitivity scale with the actual supply voltage. Two of them are used: sensor I on **PA0** measures the current drawn from the grid, and sensor A on **PA4** measures the current injected by the AHF.

3.3 The voltage divider on sensor I

The STM32 ADC measures 0 to 3.3 V. With the ACS712 sitting at 2.5 V for zero current, there is only 3.3 − 2.5 = 0.8 V of headroom for positive current, i.e. 0.8/0.185 = 4.3 A, but 13.5 A for negative current. A 6 A load therefore *clipped* on every positive half-cycle, which you saw as a flat-topped waveform and a false THD of 22.9 %. The fix is a resistive divider: 10 kΩ in series from the sensor output and 20 kΩ from the ADC pin to ground.

Divider

Ratio $\alpha = \dfrac{20}{10+20} = 0.6667$. Zero current: $2.5 \times 0.6667 = 1.667\,\text{V} \Rightarrow 1.667/3.3 \times 4095 = 2068$ counts.
Range: $-\dfrac{2.5}{0.185} = -13.5\,\text{A}$ to $\dfrac{3.3/0.6667 - 2.5}{0.185} = +13.2\,\text{A}$, almost symmetric.
The source impedance seen by the ADC is $10\,\text{k} \parallel 20\,\text{k} = 6.7\,\text{k}\Omega$, which is acceptable with the long (480-cycle) sampling time used.

3.4 From ADC counts to physical units

The 12-bit ADC returns 0 . . . 4095 for 0 . . . 3.3 V, so 1 count = 3.3/4095 = 0.806 mV. Each channel has a fixed conversion factor (defined at lines 128–130 of `main.c`): The firmware does **not** assume the zero point. It removes the DC offset by subtracting the mean of each window. Over an integer number of cycles the mean of the AC part is zero, so the mean is exactly the offset. This cancels the effect of resistor tolerance and supply drift on the zero point.

Channel	Formula	Meaning	Value per count
K_I (grid current)	$\dfrac{3.3/4095}{0.185 \times 0.6667}$	mV $\rightarrow$ A, undo divider	6.534 mA
K_A (AHF current)	$\dfrac{3.3/4095}{0.185 \times 1}$	no divider	4.356 mA
K_V (voltage)	$\dfrac{3.3}{4095} \times 1014$	ADC volts $\rightarrow$ mains volts	0.817 V

Table 3.1: Scaling constants.

3.5 Sensor noise

Even with no current or voltage present, each reading jitters by a few counts (sensor noise plus ADC noise). Measured on your hardware: about 31 mA rms on sensor I and about 6 V rms on the voltage channel with the mains disconnected. Chapter 12 explains how the firmware deals with this: deadbands, a noise-separation method for the current, and display filtering.

4 Sampling theory and timing calculations

4.1 Why sample at 10 kHz?

Nyquist theorem: to represent a signal without aliasing, the sampling rate must exceed twice the highest frequency present. The highest frequency the system processes is the 15th harmonic, 750 Hz. Sampling at 10 kHz gives a Nyquist frequency of 5 kHz, a comfortable margin above 750 Hz. It also leaves enough points per cycle to draw the waveform properly (200 points per 20 ms cycle).

4.2 Coherent (synchronous) sampling

$$f_s = 200 \times f_{\text{grid}} \qquad \Rightarrow \qquad \text{exactly 200 samples per cycle.}$$

If the number of samples per cycle is an integer and the DFT covers whole cycles, every harmonic falls *exactly* on a DFT bin. There is then no spectral leakage and no need for windowing functions. Because the real grid frequency drifts (your mains measured 49.02 to 49.45 Hz), the firmware continuously adjusts the sampling rate with a software PLL (Chapter 14). At 49.02 Hz the sample rate becomes $200 \times 49.02 = 9804$ Hz.

4.3 Windows and averaging

- One **window** = $N_{\text{WIN}} = 1000$ samples per channel = 5 cycles = 100 ms at 50 Hz.
- The 5 cycles are **folded**: sample k of the averaged cycle is the mean of samples $k, k+200, k+400, k+600, k+800$. This is **coherent averaging**. The repeating part (the load's waveform) is unchanged, while random noise falls by a factor $\sqrt{5} = 2.24$.

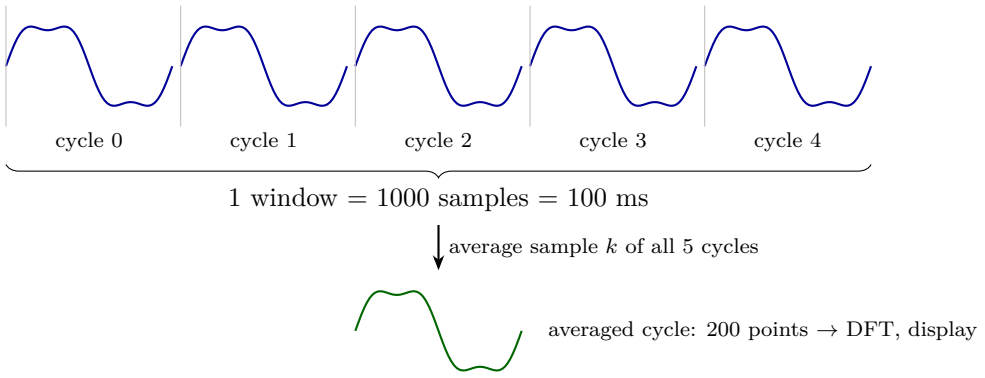


Figure 4.1: Folding five cycles into one averaged cycle.

4.4 ADC conversion time: does the scan fit?

ADC timing

ADC clock = 64 MHz/4 = 16 MHz. Per channel: 480 sampling cycles + 12 conversion cycles = 492 cycles = 492/16 MHz = 30.75 μs.
Three channels: $3 \times 30.75 = 92.25\,\mu$s. Sample period at 50 Hz: $1/10\,000 = 100\,\mu$s ✓.
At the PLL's highest allowed frequency (52.5 Hz): $1/(200 \times 52.5) = 95.2\,\mu\text{s} > 92.25\,\mu$s ✓.
A compile-time check (`L180`) refuses to build if this margin is ever violated (checked at 53 Hz).

Why such a long sampling time? The ADC charges an internal sample-and-hold capacitor through the source impedance. With the 6.7 kΩ divider and the sensor modules' output resistance, a short sampling time would not let the capacitor settle, and readings would be inaccurate. 480 cycles is the longest setting and the most accurate. The scan still fits.

4.5 Inter-channel skew

The ADC converts one channel at a time: current at t, voltage at $t + 30.75\,\mu$s, AHF current at $t + 61.5\,\mu$s. At 50 Hz a 30.75 μs delay corresponds to a phase shift of $360° \times 50 \times 30.75 \times 10^{-6} = 0.55°$. That would bias the measured reactive power slightly, so the firmware *rotates* the voltage phasor back by this angle before computing P_1 and Q_1 (`L1122–1129`). For the AHF channel the skew does not need correcting, because it is absorbed automatically by the on-line plant identification (Chapter 16).

Quantity	Value
Sample rate (50 Hz grid)	10 000 Hz (200 per cycle)
Sample period	100 μs (6400 timer ticks of 15.625 ns)
ADC conversion per channel / scan	30.75 μs / 92.25 μs
Window length	1000 samples = 5 cycles = 100 ms
DFT resolution	50 Hz (one bin per harmonic)
Harmonics computed	1 to 15 (50 to 750 Hz)
Nyquist frequency	5 kHz
Noise reduction by averaging	$\sqrt{5} \approx 2.24$

Table 4.1: Sampling numbers to remember.

5 STM32F401RE architecture

5.1 The core

The STM32F401RE is built around an **ARM Cortex-M4** core with a single-precision **floating-point unit** (FPU), 512 KB of Flash and 96 KB of SRAM. It runs at up to 84 MHz; this project uses 64 MHz (Chapter 6). The FPU executes `float` arithmetic in hardware. That is what makes it practical to run DFTs, filters and complex-number control loops every 100 ms. The FPU must be switched on by software before the first floating-point instruction (`L1462`).

5.2 Buses and peripherals

Peripherals sit on three buses: **AHB1** (GPIO ports, DMA controllers, RCC), **APB1** (TIM2, TIM4, USART2, PWR) and **APB2** (ADC1). Each bus has its own clock and prescaler. Before a peripheral can be used, its clock must be enabled in the **RCC** (Reset and Clock Control) enable registers. Every initialisation function in the firmware starts by doing exactly that.

Peripheral	Bus (clock)	Used for
GPIOA, GPIOB	AHB1 (64 MHz)	analog inputs, UART pins, LED, relays, PWM pins
DMA1 Stream 6 Channel 4	AHB1	USART2 transmit (telemetry)
DMA2 Stream 0 Channel 0	AHB1	ADC1 results to RAM
TIM2 (32-bit)	APB1 timers (64 MHz)	sample clock, grid lock, 10 kHz interrupt
TIM4 (16-bit)	APB1 timers (64 MHz)	80 kHz PWM for the H-bridge
USART2	APB1 (32 MHz)	115 200 bit/s link through ST-LINK
ADC1	APB2 (64 MHz, ADC 16 MHz)	3-channel scan conversion
PWR, FLASH	–	voltage scaling, wait states
NVIC, SCB	core	interrupts, FPU enable

Table 5.1: On-chip resources used.

5.3 Register-level programming with CMSIS

The firmware uses no HAL library. It writes directly to peripheral registers using the CMSIS device header `stm32f401xe.h`. That header defines:

- a **structure per peripheral** (for example `TIM2->ARR` is the auto-reload register of TIM2 at its fixed memory address);
- **bit masks** and positions (for example `TIM_CR1_CEN` is the counter-enable bit, `ADC_SQR3_SQ1_Pos` is the position of the first sequence field).

Typical idioms you will see in Part II:

- `REG |= MASK;` sets bits (read–modify–write).
- `REG &= ~MASK;` clears bits.
- `REG = (REG & ~FIELD) | (value << POS);` replaces one multi-bit field and leaves the rest untouched.
- `(void)RCC->APB1ENR;` a dummy read after enabling a clock. The STM32F4 errata require a short delay between enabling a peripheral clock and accessing the peripheral, and a read-back provides it.

- `GPIOB->BSRR = ...;` the Bit Set/Reset Register: writing a 1 to bit n sets pin n, writing a 1 to bit n+16 clears it. This is *atomic*: no read–modify–write, so an interrupt cannot corrupt it.

Panel tip

Why no HAL? (1) Full control and visibility of every register setting, which matters for the precise timer/ADC/DMA synchronisation; (2) smaller and faster code with deterministic interrupt timing; (3) the panel can see exactly how each peripheral is configured. Be ready to say: "Every peripheral is configured by writing its registers directly, using the bit definitions in the CMSIS header."

5.4 Memory use

Compiled with GCC `-O2`, the firmware module needs about 12 KB of Flash (2.3 % of 512 KB) and about 36 KB of static RAM (38 %). The biggest items are the 12 KB DMA buffer (`adc_buf`: $2 \times 1000 \times 3$ samples $\times$ 2 bytes) and its 6 KB processing copy (`snap`).

6 The clock system

6.1 Clock tree

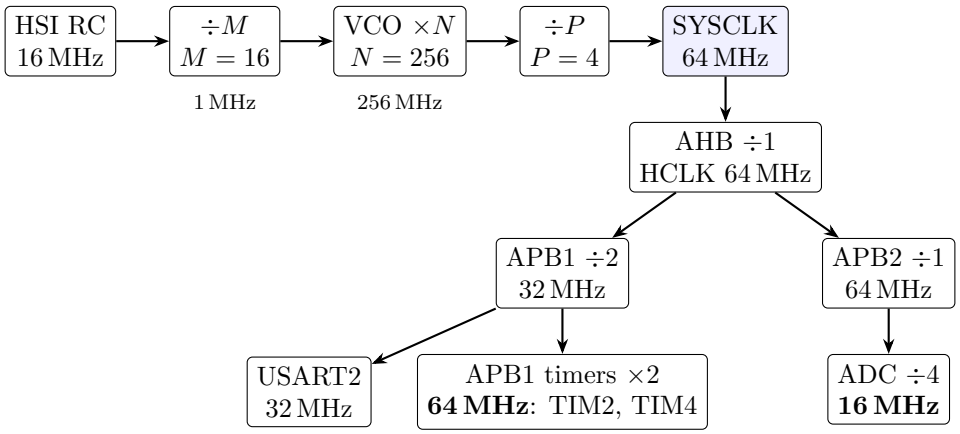


Figure 6.1: Clock tree used by the firmware.

PLL

VCO input = 16 MHz/M = 16/16 = 1 MHz (allowed 1–2 MHz).
VCO output = 1 MHz × N = 256 MHz (allowed 192–432 MHz).
SYSCLK = 256/P = 256/4 = 64 MHz. (In the register, $P = 4$ is encoded as the value 1.)
÷Q = 8 gives 32 MHz on the 48 MHz-domain output, unused here but within its limit.

Why 64 MHz and not 84 MHz? 64 MHz divides into all the frequencies the design needs with whole numbers:

- TIM2: 64 MHz/10 kHz = 6400 ticks per sample.
- TIM4: 6400/8 = 800 ticks per PWM period, giving exactly 80 kHz and exactly 8 PWM periods per sample.
- ADC: 64/4 = 16 MHz, within the ADC's 36 MHz limit.
- USART2 on APB1 at 32 MHz.

APB1 timer clock doubling: APB1 runs at 32 MHz (its maximum is 42 MHz). Whenever the APB1 prescaler is not 1, the hardware doubles the clock sent to the timers on that bus, so TIM2 and TIM4 count at 64 MHz.

6.2 Flash wait states and voltage scaling

Flash memory cannot be read in a single cycle at 64 MHz. The reference manual requires **2 wait states** for 60 MHz < HCLK ≤ 90 MHz at 2.7–3.6 V. The firmware also enables the prefetch buffer and the instruction and data caches (ART accelerator) to hide this latency. The internal regulator is set to **voltage scale 2**, which supports up to 84 MHz. The wait states must be set *before* the clock is switched to the faster PLL, otherwise the CPU would read Flash too fast and crash.

7 GPIO: digital and analog pins

7.1 Pin modes

Each GPIO pin has a 2-bit field in the `MODER` register: `00` input, `01` general-purpose output, `10` alternate function (a peripheral such as a timer or UART controls the pin), `11` analog (digital circuitry disconnected, pin connected to the ADC). Further registers per pin:

- `OTYPER` (1 bit): push-pull (0) or open-drain (1);
- `OSPEEDR` (2 bits): output edge speed (low / medium / fast / high);
- `PUPDR` (2 bits): none / pull-up / pull-down;
- `AFR[0]` (pins 0–7) and `AFR[1]` (pins 8–15), 4 bits per pin: which peripheral (AF0–AF15) is connected in alternate-function mode.

Pin (header)	Mode	Function	Why this setting
PA0 (A0)	analog	ADC1 IN0, grid current	ADC input
PA1 (A1)	analog	ADC1 IN1, voltage	ADC input
PA4 (A2)	analog	ADC1 IN4, AHF current	ADC input
PA2 / PA3	AF7	USART2 TX / RX	connected to ST-LINK USB bridge
PA5	output	LED LD2	heartbeat (toggles every 0.5 s)
PB0, PB1	output, **open-drain**	relay modules (2.5, 10 μF)	5 V active-LOW inputs
PB2	output, push-pull	SSR (20 μF)	active-HIGH input
PB6 (D10)	AF2	TIM4 CH1 $\rightarrow$ TB6612 input	PWM
PB8 (D15)	AF2	TIM4 CH3 $\rightarrow$ TB6612 input	PWM

Table 7.1: Pin configuration.

7.2 Open-drain drive of 5 V relay modules

Typical relay modules have an **active-LOW** input: pulling IN to 0 V turns the relay ON. The module has its own pull-up to 5 V through the opto-coupler LED.

- **Push-pull problem:** a 3.3 V push-pull HIGH leaves about 1.7 V across the 5 V-referenced opto LED. That can keep it partly conducting, so the relay may not fully release.
- **Open-drain solution:** the pin either pulls LOW (relay ON) or floats (relay OFF). When it floats, the module's own pull-up takes IN all the way to 5 V, so the opto LED has 0 V across it and the relay releases completely. The internal pull-up must stay disabled, because it would clamp the line to 3.3 V. This requires 5 V-tolerant (FT) pins.

In open-drain mode, writing "1" to the output means *release* (high impedance) and writing "0" means *pull LOW*. So for a relay module, ON = pin LOW and OFF = pin "HIGH" (released). For the SSR (push-pull, active-HIGH), ON = HIGH. The firmware handles both kinds with one bit mask, `RELAY_SSR_MASK` (§21.2).

7.3 Safe start-up

At reset all pins are inputs, so the relays are OFF. `gpio_init()` sets the output type and writes the OFF level *before* switching the relay pins to output mode. That way the pins never drive LOW, even for a microsecond, during start-up. Otherwise the relays could click ON at every reset.

8 Timers: sample clock and PWM

8.1 How a timer counts

A general-purpose timer has:

- a **prescaler** `PSC`: the counter clock is $f_{\text{TIM}}/(\text{PSC}+1)$;
- a **counter** `CNT` that counts up from 0;
- an **auto-reload** register `ARR`: when `CNT` reaches `ARR`, the next tick wraps it to 0 and generates an **update event** (UEV).

$$f_{\text{update}} = \frac{f_{\text{TIM}}}{(\text{PSC}+1)(\text{ARR}+1)}$$

TIM2 and TIM4

TIM2: $\dfrac{64\,\text{MHz}}{(0+1)(6399+1)} = 10\,000\,\text{Hz}$, one sample every $100\,\mu$s.

TIM4: $\dfrac{64\,\text{MHz}}{(0+1)(799+1)} = 80\,000\,\text{Hz}$, an 80 kHz PWM period of $12.5\,\mu$s.

Duty resolution: 800 steps (0.125 % per step). The TB6612FNG accepts PWM up to 100 kHz.

8.2 Preload registers (ARPE, OCxPE)

With preload enabled, a new `ARR` or compare value (`CCRx`) written by software goes into a *shadow* register. It only takes effect at the next update event. Changes therefore always happen at a period boundary and never cut a PWM pulse or a sample period in half. Both timers use preload.

8.3 Trigger output (TRGO) and master/slave timers

A timer can output an internal trigger signal, **TRGO**, on selected events. TIM2 is configured with `MMS = 010`, which means *TRGO = update event*. This one signal is used twice:

1. The **ADC** is set to start a conversion on the rising edge of TIM2_TRGO. Sampling is therefore paced by hardware, with no software jitter.
2. **TIM4** is a *slave* of TIM2 in **reset mode**: every TIM2 update resets TIM4's counter to zero. (TIM4's internal trigger input ITR1 is connected to TIM2.) The 8 PWM periods inside each sample period are therefore always aligned to the ADC sampling instant. If the PWM ran freely, its ripple would be sampled at a slowly drifting phase and appear as a false low-frequency component (aliasing).

8.4 PWM mode 1 and the "always high" trick

In PWM mode 1, the output reference `OCxREF` is HIGH while `CNT` < `CCRx` and LOW otherwise: If `CCRx` is larger than `ARR`, the condition `CNT` < `CCRx` is always true and the output stays permanently HIGH. The firmware uses `0xFFFF` for that. If `CCRx`= 0, the output stays permanently LOW.

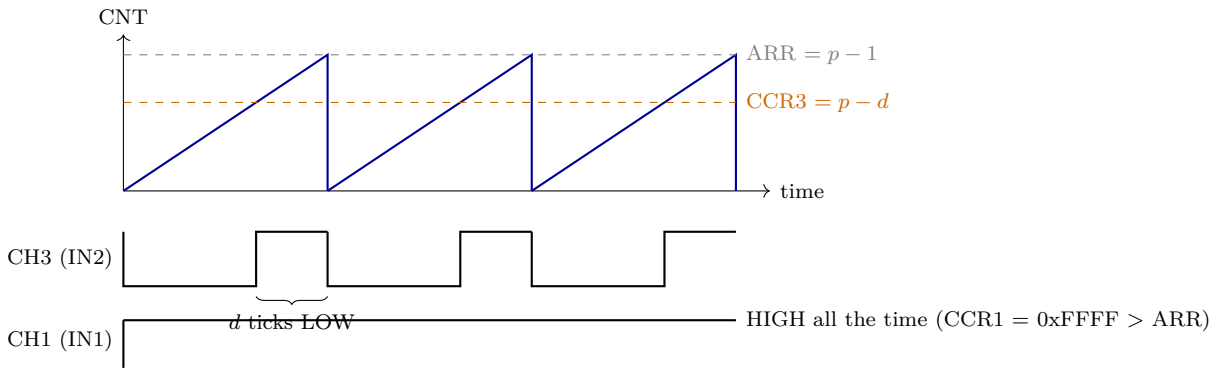


Figure 8.1: Sign-magnitude PWM for a positive duty: IN1 is held HIGH, IN2 goes LOW for d of p ticks.

8.5 TB6612FNG sign-magnitude drive

The TB6612FNG truth table for one channel (inputs IN1, IN2; the PWM and STBY pins are held HIGH *(wiring to verify)*):

IN1	IN2	Outputs	Effect
H	L	OUT1= $+V_M$, OUT2= 0	drive **positive**
L	H	OUT1= 0, OUT2= $+V_M$	drive **negative**
H	H	both LOW	short **brake** (0 V across the load)
L	L	high impedance	stop / off

For a positive duty d, the firmware holds IN1 HIGH and makes IN2 LOW for d ticks of every period. The bridge alternates between drive ($+V_M$) and brake (0 V), so the *average* output voltage is $V_M \cdot d/p$. A negative duty swaps the roles. This is called **sign-magnitude, slow-decay (drive/brake)** PWM: its average voltage is linear in the duty. With the AHF in STANDBY both compare values are 0, both inputs are LOW, and the bridge outputs are off.

8.6 Fractional period: how the timer follows the grid

To follow a grid at 49.02 Hz the sample period must be $64\,\text{MHz}/(200 \times 49.02) = 6527.95$ ticks, which is not a whole number. The firmware uses **dithering**: it keeps a 16-bit fractional accumulator, and on every sample adds the fractional part. Whenever it overflows, that one period is one tick longer. Over many samples the *average* period equals the exact value, with a resolution of $1/65\,536$ of a tick. A single-tick step would only give $50/6400 = 0.0078$ Hz resolution.

9 ADC and DMA

9.1 The successive-approximation ADC

ADC1 is a 12-bit successive-approximation converter. In **scan mode** it converts a programmed *sequence* of channels one after another after a single trigger. The sequence here is IN0 (current), IN1 (voltage), IN4 (AHF current), set in the sequence registers `SQR3` (order) and `SQR1` (length = 3). Each conversion takes the programmed sampling time (480 cycles) plus 12 cycles. The trigger is the rising edge of TIM2_TRGO (`EXTSEL`= 6, `EXTEN`= 01). An **overrun** occurs if a new result is ready before the previous one was read. It raises the ADC interrupt, and the firmware then restarts the acquisition chain.

9.2 Direct memory access

Reading three ADC results 10 000 times per second with the CPU would cost 30 000 interrupts per second. Instead, **DMA2 Stream 0, Channel 0** (the ADC1 request line) moves each 16-bit result from `ADC1->DR` to the next location of a RAM array, with no CPU involvement.

- **Peripheral and memory size** 16 bit; memory address increments, peripheral address is fixed.
- **Circular mode:** after the last element the DMA starts again at the beginning, forever.
- **Half-transfer (HT) and transfer-complete (TC) interrupts**, which give double buffering ("ping-pong"):

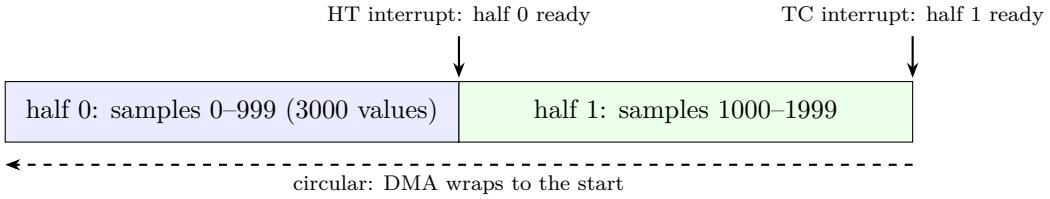


Figure 9.1: DMA ping-pong buffer `adc_buf` (6000 half-words = 12 KB, interleaved I, V, A, I, V, A, ...).

While the CPU processes one half, the DMA fills the other half. The CPU has a full 100 ms to process a window before that half is overwritten. To be completely safe, the main loop first copies the finished half into a separate array (`snap`) and processes the copy.

9.3 DMA for UART transmission

Telemetry frames (about 4 KB every 0.5 s) are sent the same way: the CPU formats the text into `tx_buf`, then **DMA1 Stream 6, Channel 4** (the USART2_TX request) feeds the bytes to the UART. Transmission takes about 0.35 s at 115 200 bit/s and runs entirely in the background. The CPU only gets a completion interrupt.

10 Interrupts, priorities and shared data

10.1 The NVIC

The Nested Vectored Interrupt Controller calls a specific function, the **interrupt service routine** (ISR), when a peripheral raises an interrupt. The function names (for example `TIM2_IRQHandler`) are fixed by the start-up file's vector table. Each interrupt has a **priority**: on the STM32F4 a lower number is more urgent, and a more urgent interrupt can *preempt* (interrupt) a less urgent ISR that is running.

ISR	Rate	Prio	Work
`TIM2_IRQHandler`	10 kHz	0	dither period, step AHF table, set PWM compare values, timed relay coil switching
`DMA2_Stream0_IRQHandler`	10 Hz	1	mark window ready, 100 ms tick, clean-window flag
`ADC_IRQHandler`	on error	1	count overrun, request restart
`USART2_IRQHandler`	per byte	2	receive command characters
`DMA1_Stream6_IRQHandler`	2 Hz	3	telemetry transmission finished

Table 10.1: Interrupts and priorities.

TIM2 has the highest priority because it is the time base for the PWM output and the relay timing. Its timing must never be delayed by a lower-priority ISR. It is also kept very short (a few dozen instructions) so that it never delays anything else noticeably.

10.2 Rules for data shared with interrupts

- **`volatile`:** a variable written in an ISR and read in the main loop (or the other way round) must be declared `volatile`. That tells the compiler it may change at any time, so it must really be read from memory each time instead of being kept in a register.
- **Critical sections:** `__disable_irq()` / `__enable_irq()` temporarily block all interrupts. They are used when several related variables must be read or changed together, without an ISR running in between: lock masks, the PLL period, the pending relay masks, the window-ready flag.
- **Memory barrier `__DMB()`:** guarantees that all memory writes before it (a newly computed PWM table) are completed before a later write (the flag telling the ISR to use the new table).
- **Double buffering:** the AHF table exists twice. The ISR plays one while the main loop writes the other, and the ISR switches only at the start of a cycle. The output is never a mix of old and new values.

10.3 Sleeping

When there is no window to process, the main loop executes `__WFI()` (Wait For Interrupt). The core stops until the next interrupt arrives, at most 100 μs later because of TIM2. This saves power and makes the program's structure clear: everything is driven by interrupts.

11 Serial communication and protocol

11.1 UART basics

A UART sends each byte as a start bit, 8 data bits (least significant first) and a stop bit. At 115 200 bit/s one byte takes $10/115\,200 = 86.8\,\mu$s, so the link carries at most 11.5 KB/s. The Nucleo's on-board ST-LINK converts USART2 (PA2/PA3) to a USB virtual COM port; that is the `COM6`/`COM35` you select on the PC.

Baud rate register

With 16× oversampling, USARTDIV = $f_{\mathrm{PCLK1}}/(16 \times \mathrm{baud})$, stored as mantissa (12 bits) and fraction (4 bits), i.e. BRR = f/baud rounded.
BRR = $(32\,000\,000 + 57\,600)/115\,200 = 278 =$ `0x116` ⇒ mantissa 17, fraction 6/16 ⇒ USARTDIV = 17.375.
Actual baud = 32 MHz/(16 × 17.375) = 115 108 bit/s, error −0.08 %, well within UART tolerance (about ±2 %).

11.2 The text protocol

All messages are ASCII lines that start with `$`. The PC sends commands:

`$C,i,0,0`	capacitor step i (0, 1, 2) back to automatic
`$C,i,1,s`	lock step i manually ON (s=1) or OFF (s=0)
`$A,e`	AHF ACTIVE (e=1) or STANDBY (e=0)

Every 0.5 s the microcontroller sends one telemetry *frame* of these lines:

`$W,V / I / A / T / C`	100 points of one cycle: voltage, grid current, AHF current, AHF target, grid current after compensation
`$M`	V_{rms}, I_{rms}, P, S, Q_1, PF, THD_V, THD_I, crest, V_{pk}, I_{pk}, DPF, D, frequency
`$H,I / A / C`	rms of harmonics 1–15 of grid current, AHF current, compensated current
`$P`	relay state, capacitance ON, PF, Q_1, hold flag, target, C_{need}, PLL lock, hold reason, steps waiting for lock
`$A`	AHF active, AHF peak, target peak, duty peak, loop residual, THD after
`$K`	manual lock mask and state, commands OK / rejected, bytes received
`$R`	per channel ADC offset, min, max, clip count; overruns, dropped windows, skipped frames, current-sensor noise
`$E`	end of frame: the PC applies everything received since the last `$E` at once

Text is slower than binary, but it is easy to debug with any serial terminal, and it is robust: a corrupted line is simply ignored.

12 Signal processing: DFT, phasors and noise

12.1 The discrete Fourier transform used here

For the averaged cycle $x_0 \dots x_{199}$ and each harmonic $h = 1 \dots 15$:

$$a_h = \frac{2}{N}\sum_{k=0}^{N-1} x_k \cos\frac{2\pi hk}{N}, \qquad b_h = \frac{2}{N}\sum_{k=0}^{N-1} x_k \sin\frac{2\pi hk}{N}, \qquad N = 200.$$

Then $x_k \approx \sum_h (a_h \cos\frac{2\pi hk}{N} + b_h \sin\frac{2\pi hk}{N})$. The peak amplitude of harmonic h is $\sqrt{a_h^2 + b_h^2}$, and its rms value is that divided by $\sqrt{2}$. Only 15 harmonics are needed, so a direct DFT (15×200 multiply-adds per channel) is cheaper and simpler than an FFT that would compute all 100 bins. The cosine and sine values are precomputed once in tables (`cos_t`, `sin_t`). For harmonic h, the angle index advances by h each sample, wrapping at 200, so no trigonometric function is called at run time.

12.2 Phasors

A sinusoid $a\cos\theta + b\sin\theta$ is represented by the complex number (phasor) $X = a - jb$, because $\mathrm{Re}\{Xe^{j\theta}\} = a\cos\theta + b\sin\theta$. Phasors make phase shifts and gains simple. Multiplying by a complex number scales the amplitude and rotates the phase, and dividing undoes it. The firmware has a tiny complex-number library (`cpx_t`, `c_mul`, `c_div`, ...) for this.

12.3 Fundamental power from the DFT

For two sinusoids at the same frequency, $v = a_v\cos\theta + b_v\sin\theta$ and $i = a_i\cos\theta + b_i\sin\theta$:

$$P_1 = \overline{v\,i} = \tfrac{1}{2}(a_v a_i + b_v b_i), \qquad Q_1 = \tfrac{1}{2}(a_v b_i - b_v a_i).$$

Check the sign: with $v = \cos\theta$ and a lagging current $i = \cos(\theta - \varphi) = \cos\varphi\cos\theta + \sin\varphi\sin\theta$, you get $Q_1 = \frac{1}{2}\sin\varphi > 0$. A lagging (inductive) load has positive Q_1.

12.4 Separating real current from sensor noise

Let the load current be s (it repeats every cycle) and the sensor noise n (random, different every cycle). Over one window of $K = 5$ cycles:

$$\underbrace{\overline{x^2}}_{\text{raw samples}} = S^2 + N^2, \qquad \underbrace{\overline{\bar{x}^2}}_{\text{averaged cycle}} = S^2 + \frac{N^2}{K}.$$

Two equations, two unknowns:

$$\boxed{S^2 = \frac{K\,\overline{\bar{x}^2} - \overline{x^2}}{K-1}}, \qquad N^2 = \overline{x^2} - S^2.$$

The firmware reports $\sqrt{S^2}$ as I_{rms} and $\sqrt{N^2}$ as the sensor noise. No calibration is needed. With no load connected, the 31 mA of noise now reads 0 A; a 38 mA charger reads within 1 % (the raw rms would be 28 % too high). Active power is not affected by noise, because random noise does not correlate with the voltage.

12.5 Deadbands

Below a threshold the value is forced to zero: voltage below 20 V rms (with no mains the sensor noise reads about 6 V), current below 0.02 A (after noise removal), and AHF current below 0.05 A for display while the bridge is idle. When a value is zero, its waveform, THD and dependent quantities are also zeroed.

13 Digital filters

13.1 Butterworth low-pass

A Butterworth filter has the flattest possible pass-band (no ripple). A second-order Butterworth low-pass has $|H(f)|^2 = 1/(1 + (f/f_c)^4)$ and falls at 40 dB per decade above the cut-off f_c. The analog prototype is converted to a digital filter with the **bilinear transform**. Its frequency axis is *pre-warped* with $K = \tan(\pi f_c/f_s)$ so that the digital cut-off lands exactly at f_c:

$$n = \frac{1}{1 + \sqrt{2}K + K^2}, \quad b_0 = K^2 n,\ b_1 = 2b_0,\ b_2 = b_0, \quad a_1 = 2(K^2 - 1)n, \quad a_2 = (1 - \sqrt{2}K + K^2)n.$$

Coefficients in this firmware

$f_c = 1$ kHz (sensor A): $b = [0.06746,\ 0.13491,\ 0.06746]$, $a = [1,\ -1.14298,\ 0.41280]$.
$f_c = 1.5$ kHz (V/I display): $b = [0.13111,\ 0.26221,\ 0.13111]$, $a = [1,\ -0.74779,\ 0.27221]$.

The filter is computed in **direct form II transposed**, which needs two state variables z_1, z_2:

$$y = b_0 x + z_1, \qquad z_1 \leftarrow b_1 x - a_1 y + z_2, \qquad z_2 \leftarrow b_2 x - a_2 y.$$

13.2 Filtering without spoiling the measurement

A causal filter delays the signal (phase lag). At 750 Hz the 1 kHz filter shifts by $-66.5°$ and its gain is 0.878. The project uses two techniques so that this never corrupts any result:

1. **Exact compensation (sensor A).** Because sampling is locked to 200 samples per cycle, harmonic h is always at the normalised frequency $\Omega_h = 2\pi h/200$. The firmware computes the filter's exact complex response $H(e^{j\Omega_h})$ once at start-up and divides every measured AHF-current phasor by it. The control loop receives the true phasor, with noise removed outside the band (residual error in simulation: 0.0001 %).
2. **Zero-phase filtering (V and I display).** The averaged cycle is periodic, so the filter can be run *forward* and then *backward* around it. The phase shifts of the two passes cancel exactly, and the magnitude becomes $|H|^2$. This is applied only to the waveforms sent to the screen. Every number (rms, P, Q, PF, THD, harmonics) is computed from the unfiltered data first. A test that builds the code with and without this filter confirms that all measured values are identical.

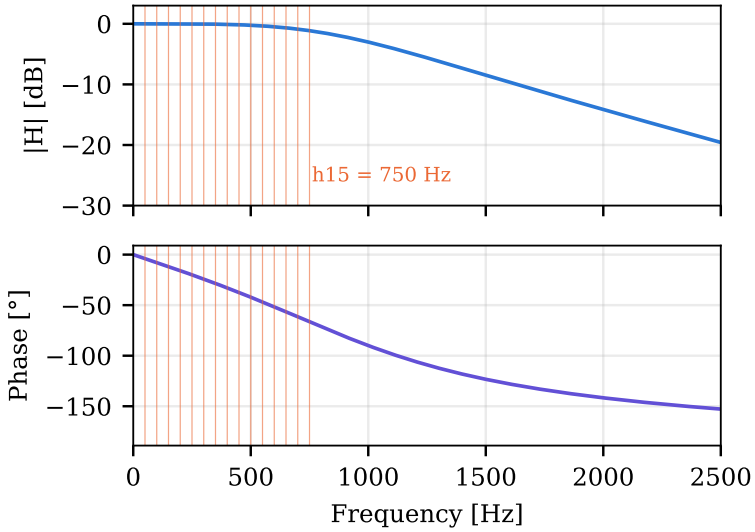


Figure 13.1: Response of the 1 kHz sensor-A Butterworth filter; vertical lines mark harmonics 1–15.

14 Software phase-locked loop (grid lock)

14.1 Why lock to the grid?

Coherent sampling (exactly 200 samples per cycle) gives leakage-free DFTs. It also makes the measured phasors *stationary*: the same harmonic has the same phase in every window. Two things depend on this. The AHF loop (its correction is computed in one window and applied over the next) would chase a rotating target if the grid were at 50.1 Hz and sampling at 50.000 Hz: the 15th harmonic would drift by about 54° per window. The point-on-wave relay timing needs to know which sample is the voltage peak.

14.2 How the lock works

1. From the DFT, the voltage fundamental phasor gives its phase at the start of the window: $\varphi = \operatorname{atan2}(-b_1, a_1)$.
2. The change of this phase from one window to the next tells how far the grid ran ahead of or behind the commanded frequency over 5 cycles:

$$\hat{f}_{\text{grid}} = f_{\text{cmd}}\Big(1 + \frac{\operatorname{wrap}(\varphi_n - \varphi_{n-1})}{2\pi \cdot 5}\Big).$$

3. The new commanded frequency is the estimate plus a phase correction,

$$f_{\text{cmd}} = \hat{f}_{\text{grid}} + K_p \cdot \operatorname{wrap}(\varphi_n - \varphi_{\text{set}}), \qquad K_p = 0.4 \text{ Hz/rad},$$

   limited to 47.5–52.5 Hz. This drives the voltage **positive peak to sample** $k = 0$ of every cycle ($\varphi_{\text{set}} = 0$).
4. The sample period is set to 64 MHz/(200 f_{cmd}) ticks with fractional dithering (Chapter 8). The PWM period becomes 1/8 of it.
5. The grid is reported as *locked* when the phase error is below 0.35 rad (about 1.1 ms).

Loop dynamics

A phase error φ produces a frequency offset $K_p\varphi$, which over one window (5 cycles) reduces the phase error by $2\pi \cdot 5 \cdot K_p\varphi/f = 31.4 \times 0.4/50 \approx 0.25\varphi$. Each window removes about a quarter of the error, so lock is reached in about 1–2 s. In simulation it locks to 49.83, 50.17 and 51.20 Hz with less than 0.005 Hz error.

15 Capacitor bank control (APFC)

15.1 Decision algorithm (every 0.5 s)

1. If all three steps are manually locked, simply apply the locks.
2. If $V \geq 10\,\mathrm{V}$ and $|P_1| \geq 0.5\,\mathrm{W}$, compute

$$k = 2\pi f V^2 \cdot 10^{-6}, \qquad Q_{\mathrm{load}} = Q_1 + k\,C_{\mathrm{on}},$$
$$Q_{\mathrm{tgt}} = |P_1| \tan(\arccos 0.99), \qquad C_{\mathrm{need}} = \max\Big(0, \frac{Q_{\mathrm{load}} - Q_{\mathrm{tgt}}}{k}\Big).$$

   The term $k\,C_{\mathrm{on}}$ adds back the capacitors already ON. Without it, switching a capacitor in would lower the measured Q_1, and the controller would switch it straight out again.
3. Try all $2^3 = 8$ combinations allowed by the manual locks. Choose the one whose total capacitance is closest to C_{need}; if two are equally close, choose the one needing fewer relay operations.
4. Switch only if the present combination is off by more than $0.75\,\mu\mathrm{F}$ *and* the best one is better (hysteresis). Otherwise hold, and report why.

Two examples

Fan ($50\,\mathrm{W}$, $Q = 51\,\mathrm{var}$, $230\,\mathrm{V}$): $Q_{\mathrm{tgt}} = 50 \times 0.1425 = 7.1\,\mathrm{var}$; $C_{\mathrm{need}} = (51 - 7.1)/16.62 = 2.64\,\mu\mathrm{F}$. Best combination: $2.5\,\mu\mathrm{F}$ (error 0.14). Present error $2.64 > 0.75$, so it **switches the 2.5 µF step ON**.

Your laptop ($246\,\mathrm{V}$, $P = 97.5\,\mathrm{W}$, $Q_1 = 15.3\,\mathrm{var}$): $k = 2\pi \cdot 50 \cdot 246^2 \cdot 10^{-6} = 19.0$; $Q_{\mathrm{tgt}} = 97.5 \times 0.1425 = 13.9$; $C_{\mathrm{need}} = (15.3 - 13.9)/19.0 = 0.07\,\mu\mathrm{F}$, exactly what your dashboard showed. The smallest step is $2.5\,\mu\mathrm{F}$, so it **holds**, reporting "load below half the smallest step".

15.2 Relay safety: why the timing matters

A capacitor that has been switched off keeps its charge, because there is no bleeder resistor. If a relay closes when the mains voltage differs from the capacitor voltage, a huge current spike (hundreds of amps for microseconds) flows through the contacts and can weld them. You observed exactly this on the $10\,\mu\mathrm{F}$ relay. The firmware applies four rules:

1. **Minimum ON and OFF time** of 5 s each, so a capacitor closes at most once every 10 s.
2. **Point-on-wave opening:** the coil is released so the contacts part 2 ms before the positive voltage peak. A capacitor's current is near zero there, so the capacitor is left charged to about $+V_{pk}$.
3. **Point-on-wave closing:** the coil is energised 7 ms (the relay's operate time) before the positive peak. The contacts close when the mains voltage equals the capacitor voltage, so almost no current spike flows. If the charge is unknown (after reset), it closes at the zero crossing instead, which limits the voltage across the contacts to V_{pk}.
4. **Lock fallback:** if the grid lock is missing for more than 2 s, it closes anyway, so compensation is never blocked forever.

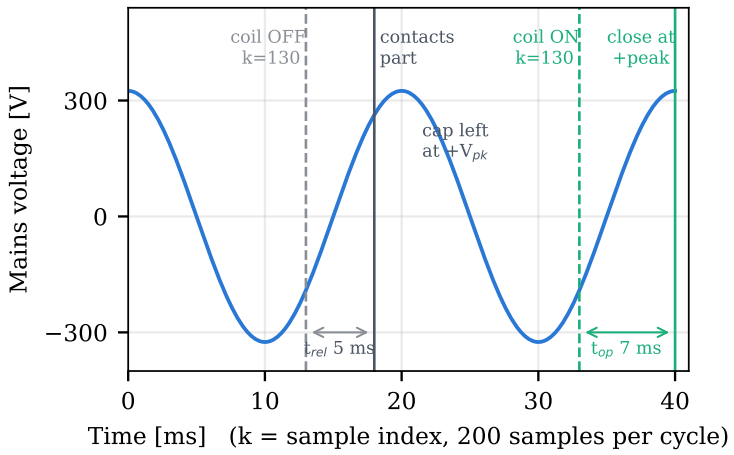


Figure 15.1: Point-on-wave timing: coil OFF at sample 130 so the contacts part 2 ms before the peak; coil ON at sample 130 of the next cycle so the contacts close at the peak.

Sample indices of the relay timing

Peak at $k = 0$ (PLL). One sample = 0.1 ms.
Close at peak: coil ON at $k = (0 - 70) \bmod 200 = \mathbf{130}$ (70 samples = 7 ms before).
Close at zero crossing: rising zero crossing at $k = 150$; coil ON at $150 - 70 = \mathbf{80}$.
Open: contacts part at $k = -20$ (2 ms before peak); coil OFF at $-20 - 50 = -70 \rightarrow \mathbf{130}$.

In simulation, an operator clicking ON/OFF every second for 2 minutes caused 30 closings at up to 650 V across the contacts with the old firmware, and 12 closings at about 0–60 V with this firmware. The hardware fix for welding is still recommended: an NTC inrush limiter in series and a bleeder resistor across each capacitor.

16 Active harmonic filter control

16.1 The problem with a simple copy

An early idea was to take the distortion waveform $-(i - i_1)$, scale it to 92 % duty, and play it on the PWM. This fails for three reasons. The bridge produces a *voltage*, and the current through the coupling inductance is that voltage divided by $jh\omega L$: the 15th harmonic comes out 5× weaker than the 3rd, and every harmonic lags by nearly 90°. The amplitude was fixed, regardless of the current actually required. And nothing was measured, so errors could never be corrected. In simulation that method *increased* THD from 27.9 % to 61.5 %.

16.2 Selective harmonic closed loop

Each harmonic $h = 2 \ldots 15$ gets its own control loop, working on phasors once per clean 100 ms window:

$R_h = -I_h$	required AHF current (cancel the load harmonic)
$E_h = R_h - M_h$	error (M_h = measured AHF current, sensor A)
$\hat{G}_h \leftarrow \hat{G}_h + 0.25(M_h/U_h - \hat{G}_h)$	identify the plant gain (amperes per unit duty)
$U_h \leftarrow 0.998\,(U_h + 0.5\,E_h/\hat{G}_h)$	integrate the duty phasor

$\hat{G}_h$ is a complex number. Its magnitude is how many amperes one unit of duty produces at that harmonic, and its angle is the phase lag of the bridge, inductor and PWM delay. It starts from a rough model ($V_{dc}/(R + jh\omega L)$ with a delay). The *first* real measurement replaces the model completely, and later measurements refine it. Dividing the error by $\hat{G}_h$ converts "amperes missing" into "duty to add", with the right amplitude and phase for each harmonic. The factor 0.5 makes the error roughly halve every update, and 0.998 slowly forgets harmonics that are no longer needed.

16.3 From phasors to PWM

$$u_k = \sum_{h=2}^{15} \mathrm{Re}\{U_h e^{j2\pi hk/200}\}, \qquad k = 0 \ldots 199.$$

If the peak of u would exceed 0.92 (92 % duty), all U_h and u are scaled down together. This is **anti-windup**: the integrators do not keep growing while the output is saturated. The values are stored as Q15 integers ($u \times 32767$) in one of two tables. The TIM2 interrupt reads one value per sample and converts it to compare values for TIM4. The tables are switched only at sample 0 of a cycle. A window during which a switch occurred is marked *not clean* and skipped by the identification, because sensor A then shows a mix of two different tables.

16.4 Measured results

On the hardware, with the AHF active: laptop charger THD 35.9 % → 10.5 %, a 0.69 A load 15.9 % → 3.2 %, a 0.2 A load 33.7 % → 14.4 %. The bridge used only 7–12 % of its duty range in all three cases. The "after" values are calculated from sensor I plus sensor A, so they are an *estimate* of the grid current; an independent grid-side measurement is the next verification step.

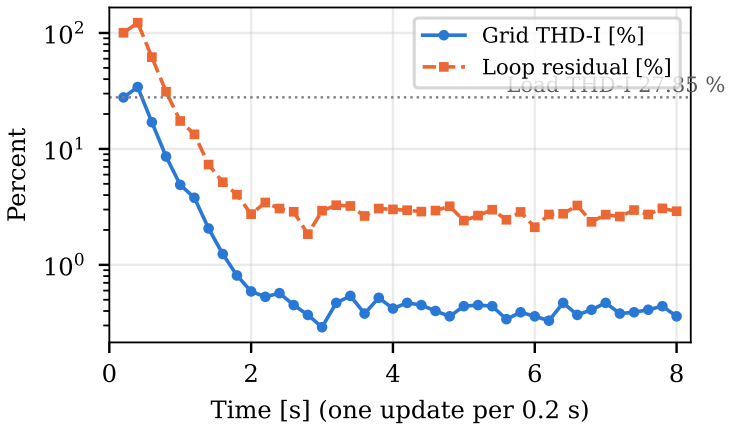


Figure 16.1: Simulated convergence with a deliberately wrong (180°) initial model: THD from 27.85 % to 0.36 %. After the first measurement of each $\hat{G}_h$ the loop converges within about 2 s.

17 Execution timeline from reset

1. **Reset.** The start-up code sets up the stack and memory and calls `main()`.
2. **FPU on** (L1462).
3. **`clock_init`:** voltage scale 2, 2 Flash wait states, HSI on, PLL configured and started, system switched to 64 MHz.
4. **`gpio_init`:** relays forced OFF first, then all pins configured.
5. **`tables_init`:** cosine/sine tables, $\tan\varphi$ target, AHF model, filter coefficients and filter response.
6. **`uart_init`, `tim2_init`, `adc_dma_init`, `tim4_init`, `nvic_init`:** peripherals configured, interrupts enabled.
7. **`sampling_sync_start`:** DMA armed, ADC on, TIM4 started, TIM2 started. From now on samples flow automatically.
8. **Every 100 μs** (TIM2 ISR): ADC scan (hardware), DMA store (hardware), period dither, AHF duty output, scheduled relay coils.
9. **Every 100 ms** (DMA ISR, then main loop): copy window, measurements, DFT, PLL, AHF control step.
10. **Every 0.5 s:** averaged measurements, APFC decision, telemetry frame, LED toggle.
11. **Continuously** (main loop, between windows): `relay_service()` applies relay changes and manual commands within microseconds. When idle, the core sleeps with `WFI`.

Part II

Line-by-line firmware walkthrough

18 File header, includes and constants

18.1 Header comment and includes (L1–58)

```
/* ============================================================================
 *  main.c  -  Dual-Stage Grid Power Quality Improvement System
 *             Stage 1: APFC with open-drain drive of 5 V active-LOW relays
 *             Stage 2: Active Harmonic Filter (TB6612FNG, 80 kHz PWM)
 *  Target   : STM32F401RE (NUCLEO-F401RE), pure CMSIS, zero HAL libraries
 *
 *  RELAY WIRING (5 V active-LOW modules on PB0 / PB1, SSR on PB2;
 *  any pin can be switched to an SSR with RELAY_SSR_MASK)
 *    Module VCC -> Nucleo 5V, GND -> GND, IN -> PB0 / PB1 directly.
 *    PB0/PB1 are OPEN-DRAIN: the pin only ever sinks current (relay ON) or
 *    floats (relay OFF). When it floats, the module's own resistor pulls IN
 *    up to 5 V, so the opto LED gets 0 V across it and the relay really
 *    releases. PB0/PB1 must be 5 V tolerant (FT) for this; do NOT enable the
 *    internal pull-up (it would clamp IN to 3.3 V again).
 *
 *  AHF: SELECTIVE HARMONIC CONTROL (this revision)
 *    Every 100 ms window the DFT gives one phasor per harmonic h = 2..15 for
 *      I  - the current to be cleaned (ACS712 #1)
 *      A  - the current the AHF actually injects (ACS712 #2)
 *    Each harmonic has its own closed loop:
 *      required  R_h = -I_h                (cancel that harmonic)
 *      error     E_h = R_h - A_h
 *      duty      U_h += AHF_MU * E_h / G_h
 *    G_h (amps per unit duty, complex) is the bridge + coupling-inductor gain
 *    at that harmonic. It starts from a rough model (AHF_VDC, AHF_L_H,
 *    AHF_R_OHM) and is then measured on line as A_h / U_h, so the inductor's
 *    1/h attenuation, its ~90 deg lag and the PWM delay are corrected per
 *    harmonic instead of being guessed. The output table is synthesised from
 *    the U_h phasors only, so noise and content above h15 are not amplified.
 *
 *    The sampling clock (TIM2) is phase-locked to the grid voltage, so one
 *    cycle is always exactly 200 samples. Without this, a 50.1 Hz grid makes
 *    h15 drift ~54 deg per window against a fixed 50.000 Hz table.
 *    TIM4 (PWM) is reset by every TIM2 update, so the 8 PWM periods per sample
 *    stay aligned with the ADC sampling instant and the PWM ripple cannot alias
 *    into the measured harmonics.
 *
 *    A_SIGN must make sensor A read +ve when the AHF pushes current in the
 *    same direction sensor I reads +ve. If grid THD rises when the AHF is
 *    ACTIVE, flip A_SIGN.
 *
 *  Telemetry changes (backwards compatible, fields appended / reused)
 *    $M  ... ,freq                 measured grid frequency [Hz]
 *    $A  active,apk,ref_pk,duty_pk,resid_pct,thd_after
 *    $H,C                          after-compensation harmonics h1..h15
 *    $R  ... ,i_noise              grid-current sensor noise [A rms]
 *    $P  ... ,lock,reason,wait_lock      grid lock, why the bank holds,
 *                                        caps waiting for the lock
 *    $W,C                          I + A (measured), was an ideal sine
 * ============================================================================ */

#include "stm32f401xe.h"
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <math.h>

#pragma GCC optimize ("O2")
```

- L1–50: the header comment documents the two stages, the target board, the relay wiring rules, the AHF control idea and the telemetry fields. Read it once: it is a summary of the whole design.
- L52 `stm32f401xe.h`: the CMSIS device header. It defines every peripheral register structure (`RCC`, `GPIOA`, `TIM2`, ...), every bit mask and interrupt number, and pulls in the Cortex-M4 core header (`NVIC_SetPriority`, `__WFI`, ...).
- L53–56: standard C headers. `stdint.h` provides exact-width integers (`uint32_t`); `stdbool.h` provides `bool`; `string.h` provides `memset` and `memcpy`; `math.h` provides `sqrtf`, `cosf`, `atan2f`. The "f" versions work in single precision, which the FPU executes in hardware.
- L58: asks GCC to optimise this file at level `-O2` even if the project is built for debugging, so the signal processing runs fast enough.

18.2 Clock and sampling constants (L60–89)

```
60 /* ------------------------------------------------------------------------ */
61 /*  Clocks & Sampling Constants                                             */
62 /* ------------------------------------------------------------------------ */
63 #define SYSCLK_HZ          64000000u
64 #define APB1_TIM_HZ        64000000u
65 #define APB1_HZ            32000000u
66 #define ADC_CLK_HZ         16000000u
67
68 #define FS_HZ              10000u                    /* nominal          */
69 #define F_GRID_HZ          50.0f
70 #define TWO_PI             6.28318531f
71 #define N_WIN              1000u
72 #define N_CYC              200u
73 #define CYC_PER_WIN        (N_WIN / N_CYC)
74 #define N_CH               3u
75 #define DMA_LEN            (2u * N_WIN * N_CH)
76 #define MAX_H              15u
77 #define WIN_MS             (1000u * N_WIN / FS_HZ)       /* 100 ms per window */
78
79 #define SLOT_I             0u
80 #define SLOT_V             1u
81 #define SLOT_A             2u
82
83 #define TIM2_N_NOM         (APB1_TIM_HZ / FS_HZ)          /* 6400 ticks       */
84
85 #define ADC_SMP_CYCLES     480u
86 #define ADC_SMP_CODE       7u
87 #define ADC_SLOT_S         ((float)(ADC_SMP_CYCLES + 12u) / (float)ADC_CLK_HZ)
88 #define ADC_CLIP_LO        8u
89 #define ADC_CLIP_HI        4087u
```

Name	Value	Meaning
`SYSCLK_HZ`	64 000 000	core clock set by `clock_init`
`APB1_TIM_HZ`	64 000 000	clock of TIM2/TIM4 (APB1 32 MHz doubled for timers)
`APB1_HZ`	32 000 000	clock of USART2, used for the baud-rate divider
`ADC_CLK_HZ`	16 000 000	ADC clock (APB2 64 MHz / 4)
`FS_HZ`	10 000	nominal sample rate (the PLL adjusts it)
`F_GRID_HZ`	50.0	nominal grid frequency
`TWO_PI`	6.28318531	2π in float

N_WIN	1000	samples per channel per window (100 ms)
N_CYC	200	samples per grid cycle
CYC_PER_WIN	5	cycles per window (1000/200)
N_CH	3	channels per scan (I, V, A)
DMA_LEN	6000	DMA buffer length: 2 halves × 1000 × 3
MAX_H	15	highest harmonic computed
WIN_MS	100	duration of one window in ms
SLOT_I/V/A	0/1/2	position of each channel inside one scan (order of conversion)
TIM2_N_NOM	6400	timer ticks per sample at 10 kHz
ADC_SMP_CYCLES	480	ADC sampling time in ADC clocks
ADC_SMP_CODE	7	register code for 480 cycles
ADC_SLOT_S	30.75 μs	time per conversion (492/16 MHz): channel-to-channel skew
ADC_CLIP_LO/HI	8 / 4087	readings this close to 0 or 4095 count as clipped

18.3 Grid lock constants (L91–96)

```
91 /* Grid lock (TIM2 sample clock follows the grid) */
92 #define PLL_F_MIN          47.5f
93 #define PLL_F_MAX          52.5f
94 #define PLL_KP_HZ          0.4f       /* Hz per rad of phase error          */
95 #define PLL_F_ALPHA        0.3f       /* smoothing of the reported frequency */
96 #define PLL_PHASE_SET      0.0f       /* V1 phase held at the window start   */
```

The PLL may command 47.5–52.5 Hz (L92–93). $K_p = 0.4$ Hz per radian of phase error (L94). PLL_F_ALPHA is the smoothing of the frequency shown on the dashboard (L95): 30 % new value, 70 % old. PLL_PHASE_SET = 0 (L96) means the voltage fundamental is held with zero phase at the window start, i.e. the *positive voltage peak sits at sample 0*. The relay timing depends on this.

18.4 Sensor calibration and deadbands (L98–130)

```
98  /* Sensor Calibration */
99  #define ADC_V_PER_LSB      (3.3f / 4095.0f)
100 #define ACS712_V_PER_A     0.185f     /* 5 A module; 20 A = 0.100, 30 A = 0.066 */
101 /* Resistor divider between an ACS712 OUT pin and its ADC pin:
102  * ratio = R_bottom / (R_top + R_bottom), 1.0f = wired directly.
103  * The ACS712 sits at 2.5 V for 0 A, but the ADC reads only up to 3.3 V, so
104  * without a divider positive current clips at about +4.3 A.             */
105 #define ACS_DIV_I          0.6667f    /* sensor I (PA0): 10k series, 20k to GND */
106 #define ACS_DIV_A          1.0f       /* sensor A (PA4, AHF): direct            */
107 #define ZMPT_V_PER_V       1014.0f
108 #define I_SIGN             1.0f
109 #define V_SIGN             1.0f
110 #define A_SIGN             1.0f
111 #define ZMPT_LAG_DEG       0.0f
112
113 #define V_DEADBAND         20.0f      /* below this the mains is treated as OFF:
114                                         * ZMPT101B + ADC noise with no mains is
115                                         * ~6 V rms, a real 230 V grid never this low */
116 #define I_DEADBAND         0.02f      /* grid current (noise already removed,
117                                         * see process_window) below this = 0  */
118 #define A_DEADBAND         0.05f      /* sensor A rms shown as 0 below this,
119                                         * only while the bridge is not driving */
120 #define VI_LPF_HZ          1500.0f    /* zero-phase Butterworth smoothing of the
121                                         * V and I waveforms sent for display;
```

```
                                     * measurements use the raw samples      */
#define A_LPF_HZ          1000.0f    /* 2nd-order Butterworth low-pass on
                                     * sensor A (h15 = 750 Hz passes; its
                                     * gain/phase is divided back out per
                                     * harmonic, so the AHF loop is exact)  */

#define K_I   (I_SIGN * ADC_V_PER_LSB / (ACS712_V_PER_A * ACS_DIV_I))
#define K_A   (A_SIGN * ADC_V_PER_LSB / (ACS712_V_PER_A * ACS_DIV_A))
#define K_V   (V_SIGN * ADC_V_PER_LSB * ZMPT_V_PER_V)
```

- L99: volts per ADC count, $3.3/4095 = 0.806\,\text{mV}$.
- L100: ACS712 sensitivity of the 5 A version, 185 mV/A.
- L105–106: divider ratios, 0.6667 for sensor I (10 k/20 k), 1.0 for sensor A (direct).
- L107: ZMPT101B calibration, mains volts per ADC volt.
- L108–110: sign corrections. Set one to `-1.0f` if that sensor is wired the other way round. For sensor A this matters: if grid THD rises when the AHF is ACTIVE, flip `A_SIGN`.
- L111: optional phase-lag correction of the ZMPT101B in degrees (0 = none).
- L113–119: deadbands. Voltage below 20 V rms means mains OFF; current below 0.02 A (after noise removal) means no load; AHF current below 0.05 A is shown as zero while the bridge is idle.
- L120–126: cut-off frequencies of the two Butterworth filters (1.5 kHz display smoothing, 1 kHz for sensor A).
- L128–130: the scaling constants K_I, K_A, K_V (Table 3.1). They are macros, so the compiler evaluates them at compile time.

18.5 APFC and AHF constants (L132–181)

```
/* APFC Constants */
#define PF_TARGET         0.99f      /* switch whenever a step improves PF   */
#define C_DEADBAND_UF     0.75f
#define CAP_RECLOSE_MS    5000u      /* min OFF time before a cap re-closes.
                                     * Without bleeder resistors the cap is
                                     * still charged after this; the
                                     * point-on-wave closing (RELAY_SYNC)
                                     * is what protects the contacts.       */
#define APFC_V_MIN        10.0f
#define APFC_P_MIN        0.5f
#define PFC_SETTLE_WIN    2u
#define TELEM_WIN         5u

/* Set to 1 to click R1, R2, R3 ON/OFF once at boot (bench test, mains off) */
#define RELAY_SELFTEST    0u

/* AHF Constants */
#define PWM_PER_SAMPLE    8u
#define PWM_PERIOD_NOM    (TIM2_N_NOM / PWM_PER_SAMPLE)  /* 800 -> 80 kHz    */
#define AHF_MOD_DEPTH     0.92f      /* max |duty| of the synthesised table  */
#define AHF_MIN_HARM_A    0.02f      /* below this target peak, do not drive */
#define AHF_PK_ALPHA      0.3f
#define AHF_BOOT_ACTIVE   1u

#define AHF_HARM_MASK     0xFFFCu    /* bit h set = cancel harmonic h (2..15) */
#define AHF_SENSE_GRID    0u         /* 0: sensor I sees the load current
                                     *    (upstream of the injection point)
                                     * 1: sensor I sees the grid current
```

```
160                                      *    (downstream, already compensated) */
161 #define AHF_INJ_RATIO      1.0f       /* PCC amps per sensor-A amp (coupling
162                                       * transformer ratio, 1 if direct)      */
163 #define AHF_VDC            12.0f      /* initial plant model only: TB6612 VM  */
164 #define AHF_L_H            0.010f     /*   coupling inductance [H]            */
165 #define AHF_R_OHM          1.0f       /*   coupling resistance [ohm]          */
166 #define AHF_DELAY_S        (1.5f / (float)FS_HZ)          /* PWM + update     */
167 #define AHF_MU             0.5f       /* loop gain per update, 0 < MU <= 1    */
168 #define AHF_MU_PROBE       0.15f      /* gain until G_h has been measured     */
169 #define AHF_ID_RATE        0.25f      /* plant estimate update rate           */
170 #define AHF_ID_MIN_DUTY    0.01f      /* identify G_h only when |U_h| > this  */
171 #define AHF_LEAK           0.002f     /* slow forgetting of unused harmonics  */
172
173 #define UART_BAUD          115200u
174 #define WAVE_DECIM         2u
175 #define TX_BUF_SZ          5632u
176 #define RX_LINE_MAX        32u
177
178 _Static_assert(APB1_TIM_HZ / (N_CYC * 50u) / PWM_PER_SAMPLE <= 65535u, "TIM4 ARR is 16 bit");
179 _Static_assert(APB1_TIM_HZ / PWM_PERIOD_NOM <= 100000u, "TB6612FNG max PWM is 100 kHz");
180 _Static_assert((uint64_t)(ADC_SMP_CYCLES + 12u) * N_CH * N_CYC * 53u < ADC_CLK_HZ,
181                "ADC scan does not fit in one sample period at PLL_F_MAX");
```

- L133–134: target PF 0.99 and the 0.75 μF switching deadband.
- L135: `CAP_RECLOSE_MS`, the minimum OFF time (5 s).
- L140–141: the APFC only runs with at least 10 V and 0.5 W.
- L142: after any relay change, the next 2 windows (0.2 s) are ignored while the transient settles.
- L143: the APFC decision and telemetry happen every 5 windows (0.5 s).
- L146: set to 1 to click each relay once at boot (bench test with the mains disconnected).
- L149–150: 8 PWM periods per sample, so the PWM period is $6400/8 = 800$ ticks (80 kHz).
- L151: the synthesised duty never exceeds 92 %, which keeps a margin in the H-bridge.
- L152: if the distortion target is below 0.02 A peak, the AHF does not drive.
- L153: smoothing factor of the target peak value.
- L154: the AHF starts ACTIVE at power-up.
- L156: bit h set means harmonic h is cancelled. `0xFFFC` sets bits 2–15, so all harmonics from 2 to 15 are cancelled.
- L157–160: where sensor I is. 0 means it measures the load current before the injection point (the default); 1 means it measures the already-compensated grid current.
- L161: coupling ratio if a transformer is used (1 for direct injection).
- L163–166: the *initial* plant model (bridge supply 12 V, 10 mH, 1 Ω, 1.5-sample delay). It is only a starting guess and is replaced by the first measurement.
- L167–171: AHF loop gains. $\mu = 0.5$ normally, 0.15 until the plant has been measured; identification rate 0.25; identification only when the duty phasor exceeds 1 %; leak 0.2 % per update.
- L173–176: UART baud rate; waveforms are sent with every 2nd sample (100 points per cycle); 5632-byte transmit buffer; 32-byte maximum command line.
- L178–181: **compile-time checks** (`_Static_assert`). If a constant is ever changed so that the TIM4 period would not fit in 16 bits, the PWM would exceed the TB6612's 100 kHz, or the 3-channel ADC scan would no longer fit in one sample period at 53 Hz, the code refuses to compile, with that message.

18.6 Relay constants and point-on-wave indices (L183–221)

```
183 /* ------------------------------------------------------------------------ */
184 /*  Relay pins                                                              */
185 /* ------------------------------------------------------------------------ */
186 /* PB0 = 2.5 uF, PB1 = 10 uF, PB2 = 20 uF. Each output is one of:
187  *   relay module : 5 V active-LOW, open-drain. ON = sink LOW, OFF = float.
188  *   SSR          : active-HIGH, push-pull.     ON = HIGH,     OFF = LOW.
189  * Bit i of RELAY_SSR_MASK set = PB<i> drives an SSR. Zero-cross SSRs are
190  * strongly recommended for capacitors: they close near 0 V, so a discharged
191  * cap draws almost no inrush (a mechanical relay closing near the 325 V peak
192  * sees hundreds of amps and its contacts weld).                            */
193 #define RELAY_SSR_MASK     0x4u       /* PB2 = SSR, PB0/PB1 = relay modules   */
194
195 /* Switching rules for the capacitor steps (AUTO and manual alike)          */
196 #define RELAY_MIN_ON_MS    5000u      /* a closed cap stays ON at least this  */
197 #define RELAY_LOCK_WAIT_MS 2000u      /* no grid lock after this: close anyway */
198 /*  CAP_RECLOSE_MS (above) is the matching minimum OFF time.                */
199
200 /* Point-on-wave switching for relay modules (not SSRs). With no bleeder the
201  * cap keeps the voltage it was disconnected at.
202  *  Opening: the contacts part shortly before the +peak. A cap's current is
203  *    ~0 there, and if an arc does form it runs on to the current zero, which
204  *    is that same +peak - either way the cap is left near +Vpk.
205  *  Closing: at the +peak, so the contacts see ~0 V. A cap whose charge is
206  *    unknown (never opened by this firmware since boot) is closed at the
207  *    rising zero crossing instead: at most Vpk across the contacts whether
208  *    it is empty or charged either way (closing at the peak could be 2 Vpk).
209  * The PLL holds the voltage peak at sample k = 0 of every cycle.           */
210 #define RELAY_SYNC         1u         /* 0 = switch immediately (old way)     */
211 #define RELAY_T_OP_MS      7.0f       /* coil ON  -> contacts closed (typ 5-9)*/
212 #define RELAY_T_REL_MS     5.0f       /* coil OFF -> contacts open   (typ 2-8)*/
213 #define RELAY_OPEN_LEAD_MS 2.0f       /* contacts part this long before +peak */
214 #define RELAY_MS_TO_K(ms)  ((uint32_t)((ms) * (float)FS_HZ / 1000.0f + 0.5f))
215 #define RELAY_K_PEAK       ((uint32_t)((float)N_CYC - ZMPT_LAG_DEG / 360.0f * (float)N_CYC) %
        N_CYC)
216 #define RELAY_K_ON         ((RELAY_K_PEAK + 2u * N_CYC - RELAY_MS_TO_K(RELAY_T_OP_MS)) % N_CYC)
217 #define RELAY_K_ON_ZERO    ((RELAY_K_PEAK + 2u * N_CYC - N_CYC / 4u - \
218                              RELAY_MS_TO_K(RELAY_T_OP_MS)) % N_CYC)
219 #define RELAY_K_OFF        ((RELAY_K_PEAK + 2u * N_CYC - RELAY_MS_TO_K(RELAY_OPEN_LEAD_MS) - \
220                              RELAY_MS_TO_K(RELAY_T_REL_MS)) % N_CYC)
```

- L193 `RELAY_SSR_MASK = 0x4`: bit i set means pin PBi drives an SSR (active-HIGH, push-pull). Clear means a relay module (active-LOW, open-drain). Here PB2 is the SSR, PB0 and PB1 are relay modules.
- L196–197: minimum ON time 5 s; if the grid lock is missing, a capacitor still closes after 2 s.
- L210: `RELAY_SYNC = 1` enables point-on-wave timing.
- L211–213: relay operate time 7 ms, release time 5 ms, and how early before the peak the contacts should part (2 ms).
- L214: converts milliseconds to samples, rounded: 7 ms → 70.
- L215: sample index of the voltage peak, 0 (shifted only if a ZMPT lag is configured).
- L216–220: the coil-switching sample indices worked out in Chapter 15: `RELAY_K_ON` = 130, `RELAY_K_ON_ZERO` = 80, `RELAY_K_OFF` = 130. The `+2u*N_CYC` keeps the subtraction positive before taking the remainder modulo 200.

19 The DSP core

The block between DSP-CORE-BEGIN and DSP-CORE-END (L226–493) touches no hardware register. That is deliberate: a test script copies this block into a PC program, so the mathematics can be tested on a computer with simulated signals.

19.1 Complex-number helpers (L222–251)

```
222 /* ========================================================================== */
223 /*  DSP core - no register access (built and simulated on a PC by           */
224 /*  firmware/test/run_sim.sh, which copies everything between the markers)   */
225 /* ========================================================================== */
226 /* DSP-CORE-BEGIN */
227 typedef struct { float re, im; } cpx_t;
228
229 static inline cpx_t c_mk(float re, float im)   { cpx_t z = { re, im }; return z; }
230 static inline cpx_t c_add(cpx_t a, cpx_t b)    { return c_mk(a.re + b.re, a.im + b.im); }
231 static inline cpx_t c_sub(cpx_t a, cpx_t b)    { return c_mk(a.re - b.re, a.im - b.im); }
232 static inline cpx_t c_scale(cpx_t a, float k)  { return c_mk(a.re * k, a.im * k); }
233 static inline float c_abs2(cpx_t a)            { return a.re * a.re + a.im * a.im; }
234 static inline cpx_t c_mul(cpx_t a, cpx_t b)
235 {
236     return c_mk(a.re * b.re - a.im * b.im, a.re * b.im + a.im * b.re);
237 }
238 static inline cpx_t c_div(cpx_t a, cpx_t b)
239 {
240     const float d = c_abs2(b);
241     return c_mk((a.re * b.re + a.im * b.im) / d, (a.im * b.re - a.re * b.im) / d);
242 }
243 /* dft() gives x = a cos(h th) + b sin(h th); as a phasor that is a - jb.     */
244 static inline cpx_t c_phasor(float a, float b) { return c_mk(a, -b); }
245
246 static float wrap_pi(float x)
247 {
248     while (x >  3.14159265f) x -= TWO_PI;
249     while (x < -3.14159265f) x += TWO_PI;
250     return x;
251 }
```

- L227: a complex number is a pair of floats.
- L229–233: make, add, subtract, scale by a real number, and squared magnitude $|z|^2 = \mathrm{re}^2 + \mathrm{im}^2$.
- L234–237: multiplication $(a + jb)(c + jd) = (ac - bd) + j(ad + bc)$.
- L238–242: division $\dfrac{a}{b} = \dfrac{a\,\bar{b}}{|b|^2}$.
- L244: converts the DFT pair (a, b) into the phasor $a - jb$ (the convention of Chapter 12).
- L246–251: brings an angle into $(-\pi, \pi]$. This is needed when phases are subtracted (PLL), so that $+179°$ minus $-179°$ is seen as $-2°$, not $358°$.

19.2 Tables and state of the DSP core (L253–272)

```
253 static float cos_t[N_CYC], sin_t[N_CYC];
254 static float tan_phi_tgt;
```

```
255
256 typedef struct { float b0, b1, b2, a1, a2; } biquad_t;
257 static biquad_t a_lpf;                  /* sensor A low-pass                  */
258 static biquad_t vi_lpf;                 /* V/I display smoothing              */
259 static cpx_t    a_lpf_h[MAX_H + 1];     /* its response at each harmonic      */
260
261 static int16_t ahf_tab[2][N_CYC];       /* Q15 duty, one cycle, played by ISR */
262 static cpx_t   ahf_tab_u[2][MAX_H + 1]; /* duty phasors that built each table */
263 static cpx_t   ahf_u[MAX_H + 1];        /* duty phasor per harmonic (state)   */
264 static cpx_t   ahf_g[MAX_H + 1];        /* plant gain, amps per unit duty     */
265 static uint8_t ahf_g_seen[MAX_H + 1];   /* G_h has been measured at least once*/
266 static float   ahf_resid_pct, ahf_duty_pk;
267
268 static float    pll_f_cmd = F_GRID_HZ;  /* sample clock = N_CYC * pll_f_cmd   */
269 static float    pll_f_est = F_GRID_HZ;  /* measured grid frequency            */
270 static float    pll_ph_prev;
271 static uint32_t pll_have_prev;
272 static uint32_t pll_locked;             /* window start sits on the V peak    */
```

- L253: one cycle of cosine and sine values (200 each), filled at start-up and used by every DFT and synthesis.
- L254: $\tan(\arccos \mathrm{PF}_t) = 0.1425$, computed once.
- L256–259: a biquad filter's five coefficients; the sensor-A filter, the display filter, and the sensor-A filter's complex response at each harmonic.
- L261: `ahf_tab[2][200]`, the two PWM duty tables (double buffer), in Q15 format: $-32767 \ldots +32767$ means $-100 \ldots +100\,\%$.
- L262: the duty phasors that produced each table. Needed for identification: "this table was playing, and this current was measured".
- L263–266: AHF state: present duty phasor per harmonic; plant gain estimate per harmonic; "measured at least once" flags; loop residual and duty peak for telemetry.
- L268–272: PLL state: commanded frequency, estimated grid frequency, previous phase, "have a previous phase" flag and lock flag.

19.3 Initial AHF plant model (L274–282)

```
274 static void ahf_model_init(void)
275 {
276     for (uint32_t h = 0; h <= MAX_H; h++) {
277         const float wh = TWO_PI * F_GRID_HZ * (float)h;
278         const cpx_t z  = c_mk(AHF_R_OHM, wh * AHF_L_H);
279         const cpx_t d  = c_mk(cosf(wh * AHF_DELAY_S), -sinf(wh * AHF_DELAY_S));
280         ahf_g[h] = c_mul(c_div(c_mk(AHF_VDC, 0.0f), z), d);
281     }
282 }
```

For each harmonic h (L276): angular frequency $h\omega$ (L277); impedance $Z = R + jh\omega L$ (L278); delay as the phasor $e^{-jh\omega\tau}$ (L279); gain $G_h = \dfrac{V_{dc}}{Z} e^{-jh\omega\tau}$ amperes per unit duty (L280). This is only the starting estimate.

19.4 Butterworth design (L284–296)

```
284 /* 2nd-order Butterworth low-pass (bilinear, pre-warped) and its exact
285  * response at every harmonic. The sample clock is locked to 200 samples per
286  * cycle, so harmonic h is always at 2*pi*h/200 rad/sample.                */
287 static void butter2(float fc, biquad_t *f)
288 {
289     const float k = tanf(3.14159265f * fc / (float)FS_HZ);
290     const float n = 1.0f / (1.0f + 1.41421356f * k + k * k);
291     f->b0 = k * k * n;
292     f->b1 = 2.0f * f->b0;
293     f->b2 = f->b0;
294     f->a1 = 2.0f * (k * k - 1.0f) * n;
295     f->a2 = (1.0f - 1.41421356f * k + k * k) * n;
296 }
```

A direct implementation of the formulas in Chapter 13: pre-warped $K = \tan(\pi f_c/f_s)$ (L289), normalisation n (L290), then the five coefficients (L291–295). `1.41421356f` is $\sqrt{2}$.

19.5 Zero-phase filter (L298–323)

```
298 /* Zero-phase low-pass of one periodic cycle: the biquad runs forward, then
299  * backward, each time wrapping around the cycle once first so its state is
300  * already settled. Net response |H|^2 with no phase shift, so filtered V, I
301  * and AHF waveforms stay exactly aligned in time. Display only.           */
302 static void lpf_zero_phase(const biquad_t *f, float *x)
303 {
304     static float y[N_CYC];
305     float z1 = 0.0f, z2 = 0.0f;
306     for (uint32_t pass = 0; pass < 2u; pass++) {
307         for (uint32_t k = 0; k < N_CYC; k++) {
308             const float o = f->b0 * x[k] + z1;
309             z1 = f->b1 * x[k] - f->a1 * o + z2;
310             z2 = f->b2 * x[k] - f->a2 * o;
311             y[k] = o;
312         }
313     }
314     z1 = z2 = 0.0f;
315     for (uint32_t pass = 0; pass < 2u; pass++) {
316         for (uint32_t k = N_CYC; k-- > 0u; ) {
317             const float o = f->b0 * y[k] + z1;
318             z1 = f->b1 * y[k] - f->a1 * o + z2;
319             z2 = f->b2 * y[k] - f->a2 * o;
320             if (pass) x[k] = o;
321         }
322     }
323 }
```

- L306–313: **forward** pass in direct form II transposed (L308–310). It is done twice: the first pass only warms up the filter state on the periodic cycle, so that at sample 0 of the second pass the filter is already in steady state. The output goes to `y`.
- L314: reset the state.
- L315–322: **backward** pass over `y` (L316 counts k from 199 down to 0), again twice, writing the result back into `x` only on the second pass (L320). The backward pass applies the opposite phase shift, so the net phase is zero.
- The array `y` is `static` so the 800-byte buffer is not placed on the stack.

19.6 Filter set-up and response table (L325–339)

```
325 static void lpf_init(void)
326 {
327     butter2(A_LPF_HZ, &a_lpf);
328     butter2(VI_LPF_HZ, &vi_lpf);
329     for (uint32_t h = 0; h <= MAX_H; h++) {
330         const float w  = TWO_PI * (float)h / (float)N_CYC;
331         const cpx_t z1 = c_mk(cosf(w), -sinf(w));            /* z^-1 */
332         const cpx_t z2 = c_mul(z1, z1);
333         const cpx_t num = c_add(c_mk(a_lpf.b0, 0.0f),
334                                 c_add(c_scale(z1, a_lpf.b1), c_scale(z2, a_lpf.b2)));
335         const cpx_t den = c_add(c_mk(1.0f, 0.0f),
336                                 c_add(c_scale(z1, a_lpf.a1), c_scale(z2, a_lpf.a2)));
337         a_lpf_h[h] = c_div(num, den);
338     }
339 }
```

L327–328 design both filters. Then for each harmonic (L329), at $\Omega = 2\pi h/200$ (L330), the firmware evaluates the transfer function

$$H(e^{j\Omega}) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}, \qquad z^{-1} = \cos\Omega - j\sin\Omega$$

(L331–337) and stores it in `a_lpf_h[h]`. Later, every measured sensor-A phasor is divided by this value.

19.7 Start-up tables (L341–352)

```
341 static void tables_init(void)
342 {
343     for (uint32_t k = 0; k < N_CYC; k++) {
344         float th = TWO_PI * (float)k / (float)N_CYC;
345         cos_t[k] = cosf(th);
346         sin_t[k] = sinf(th);
347     }
348     tan_phi_tgt = sqrtf(1.0f - PF_TARGET * PF_TARGET) / PF_TARGET;
349     memset(ahf_tab, 0, sizeof ahf_tab);
350     ahf_model_init();
351     lpf_init();
352 }
```

Fills the cosine/sine tables for one cycle (L343–347); computes $\tan\varphi_t = \sqrt{1-\mathrm{PF}^2}/\mathrm{PF}$ (L348); clears the PWM tables (L349); initialises the AHF model (L350) and the filters (L351). The expensive trigonometric functions therefore run only once, at start-up.

19.8 DFT and THD (L354–375)

```
354 static void dft(const float *x, float *a, float *b)
355 {
356     for (uint32_t h = 1; h <= MAX_H; h++) {
357         float sa = 0.0f, sb = 0.0f;
358         uint32_t idx = 0;
359         for (uint32_t k = 0; k < N_CYC; k++) {
360             sa += x[k] * cos_t[idx];
361             sb += x[k] * sin_t[idx];
362             idx += h;
363             if (idx >= N_CYC) idx -= N_CYC;
```

```
364         }
365         a[h] = sa * (2.0f / (float)N_CYC);
366         b[h] = sb * (2.0f / (float)N_CYC);
367     }
368 }
369
370 static float thd_pct(const float *h)
371 {
372     float s = 0.0f;
373     for (uint32_t k = 2; k <= MAX_H; k++) s += h[k] * h[k];
374     return (h[1] > 1e-6f) ? 100.0f * sqrtf(s) / h[1] : 0.0f;
375 }
```

- L356–367: for every harmonic $h = 1 \ldots 15$, accumulate $x_k \cos$ and $x_k \sin$ over the 200 samples. The table index advances by h per sample (L362) and wraps at 200 (L363), which is the same as computing $(h \cdot k) \bmod 200$ but needs no multiplication or division. L365–366 apply the $2/N$ scaling, so a_h, b_h are peak amplitudes.
- L370–375: THD in percent from an array of harmonic rms values; returns 0 if the fundamental is essentially zero (no division by zero).

19.9 AHF control step (L377–457)

```
377 static void ahf_reset(void)
378 {
379     memset(ahf_u, 0, sizeof ahf_u);
380     memset(ahf_tab_u, 0, sizeof ahf_tab_u);
381     memset(ahf_tab, 0, sizeof ahf_tab);
382     ahf_resid_pct = ahf_duty_pk = 0.0f;
383 }
384
385 /* One control step. ai/bi: DFT of sensor I, aa/ba: DFT of sensor A, taken
386  * over a window during which table `applied` played unchanged. Writes the
387  * next table into ahf_tab[applied ^ 1]; the caller hands it to the ISR.     */
388 static void ahf_control(const float *ai, const float *bi, const float *aa, const float *ba,
389                         uint32_t applied, bool drive)
390 {
391     static float u[N_CYC];
392     float r2 = 0.0f, e2 = 0.0f;
393
394     for (uint32_t h = 2; h <= MAX_H; h++) {
395         if (!(AHF_HARM_MASK & (1u << h))) { ahf_u[h] = c_mk(0.0f, 0.0f); continue; }
396         const cpx_t ii = c_phasor(ai[h], bi[h]);
397         const cpx_t m  = c_phasor(aa[h], ba[h]);
398         const cpx_t ua = ahf_tab_u[applied][h];
399
400         /* plant identification: what this duty phasor actually produced.
401          * The first measurement replaces the model outright: blending from a
402          * badly wrong model would pass through |G| ~ 0 and blow up E / G.  */
403         if (c_abs2(ua) > AHF_ID_MIN_DUTY * AHF_ID_MIN_DUTY) {
404             const cpx_t gm = c_div(m, ua);
405             const float r  = c_abs2(gm) / c_abs2(ahf_g[h]);
406             if (!ahf_g_seen[h]) {
407                 if (c_abs2(gm) > 0.0f) { ahf_g[h] = gm; ahf_g_seen[h] = 1u; }
408             } else if (r > 1e-4f && r < 1e4f) {
409                 ahf_g[h] = c_add(ahf_g[h], c_scale(c_sub(gm, ahf_g[h]), AHF_ID_RATE));
410             }
411         }
412
413         if (!drive) { ahf_u[h] = c_scale(ahf_u[h], 0.5f); continue; }
414 #if AHF_SENSE_GRID
415         const cpx_t e = c_scale(ii, -1.0f / AHF_INJ_RATIO);   /* grid residual */
```

```
416         const cpx_t r = c_add(e, m);
417 #else
418         const cpx_t r = c_scale(ii, -1.0f / AHF_INJ_RATIO);   /* -load harmonic */
419         const cpx_t e = c_sub(r, m);
420 #endif
421         r2 += c_abs2(r);
422         e2 += c_abs2(e);
423         const float mu = ahf_g_seen[h] ? AHF_MU : AHF_MU_PROBE;
424         ahf_u[h] = c_scale(c_add(ahf_u[h], c_scale(c_div(e, ahf_g[h]), mu)),
425                            1.0f - AHF_LEAK);
426     }
427     ahf_resid_pct = (r2 > 1e-10f) ? 100.0f * sqrtf(e2 / r2) : 0.0f;
```

- L377–383 `ahf_reset`: zero all duty phasors and tables. Called when the AHF is switched to STANDBY. The plant estimates `ahf_g` are *kept*, so the next start is fast.
- L388: inputs are the DFT of sensor I (`ai/bi`), the DFT of sensor A (`aa/ba`), which table was playing (`applied`), and whether to drive at all.
- L395: skip harmonics not enabled in the mask.
- L396–398: the load harmonic phasor I_h, the measured AHF phasor M_h, and the duty phasor U_h that was playing.
- L403–411: **identification**. Only when that harmonic was actually being driven (duty above 1 %, L403): measured gain $= M_h/U_h$ (L404). The first measurement replaces the model outright (L406–407). Later ones blend in at 25 % (L409), but only if the new value is not absurdly different from the present estimate (ratio of squared magnitudes between 10^{-4} and 10^4, L408). That protects against a corrupted window.
- L413: if not driving (no load, or nothing to cancel), halve the duty phasor each update so the output fades out smoothly.
- L414–420: required current and error. With sensor I on the load side (L418–419): $R = -I_h/n$, $E = R - M_h$. With sensor I on the grid side (L415–416): the error is simply the remaining grid harmonic.
- L421–422: accumulate $|R|^2$ and $|E|^2$ for the residual figure.
- L423–425: the **integrator**: $U_h \leftarrow (1 - 0.002)(U_h + \mu E_h/\hat{G}_h)$, with the smaller $\mu = 0.15$ until the plant has been measured once.
- L427: loop residual $= 100\sqrt{\sum |E|^2 / \sum |R|^2}$ %. This is the "loop residual" shown on the dashboard.

```
429     /* synthesise one cycle from the harmonic phasors */
430     memset(u, 0, sizeof u);
431     for (uint32_t h = 2; h <= MAX_H; h++) {
432         const float ur = ahf_u[h].re, ui = ahf_u[h].im;
433         if (ur == 0.0f && ui == 0.0f) continue;
434         uint32_t idx = 0;
435         for (uint32_t k = 0; k < N_CYC; k++) {
436             u[k] += ur * cos_t[idx] - ui * sin_t[idx];
437             idx += h;
438             if (idx >= N_CYC) idx -= N_CYC;
439         }
440     }
441     float pk = 0.0f;
442     for (uint32_t k = 0; k < N_CYC; k++) if (fabsf(u[k]) > pk) pk = fabsf(u[k]);
443     if (pk > AHF_MOD_DEPTH) {                 /* saturation: scale, no windup */
444         const float s = AHF_MOD_DEPTH / pk;
445         for (uint32_t h = 2; h <= MAX_H; h++) ahf_u[h] = c_scale(ahf_u[h], s);
446         for (uint32_t k = 0; k < N_CYC; k++) u[k] *= s;
```

```
447         pk = AHF_MOD_DEPTH;
448     }
449     ahf_duty_pk = pk;
450 
451     int16_t *t = ahf_tab[applied ^ 1u];
452     for (uint32_t k = 0; k < N_CYC; k++) {
453         const float f = u[k] * 32767.0f;
454         t[k] = (int16_t)(f + (f >= 0.0f ? 0.5f : -0.5f));
455     }
456     memcpy(ahf_tab_u[applied ^ 1u], ahf_u, sizeof ahf_u);
457 }
```

- L430–440: **synthesis**: build one cycle $u_k = \sum_h (U_r \cos - U_i \sin)$, which equals $\mathrm{Re}\{U_h e^{jh\theta}\}$, using the same index trick as the DFT. Harmonics with zero duty are skipped (L433).
- L441–448: find the peak. If it exceeds 0.92, scale *both* the waveform and the stored phasors by 0.92/peak. Scaling the stored phasors is what prevents integrator wind-up.
- L449: report the peak duty ("duty pk" on the dashboard).
- L451–455: convert to Q15 integers with correct rounding (L454 adds ± 0.5 before truncation) into the table that is *not* playing (`applied ^ 1`).
- L456: remember which phasors built that table.

19.10 PLL update and period calculation (L459–493)

```
459 /* Grid lock from the voltage fundamental (a1, b1) of the last window. With
460  * the clock locked, the window start sits at a fixed point of the cycle and
461  * every harmonic phasor is stationary from one window to the next.         */
462 static void pll_update(float a1, float b1, bool valid)
463 {
464     if (!valid) {
465         pll_have_prev = 0;
466         pll_locked    = 0;
467         pll_f_cmd += 0.2f * (F_GRID_HZ - pll_f_cmd);
468         return;
469     }
470     const float ph = atan2f(-b1, a1);
471     if (pll_have_prev) {
472         /* phase gained over CYC_PER_WIN cycles of the commanded frequency */
473         const float f_grid = pll_f_cmd *
474             (1.0f + wrap_pi(ph - pll_ph_prev) / (TWO_PI * (float)CYC_PER_WIN));
475         pll_f_est += PLL_F_ALPHA * (f_grid - pll_f_est);
476         float f = f_grid + PLL_KP_HZ * wrap_pi(ph - PLL_PHASE_SET);
477         if (f < PLL_F_MIN) f = PLL_F_MIN;
478         if (f > PLL_F_MAX) f = PLL_F_MAX;
479         pll_f_cmd  = f;
480         pll_locked = fabsf(wrap_pi(ph - PLL_PHASE_SET)) < 0.35f;    /* ~1 ms  */
481     }
482     pll_ph_prev   = ph;
483     pll_have_prev = 1;
484 }
485 
486 /* TIM2 period for frequency f: n whole ticks plus a 16-bit fraction. */
487 static void pll_period(float f, uint32_t *n, uint32_t *frac)
488 {
489     const float t = (float)APB1_TIM_HZ / ((float)N_CYC * f);
490     *n    = (uint32_t)t;
491     *frac = (uint32_t)((t - (float)*n) * 65536.0f);
492 }
493 /* DSP-CORE-END */
```

- L464–469: no valid voltage (mains OFF). Forget the previous phase, mark unlocked, and slowly return the commanded frequency to 50 Hz (20 % per window).
- L470: phase of the voltage fundamental at the window start.
- L473–474: estimated grid frequency from the phase change over 5 cycles (formula in Chapter 14).
- L475: smoothed value for display.
- L476–479: new commanded frequency = estimate + $K_p\times$ phase error, clamped to 47.5–52.5 Hz.
- L480: locked if the phase error is below 0.35 rad (about 1.1 ms).
- L482–483: remember the phase for the next window.
- L487–492 `pll_period`: ticks per sample $t = 64\,\text{MHz}/(200f)$, split into the integer part n and a 16-bit fraction ($\times 65536$). Example at 49.02 Hz: $t = 6527.95$, so $n = 6527$ and frac $\approx 62\,259$.

20 Global variables

```
/* ------------------------------------------------------------------------ */
/*  Globals                                                                 */
/* ------------------------------------------------------------------------ */
static volatile uint16_t adc_buf[DMA_LEN];
static uint16_t          snap[N_WIN * N_CH];
static volatile int32_t  win_ready = -1;
static volatile uint8_t  win_clean[2];      /* no table swap inside the half */
static volatile uint32_t win_dropped, adc_ovr_cnt, tx_skip;
static volatile bool     resync_req;
static volatile uint32_t ms_ticks;          /* advanced by the ADC DMA ISR   */

static volatile uint32_t tim2_n = TIM2_N_NOM, tim2_frac, tim2_acc;
static volatile uint32_t pwm_period = PWM_PERIOD_NOM;

static float w_v[N_CYC], w_i[N_CYC], w_a[N_CYC], w_c[N_CYC], w_t[N_CYC];
static float av[MAX_H + 1], bv[MAX_H + 1], ai[MAX_H + 1], bi[MAX_H + 1],
             aa[MAX_H + 1], ba[MAX_H + 1];

typedef struct {
    float vrms, irms, p, s, pf, p1, q1, dpf, dva;
    float thd_v, thd_i, crest, vpk, ipk, apk, i_noise;
    float h_i[MAX_H + 1], h_a[MAX_H + 1], h_c[MAX_H + 1], thd_c;
    float off[N_CH];
    uint16_t mn[N_CH], mx[N_CH];
    uint32_t clip[N_CH];
} win_t;

typedef struct { float sv2, si2, p, p1, q1, vpk, ipk; uint32_t n; } acc_t;
typedef struct { float vrms, irms, p, s, pf, p1, q1, dpf, dva, vpk, ipk; } avg_t;

static win_t   mw;
static acc_t   acc;
static uint32_t settle_win, win_count;

static volatile uint32_t ahf_sel, ahf_swap, ahf_swapped;
static volatile uint32_t ahf_idx;
static volatile uint32_t ahf_active = AHF_BOOT_ACTIVE;
static float             ahf_ref_pk_f;
static bool              ahf_is_reset;

static const float       cap_uF[3] = { 2.5f, 10.0f, 20.0f };
static volatile uint8_t  lock_mask = 0;
static volatile uint8_t  lock_state = 0;
static volatile bool     pfc_cmd_pending;
static uint8_t           relay_state = 0, relay_target = 0;
static uint32_t          relay_off_ms[3] = {0}, relay_on_ms[3] = {0};
static volatile uint8_t  relay_pend_on, relay_pend_on0, relay_pend_off;  /* TIM2 ISR */
static uint8_t           relay_v_known;     /* cap left near +Vpk by a timed open */
static bool              grid_v_present;
static uint8_t           pfc_hold = 1;
static float             pfc_c_need = 0.0f;
static uint8_t           pfc_reason;        /* 0 ok, 1 no load, 2 load below the
                                               * smallest step, 3 all manual     */
static uint8_t           relay_waiting;     /* close held back: no grid lock   */
static uint32_t          relay_wait_ms[3];
static float             lpf_z1, lpf_z2;    /* sensor A filter state           */
static bool              lpf_primed;

static char              tx_buf[TX_BUF_SZ];
static char             *tx_p;
static volatile bool     tx_busy;
```

```
556 static char              rx_line[RX_LINE_MAX];
557 static uint32_t          rx_len;
558 static volatile uint32_t cmd_ok, cmd_bad, rx_bytes;
```

Variable	Written by	Purpose
`adc_buf[6000]`	DMA hardware	raw interleaved samples, two halves; `volatile` because hardware changes it
`snap[3000]`	main loop	stable copy of the finished half
`win_ready`	DMA ISR / main	which half is ready (−1: none)
`win_clean[2]`	DMA ISR	no AHF table switch happened in that half
`win_dropped`, `adc_ovr_cnt`, `tx_skip`	ISRs / main	diagnostics counters (shown in `$R`)
`resync_req`	ISRs	request to restart acquisition after an error
`ms_ticks`	DMA ISR	time base: +100 ms per window
`tim2_n, tim2_frac, tim2_acc`	`pll_apply` / TIM2 ISR	sample period and its dithering accumulator
`pwm_period`	`pll_apply`	TIM4 period in ticks (about 800)
`w_v, w_i, w_a, w_c, w_t`	`process_window`	one averaged cycle of each waveform
`av,bv,ai,bi,aa,ba`	`process_window`	DFT coefficients of V, I, A
`win_t / mw`	`process_window`	all results of one window
`acc_t / acc`	`acc_add`	sums over the windows of one 0.5 s period
`avg_t`	`make_avg`	averaged results for APFC and telemetry
`settle_win`, `win_count`	main	windows to skip after switching; window counter
`ahf_sel, ahf_swap, ahf_swapped`	main / TIM2 ISR	which table plays; request to switch; switch happened
`ahf_idx`	TIM2 ISR	sample index within the cycle (0–199)
`ahf_active`	USART ISR	AHF ACTIVE / STANDBY command
`ahf_ref_pk_f`, `ahf_is_reset`	`ahf_step`	smoothed target peak; reset done once
`cap_uF[3]`	constant	2.5, 10, 20 μF
`lock_mask`, `lock_state`	USART ISR	manual locks: which steps, and ON/OFF
`pfc_cmd_pending`	USART ISR	legacy flag (cleared each loop, not used further)
`relay_state`, `relay_target`	main	what is ON now; what the controller wants
`relay_off_ms`, `relay_on_ms`	`relay_service`	time of last OFF / ON per step (min ON/OFF rules)
`relay_pend_on/on0/off`	main / TIM2 ISR	coil changes scheduled for the ISR
`relay_v_known`	`relay_service`	capacitor left at $+V_{pk}$ by a timed opening
`grid_v_present`	main	mains present (for relay timing)
`pfc_hold`, `pfc_c_need`, `pfc_reason`	`apfc_step`	decision outputs for telemetry
`relay_waiting`, `relay_wait_ms`	`relay_service`	closing delayed for lock, and since when
`lpf_z1, lpf_z2, lpf_primed`	`process_window`	sensor-A filter state

`tx_buf, tx_p, tx_busy`	telemetry / DMA ISR	transmit buffer, write pointer, DMA busy
`rx_line, rx_len`	USART ISR	command line being received
`cmd_ok, cmd_bad, rx_bytes`	USART ISR	command statistics

Key idea

Every variable shared between an ISR and the main loop is `volatile`. Variables used only by the main loop are not, so the compiler can optimise them. `static` at file level means the variable is private to this file.

21 Hardware initialisation

21.1 `clock_init` (L560–590)

```
560 /* ------------------------------------------------------------------------ */
561 /*  System Initialization                                                   */
562 /* ------------------------------------------------------------------------ */
563 static void clock_init(void)
564 {
565     RCC->APB1ENR |= RCC_APB1ENR_PWREN;
566     (void)RCC->APB1ENR;
567     PWR->CR = (PWR->CR & ~PWR_CR_VOS) | (2u << PWR_CR_VOS_Pos);
568 
569     FLASH->ACR = FLASH_ACR_LATENCY_2WS | FLASH_ACR_PRFTEN | FLASH_ACR_ICEN | FLASH_ACR_DCEN;
570     while ((FLASH->ACR & FLASH_ACR_LATENCY) != FLASH_ACR_LATENCY_2WS) {}
571 
572     RCC->CR |= RCC_CR_HSION;
573     while (!(RCC->CR & RCC_CR_HSIRDY)) {}
574     RCC->CFGR = 0;
575     while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI) {}
576     RCC->CR &= ~RCC_CR_PLLON;
577     while (RCC->CR & RCC_CR_PLLRDY) {}
578 
579     RCC->PLLCFGR = (16u  << RCC_PLLCFGR_PLLM_Pos) |
580                    (256u << RCC_PLLCFGR_PLLN_Pos) |
581                    (1u   << RCC_PLLCFGR_PLLP_Pos) |
582                    RCC_PLLCFGR_PLLSRC_HSI |
583                    (8u   << RCC_PLLCFGR_PLLQ_Pos);
584     RCC->CR |= RCC_CR_PLLON;
585     while (!(RCC->CR & RCC_CR_PLLRDY)) {}
586 
587     RCC->CFGR = RCC_CFGR_HPRE_DIV1 | RCC_CFGR_PPRE1_DIV2 | RCC_CFGR_PPRE2_DIV1 | RCC_CFGR_SW_PLL;
588     while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}
589     SystemCoreClock = SYSCLK_HZ;
590 }
```

Line	Explanation
L565	Enable the clock of the power controller (PWR), needed to change the regulator voltage scale.
L566	Dummy read: delay after a clock enable (errata).
L567	Set the `VOS` field to 2: regulator voltage scale 2 (up to 84 MHz).
L569	Flash: 2 wait states, prefetch, instruction cache and data cache enabled. *Before* speeding up.
L570	Wait until the new latency setting is active.
L572–573	Make sure the internal 16 MHz oscillator (HSI) is on and stable.
L574–575	Select HSI as the system clock (`CFGR`= 0) and wait for the switch. The PLL can then be safely reconfigured.
L576–577	Turn the PLL off and wait until it has stopped (it cannot be changed while running).
L579–583	PLL configuration: $M = 16$, $N = 256$, P code 1 ($\div 4$), source HSI, $Q = 8$.
L584–585	Start the PLL; wait until it is locked (`PLLRDY`).
L587	AHB $\div 1$ (64 MHz), APB1 $\div 2$ (32 MHz), APB2 $\div 1$ (64 MHz), and switch the system clock to the PLL.
L588	Wait until the switch has happened.
L589	Update the CMSIS variable `SystemCoreClock` so library code knows the speed.

21.2 Relay output functions (L592–623)

```
592 /* BSRR value that puts the relay pins in `bits` into state `st` */
593 static uint32_t relay_bsrr(uint8_t bits, uint8_t st)
594 {
595     /* pin HIGH = SSR ON, or relay module OFF (released open-drain) */
596     const uint32_t m  = (uint32_t)bits & 7u;
597     const uint32_t hi = (((uint32_t)st & RELAY_SSR_MASK) |
598                          (~(uint32_t)st & ~RELAY_SSR_MASK)) & m;
599     return hi | ((m & ~hi) << 16);
600 }
601
602 static void relay_write(uint8_t st)
603 {
604     GPIOB->BSRR = relay_bsrr(7u, st);
605 }
606
607 /* Called by the TIM2 ISR for every sample; i = sample index in the cycle.
608  * Drives the coils that relay_service() scheduled for point-on-wave.     */
609 static inline void relay_isr_step(uint32_t i)
610 {
611     if (relay_pend_on && i == RELAY_K_ON) {           /* close at +peak       */
612         GPIOB->BSRR   = relay_bsrr(relay_pend_on, 7u);
613         relay_pend_on = 0u;
614     }
615     if (relay_pend_on0 && i == RELAY_K_ON_ZERO) {     /* close at zero cross  */
616         GPIOB->BSRR    = relay_bsrr(relay_pend_on0, 7u);
617         relay_pend_on0 = 0u;
618     }
619     if (relay_pend_off && i == RELAY_K_OFF) {
620         GPIOB->BSRR    = relay_bsrr(relay_pend_off, 0u);
621         relay_pend_off = 0u;
622     }
623 }
```

`relay_bsrr(bits, st)` returns the value to write to `GPIOB->BSRR` so that the relay pins selected by `bits` take the ON/OFF state given by `st`. Other pins are untouched.

- L596: only bits 0–2 (PB0–PB2) are relay pins.
- L597–598: which pins must be electrically HIGH. For an SSR bit (in the mask), HIGH means ON. For a relay-module bit (not in the mask), HIGH means released, i.e. OFF. So: HIGH = (`st` AND SSR-mask) OR (NOT `st` AND NOT SSR-mask).
- L599: those pins go in the lower 16 bits (set); the remaining selected pins go in the upper 16 bits (reset).

Pin	Type	Command ON	Command OFF
PB0 (2.5 μF)	relay module, open-drain	pin LOW (reset bit 16)	released (set bit 0)
PB1 (10 μF)	relay module, open-drain	pin LOW (reset bit 17)	released (set bit 1)
PB2 (20 μF)	SSR, push-pull	pin HIGH (set bit 2)	pin LOW (reset bit 18)

`relay_write(st)` (L602–605) applies a complete state to all three pins at once.

`relay_isr_step(i)` (L609–623) runs inside the 10 kHz TIM2 interrupt with the current sample index i. If a relay was scheduled to close at the peak and i has reached 130, it energises those coils (L611–614). For the zero-crossing closing it waits for $i = 80$ (L615–618); for opening, $i = 130$ (L619–622). Then it clears the schedule. This gives the coil switching a timing accuracy of one sample (0.1 ms).

21.3 gpio_init (L625–665)

```
625 static void gpio_init(void)
626 {
627     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOBEN |
628                     RCC_AHB1ENR_DMA1EN  | RCC_AHB1ENR_DMA2EN;
629     (void)RCC->AHB1ENR;
630
631     /* PA0, PA1, PA4 analog */
632     GPIOA->MODER |= (3u << (0 * 2)) | (3u << (1 * 2)) | (3u << (4 * 2));
633     GPIOA->PUPDR &= ~((3u << (0 * 2)) | (3u << (1 * 2)) | (3u << (4 * 2)));
634
635     /* PA2/PA3 AF7 USART2 */
636     GPIOA->MODER   = (GPIOA->MODER & ~((3u << (2 * 2)) | (3u << (3 * 2)))) |
637                      (2u << (2 * 2)) | (2u << (3 * 2));
638     GPIOA->AFR[0]  = (GPIOA->AFR[0] & ~((0xFu << (2 * 4)) | (0xFu << (3 * 4)))) |
639                      (7u << (2 * 4)) | (7u << (3 * 4));
640     GPIOA->PUPDR   = (GPIOA->PUPDR & ~(3u << (3 * 2))) | (1u << (3 * 2));
641
642     /* PA5 LD2 */
643     GPIOA->MODER   = (GPIOA->MODER & ~(3u << (5 * 2))) | (1u << (5 * 2));
644
645     /* Relays. Order matters: output type and OFF level are set while the pins
646      * are still inputs, so they never drive LOW (= relay ON) even briefly.  */
647     GPIOB->OTYPER  = (GPIOB->OTYPER & ~7u) | (~RELAY_SSR_MASK & 7u);  /* modules OD */
648     GPIOB->PUPDR  &= ~((3u << (0 * 2)) | (3u << (1 * 2)) | (3u << (2 * 2)));
649     relay_write(0);                                          /* all OFF       */
650     relay_state  = 0;
651     relay_target = 0;
652     GPIOB->MODER   = (GPIOB->MODER & ~((3u << 0) | (3u << 2) | (3u << 4))) |
653                      (1u << 0) | (1u << 2) | (1u << 4);
654
655     /* PB6/PB8 AF2 TIM4 */
656     GPIOB->BSRR    = GPIO_BSRR_BR6 | GPIO_BSRR_BR8;
657     GPIOB->PUPDR   = (GPIOB->PUPDR & ~((3u << (6 * 2)) | (3u << (8 * 2)))) |
658                      (2u << (6 * 2)) | (2u << (8 * 2));
659     GPIOB->OSPEEDR = (GPIOB->OSPEEDR & ~((3u << (6 * 2)) | (3u << (8 * 2)))) |
660                      (2u << (6 * 2)) | (2u << (8 * 2));
661     GPIOB->AFR[0]  = (GPIOB->AFR[0] & ~(0xFu << (6 * 4))) | (2u << (6 * 4));
662     GPIOB->AFR[1]  = (GPIOB->AFR[1] & ~(0xFu << ((8 - 8) * 4))) | (2u << ((8 - 8) * 4));
663     GPIOB->MODER   = (GPIOB->MODER & ~((3u << (6 * 2)) | (3u << (8 * 2)))) |
664                      (2u << (6 * 2)) | (2u << (8 * 2));
665 }
```

Line	Explanation
L627–629	Enable clocks of GPIOA, GPIOB, DMA1 and DMA2 (all on AHB1), followed by a dummy read.
L632	PA0, PA1, PA4: `MODER=11`, analog mode (each pin has 2 bits at position 2×pin).
L633	No pull-up or pull-down on the analog pins (it would load the sensor signal).
L636–637	PA2, PA3: `MODER=10`, alternate function.
L638–639	`AFR[0]`: AF7 (USART2) for PA2 (TX) and PA3 (RX); 4 bits per pin.
L640	Pull-up on PA3 (RX), so the line idles HIGH if nothing is connected (UART idle level).
L643	PA5 (on-board LED LD2): general-purpose output.
L647	Output type: PB0, PB1 open-drain (1); PB2 push-pull (0), computed from the SSR mask. Done while the pins are still inputs.
L648	No pull-ups on the relay pins (an internal pull-up would clamp a module input to 3.3 V).
L649	Write the OFF levels into the output register *before* enabling the outputs.

L650–651	Software state: nothing ON, nothing requested.
L652–653	Only now switch PB0, PB1, PB2 to output mode. The first level they drive is already OFF.
L656	PB6, PB8 output data LOW (safe default for the H-bridge inputs).
L657–658	Pull-downs on PB6, PB8: if the timer is not driving, the bridge inputs stay LOW (off).
L659–660	Fast output speed for clean 80 kHz edges.
L661–662	AF2 (TIM4) for PB6 (`AFR[0]`, pin 6) and PB8 (`AFR[1]`, pin $8 - 8 = 0$).
L663–664	Switch PB6, PB8 to alternate-function mode: TIM4 now drives them.

21.4 Optional relay self-test (L667–686)

```
667 #if RELAY_SELFTEST
668 /* Busy-wait on the DWT cycle counter (no interrupts needed). */
669 static void delay_ms(uint32_t ms)
670 {
671     CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
672     DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
673     const uint32_t t0 = DWT->CYCCNT, n = ms * (SYSCLK_HZ / 1000u);
674     while ((DWT->CYCCNT - t0) < n) {}
675 }
676
677 static void relay_selftest(void)
678 {
679     for (uint8_t i = 0; i < 3u; i++) {
680         relay_write((uint8_t)(1u << i));      /* LED + click ON  */
681         delay_ms(1000);
682         relay_write(0);                       /* LED + click OFF */
683         delay_ms(1000);
684     }
685 }
686 #endif
```

Compiled only if `RELAY_SELFTEST` is 1. `delay_ms` uses the core's cycle counter (DWT `CYCCNT`), which counts CPU clocks: enable it (L671–672), then wait until $ms \times 64\,000$ cycles have passed (L673–674). The unsigned subtraction handles counter wrap-around. The test turns each relay ON for one second and OFF for one second, for wiring checks with the mains disconnected.

22 Acquisition: TIM2, ADC, DMA

22.1 tim2_init (L688–703)

```
688 /* ============================================================================ */
689 /*  TIM2 (grid-locked sample clock), ADC1 + DMA2 Stream0                        */
690 /* ============================================================================ */
691 static void tim2_init(void)
692 {
693     RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
694     (void)RCC->APB1ENR;
695     TIM2->CR1  = TIM_CR1_ARPE;
696     TIM2->PSC  = 0;
697     TIM2->ARR  = TIM2_N_NOM - 1u;
698     TIM2->CR2  = TIM_CR2_MMS_1;               /* TRGO = update            */
699     TIM2->EGR  = TIM_EGR_UG;
700     TIM2->SR   = 0;
701     TIM2->CNT  = 0;
702     TIM2->DIER = TIM_DIER_UIE;
703 }
```

- L693–694: enable the TIM2 clock (APB1).
- L695: CR1 = ARPE: auto-reload preload, so a new period takes effect at the next update. The counter is not yet started.
- L696: prescaler 0, counting at 64 MHz (15.625 ns per tick).
- L697: ARR = 6399: 6400 ticks = 100 μs.
- L698: MMS = 010: TRGO pulses on every update. This is the ADC trigger and the TIM4 reset.
- L699: UG forces an update so that ARR and PSC are loaded into the shadow registers now.
- L700–701: clear the flag that UG raised; counter to 0.
- L702: enable the update interrupt (TIM2_IRQHandler).

22.2 adc_dma_init (L705–730)

```
705 static void adc_dma_init(void)
706 {
707     RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;
708     (void)RCC->APB2ENR;
709
710     ADC->CCR    = (1u << ADC_CCR_ADCPRE_Pos);                  /* PCLK2 / 4  */
711     ADC1->CR1   = ADC_CR1_SCAN | ADC_CR1_OVRIE;
712     ADC1->SMPR2 = (ADC_SMP_CODE << ADC_SMPR2_SMP0_Pos) |
713                   (ADC_SMP_CODE << ADC_SMPR2_SMP1_Pos) |
714                   (ADC_SMP_CODE << ADC_SMPR2_SMP4_Pos);
715     ADC1->SQR1  = (N_CH - 1u) << ADC_SQR1_L_Pos;
716     ADC1->SQR3  = (0u << ADC_SQR3_SQ1_Pos) | (1u << ADC_SQR3_SQ2_Pos) | (4u << ADC_SQR3_SQ3_Pos);
717     ADC1->CR2   = (6u << ADC_CR2_EXTSEL_Pos) |
718                   (1u << ADC_CR2_EXTEN_Pos)  |
719                   ADC_CR2_DDS | ADC_CR2_ADON;
720
721     DMA2_Stream0->CR = 0;
722     while (DMA2_Stream0->CR & DMA_SxCR_EN) {}
723     DMA2_Stream0->PAR  = (uint32_t)&ADC1->DR;
```

```
724     DMA2_Stream0->M0AR = (uint32_t)adc_buf;
725     DMA2_Stream0->NDTR = DMA_LEN;
726     DMA2_Stream0->FCR  = 0;
727     DMA2_Stream0->CR   = (0u << DMA_SxCR_CHSEL_Pos) | DMA_SxCR_PL_1 |
728                          DMA_SxCR_MSIZE_0 | DMA_SxCR_PSIZE_0 | DMA_SxCR_MINC |
729                          DMA_SxCR_CIRC | DMA_SxCR_HTIE | DMA_SxCR_TCIE | DMA_SxCR_TEIE;
730 }
```

Line	Explanation
L707–708	Enable the ADC1 clock (APB2).
L710	Common ADC register `CCR`: `ADCPRE`= 1, ADC clock = PCLK2/4 = 16 MHz.
L711	`CR1`: scan mode (convert the whole sequence per trigger) and overrun interrupt enable.
L712–714	`SMPR2`: sampling time code 7 (480 cycles) for channels 0, 1 and 4.
L715	`SQR1.L` = 2: sequence length 3 (the field stores length −1).
L716	`SQR3`: 1st conversion channel 0 (I), 2nd channel 1 (V), 3rd channel 4 (A).
L717–719	`CR2`: external trigger select 6 (`TIM2_TRGO`), trigger on the rising edge (`EXTEN`= 01), `DDS` (keep issuing DMA requests continuously, required for circular DMA), and `ADON` (power up the ADC).
L721–722	Disable DMA2 Stream 0 and wait until it is really off (required before reconfiguring).
L723	Peripheral address: the ADC data register.
L724	Memory address: `adc_buf`.
L725	Number of transfers: 6000 (both halves).
L726	FIFO disabled (direct mode).
L727–729	Channel 0 (ADC1), priority high, 16-bit memory and peripheral size, memory increment, circular mode, half-transfer, transfer-complete and transfer-error interrupts. Direction is the default (peripheral to memory). The stream is enabled later, in `sampling_sync_start`.

22.3 Window hand-off and error interrupts (L732–758)

```
732 static inline void win_post(int32_t half)
733 {
734     if (win_ready >= 0) win_dropped++;
735     __disable_irq();
736     win_clean[half] = (uint8_t)(ahf_swapped ? 0u : 1u);
737     ahf_swapped = 0u;
738     __enable_irq();
739     win_ready = half;
740     ms_ticks += WIN_MS;               /* each half-buffer = 5 grid cycles */
741 }
742
743 void DMA2_Stream0_IRQHandler(void)
744 {
745     uint32_t isr = DMA2->LISR;
746     if (isr & DMA_LISR_TEIF0) { DMA2->LIFCR = DMA_LIFCR_CTEIF0; resync_req = true; }
747     if (isr & DMA_LISR_HTIF0) { DMA2->LIFCR = DMA_LIFCR_CHTIF0; win_post(0); }
748     if (isr & DMA_LISR_TCIF0) { DMA2->LIFCR = DMA_LIFCR_CTCIF0; win_post(1); }
749 }
750
751 void ADC_IRQHandler(void)
752 {
753     if (ADC1->SR & ADC_SR_OVR) {
754         ADC1->SR = ~ADC_SR_OVR;
755         adc_ovr_cnt++;
756         resync_req = true;
757     }
```

```
758 }
```

- `win_post(half)` (L732–741) runs inside the DMA interrupt when a half-buffer is full.
  - L734: if the main loop has not yet taken the previous window, count a dropped window.
  - L735–738: atomically read and clear `ahf_swapped`, the flag the TIM2 ISR sets when it switches PWM tables. If a switch happened during this half, the window is *not clean*. Interrupts are disabled for these two instructions because TIM2 has the higher priority and could otherwise set the flag between the read and the clear.
  - L739: publish which half is ready.
  - L740: advance the millisecond time base by 100. This replaces a SysTick timer.
- `DMA2_Stream0_IRQHandler` (L743–749): read the status register `LISR`. On a transfer error, clear the flag and request a restart (L746). On half-transfer, half 0 is full (L747); on transfer-complete, half 1 is full (L748). Each flag is cleared by writing to `LIFCR`.
- `ADC_IRQHandler` (L751–758): on an overrun, clear it, count it and request a restart.

22.4 `tim4_init` (L760–779)

```
760 /* ========================================================================== */
761 /*  TIM4  -  80 kHz TB6612FNG PWM, reset by every TIM2 update                 */
762 /* ========================================================================== */
763 static void tim4_init(void)
764 {
765     RCC->APB1ENR |= RCC_APB1ENR_TIM4EN;
766     (void)RCC->APB1ENR;
767     TIM4->CR1   = TIM_CR1_ARPE;               /* URS = 0: reset loads CCRs */
768     TIM4->PSC   = 0;
769     TIM4->ARR   = PWM_PERIOD_NOM - 1u;
770     TIM4->CCMR1 = (6u << TIM_CCMR1_OC1M_Pos) | TIM_CCMR1_OC1PE;
771     TIM4->CCMR2 = (6u << TIM_CCMR2_OC3M_Pos) | TIM_CCMR2_OC3PE;
772     TIM4->CCR1  = 0;
773     TIM4->CCR3  = 0;
774     TIM4->CCER  = TIM_CCER_CC1E | TIM_CCER_CC3E;
775     TIM4->EGR   = TIM_EGR_UG;
776     TIM4->SR    = 0;
777     TIM4->SMCR  = (1u << TIM_SMCR_TS_Pos) | (4u << TIM_SMCR_SMS_Pos);  /* ITR1 = TIM2, reset */
778     TIM4->DIER  = 0;
779 }
```

Line	Explanation
L765–766	Enable the TIM4 clock.
L767	Auto-reload preload. `URS`= 0, so a reset by the slave controller also generates an update event, which loads the preloaded compare values.
L768	Prescaler 0 (64 MHz).
L769	`ARR`= 799: 80 kHz.
L770–771	Channels 1 and 3: PWM mode 1 (`OCxM`= 110) with compare preload.
L772–773	Both compare values 0 (outputs LOW, bridge off).
L774	Enable the outputs of channels 1 and 3 (to PB6 and PB8).
L775–776	Load the settings, clear the flag.
L777	Slave mode: trigger input ITR1 (TIM2 TRGO), `SMS`= 100 reset mode. Every TIM2 update restarts the TIM4 counter.

L778 No TIM4 interrupts. The PWM runs in hardware; the duty is updated by the TIM2 ISR.

22.5 The 10 kHz interrupt: `TIM2_IRQHandler` (L781–815)

```
781 /* One sample: trim the sample period (grid lock) and step the AHF table. */
782 void TIM2_IRQHandler(void)
783 {
784     TIM2->SR = ~TIM_SR_UIF;
785 
786     /* fractional period: ARR alternates between n-1 and n */
787     const uint32_t a = tim2_acc + tim2_frac;
788     tim2_acc  = a & 0xFFFFu;
789     TIM2->ARR = tim2_n - 1u + (a >> 16);
790 
791     const uint32_t i = ahf_idx;
792     if (i == 0u && ahf_swap) { ahf_sel ^= 1u; ahf_swap = 0u; ahf_swapped = 1u; }
793     ahf_idx = (i + 1u < N_CYC) ? i + 1u : 0u;
794 
795     relay_isr_step(i);
796 
797     if (!ahf_active) {
798         TIM4->CCR1 = 0;
799         TIM4->CCR3 = 0;
800         return;
801     }
802 
803     /* Sign-magnitude, drive/brake (slow decay): one input held HIGH
804      * (CCR > ARR), the other pulses LOW for d of the period.              */
805     const int32_t  v = ahf_tab[ahf_sel][i];
806     const uint32_t p = pwm_period;
807     const uint32_t d = ((uint32_t)(v >= 0 ? v : -v) * p) >> 15;
808     if (v >= 0) {
809         TIM4->CCR1 = 0xFFFFu;
810         TIM4->CCR3 = p - d;
811     } else {
812         TIM4->CCR3 = 0xFFFFu;
813         TIM4->CCR1 = p - d;
814     }
815 }
```

This is the heartbeat of the system. It runs 10 000 times per second at the highest priority.

- L784: clear the update flag, otherwise the interrupt would re-enter immediately.
- L787–789: **fractional period**. Add the 16-bit fraction to the accumulator; keep the low 16 bits; the carry (0 or 1) makes this period one tick longer. Because of `ARPE`, the new `ARR` applies from the next period.
- L791: i is the sample index of the conversion just triggered, 0–199.
- L792: at the start of a cycle ($i = 0$), if a new table is waiting, switch tables and record that a switch happened.
- L793: advance the index, wrapping at 200.
- L795: timed relay coil switching (§21.2).
- L797–801: STANDBY: both compare values 0, so both bridge inputs are LOW (outputs off).
- L805–807: read this sample's Q15 duty v, and convert $|v|$ into PWM ticks: $d = |v| \cdot p/32768$, done with a shift (`» 15`) instead of a division.

- L808–814: sign-magnitude output. For $v \geq 0$, CH1 (IN1) is held HIGH (`0xFFFF` > ARR) and CH3 (IN2) is HIGH for $p - d$ ticks, then LOW for d ticks: d/p of the time the bridge drives positive, the rest it brakes. For $v < 0$ the roles swap.

Example

$v = 16384$ (50 % in Q15), $p = 800$: $d = 16384 \times 800/32768 = 400$ ticks. IN2 is LOW for 400 of 800 ticks, so the bridge drives $+V_M$ for 50 % of each 12.5 μs period: average $0.5\,V_M$.

22.6 Starting acquisition: `sampling_sync_start` (L817–852)

```
817 static void sampling_sync_start(void)
818 {
819     __disable_irq();
820     TIM2->CR1 &= ~TIM_CR1_CEN;
821     TIM4->CR1 &= ~TIM_CR1_CEN;
822 
823     ADC1->CR2 &= ~(ADC_CR2_DMA | ADC_CR2_ADON);
824     DMA2_Stream0->CR &= ~DMA_SxCR_EN;
825     while (DMA2_Stream0->CR & DMA_SxCR_EN) {}
826     DMA2->LIFCR = DMA_LIFCR_CTCIF0 | DMA_LIFCR_CHTIF0 | DMA_LIFCR_CTEIF0 |
827                   DMA_LIFCR_CDMEIF0 | DMA_LIFCR_CFEIF0;
828     DMA2_Stream0->M0AR = (uint32_t)adc_buf;
829     DMA2_Stream0->NDTR = DMA_LEN;
830     DMA2_Stream0->CR  |= DMA_SxCR_EN;
831     ADC1->SR = 0;
832     ADC1->CR2 |= ADC_CR2_ADON;
833     for (volatile uint32_t d = 0; d < 400u; d++) {}
834     ADC1->CR2 |= ADC_CR2_DMA;
835 
836     TIM4->CCR1 = 0;
837     TIM4->CCR3 = 0;
838     TIM4->EGR  = TIM_EGR_UG;
839     TIM4->SR   = 0;
840     TIM4->CNT  = 0;
841     TIM4->CR1 |= TIM_CR1_CEN;
842     TIM2->CNT  = 0;
843     TIM2->SR   = 0;
844     tim2_acc   = 0;
845     ahf_idx    = 0;                    /* table index == sample index mod 200 */
846     lpf_primed = false;
847     ahf_swapped = 0;
848     win_ready  = -1;
849 
850     TIM2->CR1 |= TIM_CR1_CEN;
851     __enable_irq();
852 }
```

Used at start-up and after any ADC or DMA error, so that sampling always starts from a known, aligned state.

- L819: no interrupts during the restart.
- L820–821: stop both timers.
- L823: stop the ADC's DMA requests and power it down.
- L824–825: stop the DMA stream and wait.
- L826–827: clear all its pending flags.
- L828–830: rearm: start address, 6000 transfers, enable.

- L831–834: clear the ADC status, power the ADC on, wait briefly (ADC stabilisation time), then enable its DMA requests.
- L836–841: TIM4 compare values to 0, load and start it (it is reset by TIM2 anyway).
- L842–848: TIM2 counter to 0, clear flags, reset the dithering accumulator, **sample index to 0** (so table index = sample index mod 200, aligned with the DMA buffer start), re-prime the sensor-A filter, no table switch pending, no window ready.
- L850: start TIM2. The first update triggers the first conversion and the first interrupt.
- L851: interrupts back on.

22.7 Applying the PLL result: `pll_apply` (L854–868)

```
854 /* Apply the PLL frequency to TIM2 (and keep 8 PWM periods per sample). */
855 static void pll_apply(void)
856 {
857     uint32_t n, frac;
858     pll_period(pll_f_cmd, &n, &frac);
859     const uint32_t p = (n + PWM_PER_SAMPLE / 2u) / PWM_PER_SAMPLE;
860     __disable_irq();
861     tim2_n    = n;
862     tim2_frac = frac;
863     if (p != pwm_period) {
864         pwm_period = p;
865         TIM4->ARR  = p - 1u;
866     }
867     __enable_irq();
868 }
```

Computes the new period n + fraction (L858). The PWM period is $n/8$ rounded (L859), so 8 PWM periods still fit one sample. In a critical section (L860–867), it hands both values to the TIM2 ISR and, if the PWM period changed, updates TIM4's `ARR`. The critical section guarantees the ISR never sees a new `n` with an old `frac`.

23 UART: telemetry and commands

23.1 UART set-up, DMA transmit and its interrupt (L870–912)

```
870 /* ========================================================================== */
871 /*  UART & Command Parsing                                                    */
872 /* ========================================================================== */
873 static void uart_init(void)
874 {
875     RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
876     (void)RCC->APB1ENR;
877     USART2->CR1 = 0;
878     USART2->BRR = (APB1_HZ + UART_BAUD / 2u) / UART_BAUD;
879     USART2->CR3 = USART_CR3_DMAT;
880     USART2->CR1 = USART_CR1_TE | USART_CR1_RE | USART_CR1_RXNEIE | USART_CR1_UE;
881
882     DMA1_Stream6->CR = 0;
883     while (DMA1_Stream6->CR & DMA_SxCR_EN) {}
884     DMA1_Stream6->PAR = (uint32_t)&USART2->DR;
885     DMA1_Stream6->FCR = 0;
886     DMA1_Stream6->CR  = (4u << DMA_SxCR_CHSEL_Pos) | DMA_SxCR_MINC |
887                         DMA_SxCR_DIR_0 | DMA_SxCR_TCIE | DMA_SxCR_TEIE;
888 }
889
890 static bool uart_tx_dma(uint32_t len)
891 {
892     if (tx_busy || len == 0u) return false;
893     tx_busy = true;
894     DMA1_Stream6->CR &= ~DMA_SxCR_EN;
895     while (DMA1_Stream6->CR & DMA_SxCR_EN) {}
896     DMA1->HIFCR = DMA_HIFCR_CTCIF6 | DMA_HIFCR_CHTIF6 | DMA_HIFCR_CTEIF6 |
897                   DMA_HIFCR_CDMEIF6 | DMA_HIFCR_CFEIF6;
898     DMA1_Stream6->M0AR = (uint32_t)tx_buf;
899     DMA1_Stream6->NDTR = len;
900     USART2->SR &= ~USART_SR_TC;
901     DMA1_Stream6->CR |= DMA_SxCR_EN;
902     return true;
903 }
904
905 void DMA1_Stream6_IRQHandler(void)
906 {
907     uint32_t isr = DMA1->HISR;
908     if (isr & (DMA_HISR_TCIF6 | DMA_HISR_TEIF6)) {
909         DMA1->HIFCR = DMA_HIFCR_CTCIF6 | DMA_HIFCR_CTEIF6;
910         tx_busy = false;
911     }
912 }
```

- L875–876: USART2 clock on.
- L877: disable the UART while configuring.
- L878: baud divider = 278 (115 108 bit/s, Chapter 11); `+ UART_BAUD/2` rounds to nearest.
- L879: `DMAT`: the UART requests DMA for transmission.
- L880: transmitter, receiver, receive interrupt, UART enable. Default frame: 8 data bits, no parity, 1 stop bit.
- L882–887: DMA1 Stream 6: disable and wait, peripheral address = UART data register, direct mode, channel 4 (USART2_TX), memory increment, direction memory-to-peripheral (`DIR_0`),

transfer-complete and error interrupts.

- `uart_tx_dma(len)` (L890–903): refuse if a transfer is still running (L892), mark busy, stop the stream, clear its flags, set the source (`tx_buf`) and length, clear the UART's "transmission complete" flag, start.
- `DMA1_Stream6_IRQHandler` (L905–912): when finished (or on error), clear the flags and mark not busy.

23.2 Command reception and parsing (L914–981)

```
914 static const char *parse_u(const char *p, uint32_t *v)
915 {
916     if (*p < '0' || *p > '9') return 0;
917     uint32_t x = 0;
918     while (*p >= '0' && *p <= '9') {
919         x = x * 10u + (uint32_t)(*p++ - '0');
920         if (x > 1000u) return 0;
921     }
922     *v = x;
923     return p;
924 }
925
926 static void cmd_exec(const char *s)
927 {
928     const char *p;
929     uint32_t a, b, c;
930
931     if (s[1] == 'C' && s[2] == ',') {
932         if (!(p = parse_u(s + 3, &a)) || *p++ != ',') goto bad;
933         if (!(p = parse_u(p, &b))     || *p++ != ',') goto bad;
934         if (!(p = parse_u(p, &c))     || *p != '\0')  goto bad;
935         if (a > 2u || b > 1u || c > 1u) goto bad;
936
937         uint8_t bit = (uint8_t)(1u << a);
938         if (b) {
939             lock_mask  |= bit;
940             lock_state  = c ? (lock_state | bit) : (lock_state & (uint8_t)~bit);
941         } else {
942             lock_mask  &= (uint8_t)~bit;
943             lock_state &= (uint8_t)~bit;
944         }
945         pfc_cmd_pending = true;          /* main loop applies it at once */
946         cmd_ok++;
947         return;
948     }
949     if (s[1] == 'A' && s[2] == ',') {
950         if (!(p = parse_u(s + 3, &a)) || *p != '\0' || a > 1u) goto bad;
951         ahf_active = a;
952         cmd_ok++;
953         return;
954     }
955 bad:
956     cmd_bad++;
957 }
958
959 static void rx_feed(char ch)
960 {
961     if (ch == '$') { rx_line[0] = ch; rx_len = 1; return; }
962     if (ch == '\n' || ch == '\r') {
963         if (rx_len) { rx_line[rx_len] = '\0'; cmd_exec(rx_line); }
964         rx_len = 0;
965         return;
```

```
966     }
967     if (rx_len == 0u) return;
968     if (rx_len < RX_LINE_MAX - 1u) rx_line[rx_len++] = ch;
969     else { rx_len = 0; cmd_bad++; }
970 }
971
972 void USART2_IRQHandler(void)
973 {
974     uint32_t sr = USART2->SR;
975     if (sr & (USART_SR_RXNE | USART_SR_ORE | USART_SR_FE | USART_SR_NE)) {
976         char ch = (char)USART2->DR;
977         rx_bytes++;
978         if (sr & (USART_SR_FE | USART_SR_NE)) { rx_len = 0; return; }
979         rx_feed(ch);
980     }
981 }
```

- `parse_u` (L914–924): reads an unsigned decimal number. Returns `NULL` (0) if there is no digit or the value exceeds 1000 (overflow guard). Otherwise returns a pointer to the first character after the number.
- `cmd_exec` (L926–957):
  - L931–935: `$C,a,b,c`: three numbers separated by commas and nothing after. Range check: step $a \leq 2$, mode $b \leq 1$, state $c \leq 1$. Any violation jumps to `bad`.
  - L937–944: `b=1` (manual) sets the step's bit in `lock_mask` and sets or clears it in `lock_state`; `b=0` (automatic) clears both.
  - L945–947: count OK. The relay itself is switched by `relay_service` in the main loop (not in this ISR), so all relay rules apply.
  - L949–954: `$A,e`: AHF on/off.
  - L955–956: anything else: count as a bad command.
- `rx_feed` (L959–970): a small line assembler. `$` always starts a new line (L961); newline or carriage return ends it and executes it (L962–966); characters before any `$` are ignored (L967); an over-long line is discarded (L968–969).
- `USART2_IRQHandler` (L972–981): read the status, then the data register. Reading `SR` then `DR` clears RXNE and the overrun flag. On a framing or noise error, discard the partial line (L978); otherwise feed the character.

24 Signal processing: `process_window`

This function turns one window of raw samples (1000 sets of I, V, A) into every measured quantity. It is the most important function for the panel, so it is shown in five parts.

24.1 Part 1: folding and the sensor-A filter (L983–1018)

```
983 /* ========================================================================== */
984 /*  Signal Processing                                                         */
985 /* ========================================================================== */
986 static void process_window(const uint16_t *s, win_t *m)
987 {
988     static uint32_t csum[N_CH][N_CYC];
989     static float    csa[N_CYC];             /* sensor A after the low-pass    */
990     uint32_t sum[N_CH] = { 0 };
991 
992     memset(csum, 0, sizeof csum);
993     memset(csa, 0, sizeof csa);
994     if (!lpf_primed) {                      /* start in steady state: no step */
995         const float x0 = (float)s[SLOT_A];
996         lpf_z2 = (a_lpf.b2 - a_lpf.a2) * x0;
997         lpf_z1 = (a_lpf.b1 - a_lpf.a1) * x0 + lpf_z2;
998         lpf_primed = true;
999     }
1000    for (uint32_t c = 0; c < N_CH; c++) { m->mn[c] = 0xFFFFu; m->mx[c] = 0; m->clip[c] = 0; }
1001
1002    for (uint32_t n = 0, k = 0; n < N_WIN; n++) {
1003        const uint16_t *x = &s[n * N_CH];
1004        for (uint32_t c = 0; c < N_CH; c++) {
1005            uint16_t r = x[c];
1006            sum[c]     += r;
1007            csum[c][k] += r;
1008            if (r < m->mn[c]) m->mn[c] = r;
1009            if (r > m->mx[c]) m->mx[c] = r;
1010            if (r <= ADC_CLIP_LO || r >= ADC_CLIP_HI) m->clip[c]++;
1011        }
1012        const float xa = (float)x[SLOT_A];                  /* Butterworth, DF2T */
1013        const float ya = a_lpf.b0 * xa + lpf_z1;
1014        lpf_z1 = a_lpf.b1 * xa - a_lpf.a1 * ya + lpf_z2;
1015        lpf_z2 = a_lpf.b2 * xa - a_lpf.a2 * ya;
1016        csa[k] += ya;
1017        if (++k == N_CYC) k = 0;
1018    }
```

- L988–990: `csum[ch][k]` will hold the sum of the 5 samples at position k of each cycle (for folding); `csa[k]` the same for the filtered sensor-A signal; `sum[ch]` the sum of all 1000 samples (for the offset). `static` keeps the 2.4 KB array off the stack.
- L992–993: clear the folding sums.
- L994–999: on the first window after a restart, preset the filter state to the steady state for a constant input equal to the first sample. Without this the filter would start from 0 and show a large fake transient. (For a DC input x_0: $z_2 = (b_2 - a_2)x_0$, $z_1 = (b_1 - a_1)x_0 + z_2$.)
- L1000: reset the min/max/clip statistics.
- L1002–1018: one pass over the 1000 sample sets. n is the sample number; k is the position in the cycle (0–199, wrapping at L1017).

- L1003: pointer to sample set n (interleaved I, V, A).
- L1006–1007: add to the total sum and to the folding sum at position k.
- L1008–1010: track minimum, maximum and samples near the ADC limits (clipping).
- L1012–1016: run the 1 kHz Butterworth on the raw sensor-A sample (DF2T) and fold the output.

24.2 Part 2: offsets, rms, noise separation, power (L1019–1067)

```
1019    for (uint32_t c = 0; c < N_CH; c++) m->off[c] = (float)sum[c] / (float)N_WIN;
1020
1021    const float oi = m->off[SLOT_I], ov = m->off[SLOT_V], oa = m->off[SLOT_A];
1022    float si2 = 0, sv2 = 0, svi = 0, ipk = 0, vpk = 0;
1023    for (uint32_t n = 0; n < N_WIN; n++) {
1024        const uint16_t *x = &s[n * N_CH];
1025        float fi = ((float)x[SLOT_I] - oi) * K_I;
1026        float fv = ((float)x[SLOT_V] - ov) * K_V;
1027        si2 += fi * fi;
1028        sv2 += fv * fv;
1029        svi += fv * fi;
1030        fi = fabsf(fi); fv = fabsf(fv);
1031        if (fi > ipk) ipk = fi;
1032        if (fv > vpk) vpk = fv;
1033    }
1034    m->vrms = sqrtf(sv2 / (float)N_WIN);
1035
1036    /* Grid current: separate the part that repeats every cycle (the load)
1037     * from the part that does not (ACS712 + ADC noise, ~30 mA rms). Over K
1038     * averaged cycles  raw^2 = S^2 + N^2  and  averaged^2 = S^2 + N^2/K,
1039     * so S^2 = (K*averaged^2 - raw^2)/(K-1). Noise is uncorrelated with the
1040     * voltage, so P is unaffected; Irms, S, PF and D no longer include it.
1041     * The peak is taken from the averaged cycle for the same reason.       */
1042    {
1043        const float inv_k = 1.0f / (float)CYC_PER_WIN;
1044        float c2 = 0.0f, cpk = 0.0f;
1045        for (uint32_t k = 0; k < N_CYC; k++) {
1046            const float d = ((float)csum[SLOT_I][k] * inv_k - oi) * K_I;
1047            c2 += d * d;
1048            if (fabsf(d) > cpk) cpk = fabsf(d);
1049        }
1050        c2 /= (float)N_CYC;
1051        const float r2 = si2 / (float)N_WIN;
1052        float s2 = ((float)CYC_PER_WIN * c2 - r2) / (float)(CYC_PER_WIN - 1u);
1053        if (s2 < 0.0f) s2 = 0.0f;
1054        m->i_noise = (r2 > s2) ? sqrtf(r2 - s2) : 0.0f;
1055        m->irms    = sqrtf(s2);
1056        ipk        = cpk;
1057    }
1058
1059    if (m->vrms < V_DEADBAND) { m->vrms = 0.0f; vpk = 0.0f; }
1060    if (m->irms < I_DEADBAND) { m->irms = 0.0f; ipk = 0.0f; }
1061
1062    m->p     = (m->vrms > 0.0f && m->irms > 0.0f) ? (svi / (float)N_WIN) : 0.0f;
1063    m->s     = m->vrms * m->irms;
1064    m->pf    = (m->s > 1e-3f) ? m->p / m->s : 0.0f;
1065    m->ipk   = ipk;
1066    m->vpk   = vpk;
1067    m->crest = (m->irms > 1e-4f) ? ipk / m->irms : 0.0f;
```

- L1019: offset of each channel = mean of 1000 samples. Exact, because the window holds 5 whole cycles.

- L1022–1033: second pass over the raw samples. Convert to amperes and volts (offset removed, scaled by K), and accumulate $\sum i^2$, $\sum v^2$, $\sum vi$ and the raw peaks.
- L1034: V_{rms} from raw samples.
- L1042–1057: **noise separation** for the current (Chapter 12). L1045–1050: mean square of the averaged cycle and its peak. L1051: raw mean square. L1052: $S^2 = (5 \cdot \overline{\bar{x}^2} - \overline{x^2})/4$. L1054: noise rms (sent to the dashboard). L1055: I_{rms} without noise. L1056: peak current from the averaged cycle.
- L1059–1060: deadbands.
- L1062: active power $P = \overline{vi}$ from raw samples, only if both voltage and current are present.
- L1063–1067: apparent power, power factor (guarded against division by tiny S), peaks and crest factor (I_{pk}/I_{rms}; a sine has 1.41).

24.3 Part 3: averaged waveforms, DFT, sensor-A correction (L1069–1120)

```
1069    const float inv = 1.0f / (float)CYC_PER_WIN;
1070    float apk = 0.0f;
1071    for (uint32_t k = 0; k < N_CYC; k++) {
1072        w_i[k] = (m->irms > 0.0f) ? (((float)csum[SLOT_I][k] * inv - oi) * K_I) : 0.0f;
1073        w_v[k] = (m->vrms > 0.0f) ? (((float)csum[SLOT_V][k] * inv - ov) * K_V) : 0.0f;
1074        w_a[k] = (csa[k] * inv - oa) * K_A;
1075        if (fabsf(w_a[k]) > apk) apk = fabsf(w_a[k]);
1076    }
1077
1078    dft(w_i, ai, bi);
1079    dft(w_v, av, bv);
1080    dft(w_a, aa, ba);
1081    for (uint32_t h = 1; h <= MAX_H; h++) {     /* undo the filter per harmonic */
1082        const cpx_t x = c_div(c_phasor(aa[h], ba[h]), a_lpf_h[h]);
1083        aa[h] = x.re;
1084        ba[h] = -x.im;
1085    }
1086
1087    /* Noise floor (telemetry only; the AHF loop always uses aa/ba): while
1088     * the bridge is not driving, a small sensor-A reading is pure noise and
1089     * is shown as 0. While it drives, the real current is shown however
1090     * small - the charger's harmonics need only ~0.04 A.                    */
1091    float a2 = 0.0f;
1092    for (uint32_t k = 0; k < N_CYC; k++) a2 += w_a[k] * w_a[k];
1093    const bool driving = ahf_active && ahf_duty_pk > 0.005f;
1094    const bool a_on    = driving || sqrtf(a2 / (float)N_CYC) >= A_DEADBAND;
1095
1096    /* Shown waveform = the filter-corrected harmonics h1..h15: in time with
1097     * sensor I and the target, without the filter's lag or the noise.      */
1098    memset(w_a, 0, sizeof w_a);
1099    apk = 0.0f;
1100    if (a_on) {
1101        for (uint32_t h = 1; h <= MAX_H; h++) {
1102            uint32_t idx = 0;
1103            for (uint32_t k = 0; k < N_CYC; k++) {
1104                w_a[k] += aa[h] * cos_t[idx] + ba[h] * sin_t[idx];
1105                idx += h;
1106                if (idx >= N_CYC) idx -= N_CYC;
1107            }
1108        }
1109        for (uint32_t k = 0; k < N_CYC; k++) if (fabsf(w_a[k]) > apk) apk = fabsf(w_a[k]);
1110    }
1111    m->apk = apk;
```

```
1112     float hv[MAX_H + 1];
1113     for (uint32_t h = 1; h <= MAX_H; h++) {
1114         m->h_i[h] = 0.70710678f * sqrtf(ai[h] * ai[h] + bi[h] * bi[h]);
1115         m->h_a[h] = a_on ? 0.70710678f * sqrtf(aa[h] * aa[h] + ba[h] * ba[h]) : 0.0f;
1116         hv[h]     = 0.70710678f * sqrtf(av[h] * av[h] + bv[h] * bv[h]);
1117     }
1118     m->h_i[0] = m->h_a[0] = 0.0f;
1119     m->thd_i = (m->irms > 0.0f) ? thd_pct(m->h_i) : 0.0f;
1120     m->thd_v = (m->vrms > 0.0f) ? thd_pct(hv)     : 0.0f;
```

- L1071–1076: averaged cycles in physical units: sum/5 minus offset, times K. Zeroed if the deadband removed that signal.
- L1078–1080: DFT of current, voltage and AHF current, harmonics 1–15.
- L1081–1085: divide each sensor-A phasor by the filter response at that harmonic, which exactly undoes the 1 kHz Butterworth.
- L1091–1094: decide whether sensor A shows real current: the bridge is driving (active and duty above 0.5 %), or its rms exceeds 50 mA.
- L1098–1110: rebuild the displayed AHF waveform from the corrected harmonics h1–h15. It is noise-free, without filter delay, and time-aligned with sensor I. Zero if not `a_on`.
- L1113–1117: rms of every harmonic ($\sqrt{a^2+b^2}/\sqrt{2}$, i.e. $\times 0.7071$) for current, AHF current and voltage.
- L1119–1120: THD of current and voltage.

24.4 Part 4: fundamental powers and the compensated spectrum (L1122–1150)

```
1122     if (m->vrms > 0.0f && m->irms > 0.0f) {
1123         const float ph = ZMPT_LAG_DEG * 0.01745329f -
1124                          TWO_PI * pll_f_est * ADC_SLOT_S * (float)(SLOT_V - SLOT_I);
1125         const float cp = cosf(ph), sp = sinf(ph);
1126         const float a1v = av[1] * cp + bv[1] * sp;
1127         const float b1v = bv[1] * cp - av[1] * sp;
1128         m->p1  = 0.5f * (a1v * ai[1] + b1v * bi[1]);
1129         m->q1  = 0.5f * (a1v * bi[1] - b1v * ai[1]);
1130         const float s1 = sqrtf(m->p1 * m->p1 + m->q1 * m->q1);
1131         m->dpf = (s1 > 1e-3f) ? m->p1 / s1 : 0.0f;
1132         const float d2 = m->s * m->s - m->p * m->p - m->q1 * m->q1;
1133         m->dva = (d2 > 0.0f) ? sqrtf(d2) : 0.0f;
1134     } else {
1135         m->p1 = m->q1 = m->dpf = m->dva = 0.0f;
1136     }
1137
1138     /* Grid current after compensation, per harmonic, from the exact phasors
1139      * (sent as $H,C so the dashboard does not have to use a smoothed curve). */
1140     for (uint32_t h = 1; h <= MAX_H; h++) {
1141 #if AHF_SENSE_GRID
1142         const float a = ai[h], b = bi[h];
1143 #else
1144         const float a = ai[h] + (a_on ? AHF_INJ_RATIO * aa[h] : 0.0f);
1145         const float b = bi[h] + (a_on ? AHF_INJ_RATIO * ba[h] : 0.0f);
1146 #endif
1147         m->h_c[h] = 0.70710678f * sqrtf(a * a + b * b);
1148     }
1149     m->h_c[0] = 0.0f;
1150     m->thd_c  = (m->irms > 0.0f) ? thd_pct(m->h_c) : 0.0f;
```

- L1123–1124: angle correcting the voltage for being sampled 30.75 μs after the current (plus optional ZMPT lag): $\varphi = -2\pi f \cdot T_{\text{slot}}$.
- L1125–1127: rotate the voltage phasor by that angle (complex multiplication written out).
- L1128–1129: P_1 and Q_1 (Chapter 12).
- L1130–1131: DPF $= P_1/|S_1|$.
- L1132–1133: distortion power $D = \sqrt{S^2 - P^2 - Q_1^2}$.
- L1140–1150: the grid current *after* compensation, per harmonic, from exact phasors: $I_h + n\,M_h$ (load-side sensor) or I_h (grid-side sensor); its THD is sent as "THD after".

24.5 Part 5: display smoothing, target and compensated waveform (L1152–1166)

```
1152     /* Display waveforms: zero-phase Butterworth removes ADC/sensor noise.
1153      * Every number above was computed from the unfiltered data.            */
1154     if (m->vrms > 0.0f) lpf_zero_phase(&vi_lpf, w_v);
1155     if (m->irms > 0.0f) lpf_zero_phase(&vi_lpf, w_i);
1156
1157     for (uint32_t k = 0; k < N_CYC; k++) {
1158         float i_fund = ai[1] * cos_t[k] + bi[1] * sin_t[k];
1159         w_t[k] = -(w_i[k] - i_fund);
1160 #if AHF_SENSE_GRID
1161         w_c[k] = w_i[k];                                    /* already the grid */
1162 #else
1163         w_c[k] = w_i[k] + AHF_INJ_RATIO * w_a[k];           /* load + injection */
1164 #endif
1165     }
1166 }
```

- L1154–1155: zero-phase 1.5 kHz Butterworth on the displayed voltage and current waveforms. This happens *after* all numbers have been computed.
- L1157–1165: for display: the AHF target $-(i - i_1)$ (current minus its fundamental, inverted), and the grid current after compensation $i + n\,i_a$.

25 Control: AHF step and APFC

25.1 ahf_step (L1168–1188)

```
1168 /* clean: the AHF table did not change during this window, so sensor A shows
1169  * the response to exactly one known table (needed for identification).    */
1170 static void ahf_step(bool clean)
1171 {
1172     float pk = 0.0f;
1173     for (uint32_t k = 0; k < N_CYC; k++) if (fabsf(w_t[k]) > pk) pk = fabsf(w_t[k]);
1174     ahf_ref_pk_f = (ahf_ref_pk_f <= 0.0f) ? pk
1175                  : (1.0f - AHF_PK_ALPHA) * ahf_ref_pk_f + AHF_PK_ALPHA * pk;
1176
1177     if (!ahf_active) {
1178         if (!ahf_is_reset) { ahf_swap = 0u; ahf_reset(); ahf_is_reset = true; }
1179         return;
1180     }
1181     ahf_is_reset = false;
1182     if (!clean || ahf_swap) return;
1183
1184     const bool drive = (mw.irms > 0.0f) && (ahf_ref_pk_f >= AHF_MIN_HARM_A);
1185     ahf_control(ai, bi, aa, ba, ahf_sel, drive);
1186     __DMB();
1187     ahf_swap = 1u;
1188 }
```

- L1172–1175: peak of the target waveform, smoothed (30 % new). Reported as “target peak”.
- L1177–1180: in STANDBY, reset the AHF once (cancel any pending table switch, clear the tables) and return.
- L1182: run the control step only on a *clean* window, and only if the previous table has already been taken by the ISR.
- L1184: drive only when a load is present and there is distortion worth cancelling ($\geq$0.02 A peak).
- L1185: the control step writes the new table.
- L1186–1187: memory barrier, then tell the ISR a new table is ready. It is switched in at the next cycle start.

25.2 APFC helpers (L1190–1237)

```
1190 static float cap_sum(uint8_t st)
1191 {
1192     return ((st & 1u) ? cap_uF[0] : 0.0f) + ((st & 2u) ? cap_uF[1] : 0.0f) +
1193            ((st & 4u) ? cap_uF[2] : 0.0f);
1194 }
1195
1196 static uint32_t popcnt3(uint8_t x) { return (x & 1u) + ((x >> 1) & 1u) + ((x >> 2) & 1u); }
1197
1198 static void acc_reset(void) { memset(&acc, 0, sizeof acc); }
1199
1200 static void acc_add(const win_t *m)
1201 {
1202     acc.sv2 += m->vrms * m->vrms;
1203     acc.si2 += m->irms * m->irms;
1204     acc.p   += m->p;
1205     acc.p1  += m->p1;
```

```
1206     acc.q1  += m->q1;
1207     if (m->vpk > acc.vpk) acc.vpk = m->vpk;
1208     if (m->ipk > acc.ipk) acc.ipk = m->ipk;
1209     acc.n++;
1210 }
1211
1212 static void make_avg(avg_t *a, const win_t *m)
1213 {
1214     if (acc.n == 0u) {
1215         a->vrms = m->vrms; a->irms = m->irms; a->p = m->p; a->p1 = m->p1; a->q1 = m->q1;
1216         a->vpk = m->vpk;   a->ipk = m->ipk;
1217     } else {
1218         const float n = (float)acc.n;
1219         a->vrms = sqrtf(acc.sv2 / n); a->irms = sqrtf(acc.si2 / n);
1220         a->p = acc.p / n; a->p1 = acc.p1 / n; a->q1 = acc.q1 / n;
1221         a->vpk = acc.vpk; a->ipk = acc.ipk;
1222     }
1223     a->s   = a->vrms * a->irms;
1224     a->pf  = (a->s > 1e-3f) ? a->p / a->s : 0.0f;
1225     const float s1 = sqrtf(a->p1 * a->p1 + a->q1 * a->q1);
1226     a->dpf = (s1 > 1e-3f) ? a->p1 / s1 : 0.0f;
1227     const float d2 = a->s * a->s - a->p * a->p - a->q1 * a->q1;
1228     a->dva = (d2 > 0.0f) ? sqrtf(d2) : 0.0f;
1229 }
1230
1231 static void locks_read(uint8_t *mask, uint8_t *st)
1232 {
1233     __disable_irq();
1234     *mask = lock_mask & 7u;
1235     *st   = lock_state & 7u;
1236     __enable_irq();
1237 }
```

- `cap_sum(st)`: total capacitance of a 3-bit combination (bit 0 = 2.5, bit 1 = 10, bit 2 = 20 μF).
- `popcnt3(x)`: number of set bits; applied to `st ^ cur`, it counts the relays that would have to change.
- `acc_reset / acc_add`: sum the window results over 0.5 s. Rms values are averaged as mean squares (L1202–1203), which is the correct way to average rms values. Powers are averaged directly, and peaks keep the maximum.
- `make_avg` (L1212–1229): the 0.5 s average (or the last window if none were accumulated), then S, PF, DPF and D from the averaged values.
- `locks_read` (L1231–1237): read both lock bytes together with interrupts off, because the USART ISR may change them at any moment.

25.3 `apfc_step` (L1239–1282)

```
1239 static void apfc_step(const avg_t *a)
1240 {
1241     uint8_t mask, lst;
1242     locks_read(&mask, &lst);
1243
1244     if (mask == 7u) {
1245         relay_target = lst;
1246         pfc_hold = 1;
1247         pfc_c_need = cap_sum(lst);
1248         pfc_reason = 3u;
1249         return;
1250     }
```

```
1251
1252     const uint8_t cur = (uint8_t)((relay_target & ~mask) | (lst & mask));
1253
1254     float c_need = 0.0f;
1255     if (a->vrms >= APFC_V_MIN && fabsf(a->p1) >= APFC_P_MIN) {
1256         const float k      = TWO_PI * pll_f_est * a->vrms * a->vrms * 1e-6f;
1257         const float q_load = a->q1 + k * cap_sum(relay_state);
1258         const float q_tgt  = fabsf(a->p1) * tan_phi_tgt;
1259         c_need = (q_load - q_tgt) / k;
1260         if (c_need < 0.0f) c_need = 0.0f;
1261     }
1262     pfc_c_need = c_need;
1263
1264     const float e_cur = fabsf(cap_sum(cur) - c_need);
1265     uint8_t best = cur;
1266     float   e_best = e_cur;
1267     for (uint8_t st = 0; st < 8u; st++) {
1268         if ((st & mask) != (lst & mask)) continue;
1269         const float e = fabsf(cap_sum(st) - c_need);
1270         if (e < e_best - 0.01f ||
1271             (e <= e_best + 0.01f && popcnt3(st ^ cur) < popcnt3(best ^ cur))) {
1272             best = st; e_best = e;
1273         }
1274     }
1275
1276     if (best != cur && e_cur > C_DEADBAND_UF) { relay_target = (uint8_t)((best & ~mask) | (lst &
        mask)); pfc_hold = 0; }
1277     else                                      { relay_target = cur;  pfc_hold = 1; }
1278
1279     if (a->vrms < APFC_V_MIN || fabsf(a->p1) < APFC_P_MIN) pfc_reason = 1u;
1280     else if (pfc_hold && cur == 0u && c_need > 0.0f)       pfc_reason = 2u;
1281     else                                                   pfc_reason = 0u;
1282 }
```

- L1244–1250: all 3 steps manual: the target is simply the locked state; reason 3.
- L1252: the present combination, with manual locks applied.
- L1255–1261: the C_{need} formula of Chapter 15. k uses the measured grid frequency (L1256); L1257 adds back the reactive power of the capacitors that are ON (`relay_state` is what is physically ON); L1258 is the reactive power allowed at PF 0.99.
- L1264–1274: search all 8 combinations that respect the locks (L1268). Prefer a smaller error by more than 0.01 μF; for practically equal errors, prefer fewer relay changes (L1270–1271).
- L1276–1277: switch only if a better combination exists and the present error exceeds 0.75 μF. `pfc_hold` tells the dashboard SWITCHING or HOLD.
- L1279–1281: hold reason for the dashboard: 1 = no load, 2 = the load needs less than half the smallest step, 0 = normal.

25.4 `relay_service` (L1284–1333)

```
1284 static void relay_service(uint32_t now)
1285 {
1286     uint8_t mask, lst;
1287     locks_read(&mask, &lst);
1288
1289     relay_target = (uint8_t)((relay_target & ~mask) | (lst & mask));
1290
1291     /* With no mains voltage there is nothing to time against and nothing
1292      * to protect: switch at once. Otherwise relay modules wait for the PLL. */
```

```
1293     const bool sync = RELAY_SYNC && grid_v_present;
1294     uint8_t st = relay_state, now_bits = 0, at_on = 0, at_on0 = 0, at_off = 0;
1295     relay_waiting &= (uint8_t)(relay_target & ~st);
1296     for (uint32_t i = 0; i < 3u; i++) {
1297         const uint8_t b    = (uint8_t)(1u << i);
1298         const bool    timed = sync && !(RELAY_SSR_MASK & b);
1299         if ((relay_target & b) && !(st & b)) {
1300             if ((now - relay_off_ms[i]) < CAP_RECLOSE_MS) continue;   /* min OFF  */
1301             bool at_wave = timed;
1302             if (timed && !pll_locked) {           /* wait for the peak timing ...  */
1303                 if (!(relay_waiting & b)) { relay_waiting |= b; relay_wait_ms[i] = now; }
1304                 if ((now - relay_wait_ms[i]) < RELAY_LOCK_WAIT_MS) continue;
1305                 at_wave = false;                  /* ... but not forever          */
1306             }
1307             relay_waiting &= (uint8_t)~b;
1308             st |= b;
1309             relay_on_ms[i] = now;
1310             if (!at_wave) { now_bits |= b; relay_v_known &= (uint8_t)~b; }
1311             else if (relay_v_known & b) at_on  |= b;
1312             else                        at_on0 |= b;
1313         } else if (!(relay_target & b) && (st & b)) {
1314             if ((now - relay_on_ms[i]) < RELAY_MIN_ON_MS) continue;   /* min ON   */
1315             st &= (uint8_t)~b;
1316             relay_off_ms[i] = now;
1317             if (timed && pll_locked) { at_off |= b; relay_v_known |= b; }
1318             else { now_bits |= b; relay_v_known &= (uint8_t)~b; }
1319         }
1320     }
1321     if (st != relay_state) {
1322         __disable_irq();
1323         const uint8_t on_any = (uint8_t)(at_on | at_on0);
1324         relay_pend_on  = (uint8_t)((relay_pend_on  & ~(now_bits | at_off | at_on0)) | at_on);
1325         relay_pend_on0 = (uint8_t)((relay_pend_on0 & ~(now_bits | at_off | at_on))  | at_on0);
1326         relay_pend_off = (uint8_t)((relay_pend_off & ~(now_bits | on_any)) | at_off);
1327         if (now_bits) GPIOB->BSRR = relay_bsrr(now_bits, st);
1328         __enable_irq();
1329         relay_state = st;
1330         acc_reset();
1331         settle_win = PFC_SETTLE_WIN;
1332     }
1333 }
```

Called on every pass of the main loop, so commands take effect within microseconds. It turns the *wish* (`relay_target`) into the *action*, applying all safety rules.

- L1289: manual locks override the automatic target.
- L1293: time the switching only when mains is present (otherwise there is no peak to time against, and nothing to protect).
- L1295: forget "waiting for lock" for steps that no longer need to close.
- L1299–1312: a step must **close**:
  - L1300: not before 5 s after its last opening (minimum OFF time).
  - L1302–1306: a relay module waits for the PLL lock (to know where the peak is), but at most 2 s.
  - L1308–1309: mark it ON and remember when.
  - L1310–1312: untimed (SSR, no mains, or no lock): switch now, charge unknown. Timed with known charge ($+V_{pk}$ from a timed opening): close at the peak. Timed with unknown charge: close at the zero crossing.
- L1313–1319: a step must **open**: not before 5 s ON (L1314). If timed and locked, open at the

planned instant and remember the capacitor is now at $+V_{pk}$; otherwise open now, charge unknown.

- L1321–1332: if anything changed, with interrupts off: update the three schedules for the TIM2 ISR. A new action on a step cancels any older pending action on it. Pins that switch immediately are written now via BSRR. Then update the state, restart the 0.5 s averaging, and ignore the next 2 windows while the transient settles.

26 Telemetry, start-up and the main loop

26.1 Text formatting helpers (L1335–1372)

```
1335 /* ========================================================================== */
1336 /*  Telemetry                                                                 */
1337 /* ========================================================================== */
1338 static void tx_c(char c) { if (tx_p < tx_buf + TX_BUF_SZ) *tx_p++ = c; }
1339 static void tx_s(const char *s) { while (*s) tx_c(*s++); }
1340
1341 static void tx_u(uint32_t v)
1342 {
1343     char t[10];
1344     uint32_t n = 0;
1345     do { t[n++] = (char)('0' + v % 10u); v /= 10u; } while (v);
1346     while (n) tx_c(t[--n]);
1347 }
1348
1349 static void tx_f(float x, uint32_t dec)
1350 {
1351     static const uint32_t p10[] = { 1u, 10u, 100u, 1000u, 10000u };
1352     if (!(x == x) || x > 2.0e5f || x < -2.0e5f) { tx_c('0'); return; }
1353     const uint32_t sc = p10[dec];
1354     int32_t v = (int32_t)(x * (float)sc + (x >= 0.0f ? 0.5f : -0.5f));
1355     if (v < 0) { tx_c('-'); v = -v; }
1356     tx_u((uint32_t)v / sc);
1357     if (dec) {
1358         uint32_t fr = (uint32_t)v % sc;
1359         tx_c('.');
1360         for (uint32_t d = sc / 10u; d; d /= 10u) { tx_c((char)('0' + fr / d)); fr %= d; }
1361     }
1362 }
1363
1364 static void tx_cf(float x, uint32_t dec) { tx_c(','); tx_f(x, dec); }
1365 static void tx_cu(uint32_t v)            { tx_c(','); tx_u(v); }
1366
1367 static void tx_wave(char tag, const float *w, uint32_t dec)
1368 {
1369     tx_s("$W,"); tx_c(tag);
1370     for (uint32_t k = 0; k < N_CYC; k += WAVE_DECIM) tx_cf(w[k], dec);
1371     tx_c('\n');
1372 }
```

- `tx_c`: append one character, never beyond the buffer (overflow-safe).
- `tx_s`: append a string.
- `tx_u`: unsigned integer to decimal text. Digits are produced in reverse into a small array, then copied out in the right order.
- `tx_f`: float to text with a fixed number of decimals, without `printf` (which is large and slow on a microcontroller). NaN or out-of-range values print as 0 (L1352); otherwise the value is scaled by 10^{dec} and rounded (L1354), the sign is printed, then the integer part and the zero-padded fraction.
- `tx_cf / tx_cu`: comma followed by a float or an integer.
- `tx_wave` (L1367–1372): `$W,<tag>` followed by every 2nd sample of a cycle (100 values).

26.2 telemetry_send (L1374–1419)

```
1374 static void telemetry_send(const win_t *m, const avg_t *a)
1375 {
1376     if (tx_busy) { tx_skip++; return; }
1377     tx_p = tx_buf;
1378 
1379     tx_wave('V', w_v, 1);
1380     tx_wave('I', w_i, 3);
1381     tx_wave('A', w_a, 3);
1382     tx_wave('T', w_t, 3);
1383     tx_wave('C', w_c, 3);
1384 
1385     tx_s("$M");
1386     tx_cf(a->vrms, 2); tx_cf(a->irms, 4); tx_cf(a->p, 2);   tx_cf(a->s, 2);
1387     tx_cf(a->q1, 2);   tx_cf(a->pf, 4);   tx_cf(m->thd_v, 2); tx_cf(m->thd_i, 2);
1388     tx_cf(m->crest, 3); tx_cf(a->vpk, 1); tx_cf(a->ipk, 4); tx_cf(a->dpf, 4);
1389     tx_cf(a->dva, 2);  tx_cf(pll_f_est, 3);
1390     tx_c('\n');
1391 
1392     tx_s("$H,I"); for (uint32_t h = 1; h <= MAX_H; h++) tx_cf(m->h_i[h], 4); tx_c('\n');
1393     tx_s("$H,A"); for (uint32_t h = 1; h <= MAX_H; h++) tx_cf(m->h_a[h], 4); tx_c('\n');
1394     tx_s("$H,C"); for (uint32_t h = 1; h <= MAX_H; h++) tx_cf(m->h_c[h], 4); tx_c('\n');
1395 
1396     uint8_t mask, lst;
1397     locks_read(&mask, &lst);
1398     tx_s("$P"); tx_cu(relay_state); tx_cf(cap_sum(relay_state), 1); tx_cf(a->pf, 4);
1399     tx_cf(a->q1, 2); tx_cu(pfc_hold); tx_cu(relay_target); tx_cf(pfc_c_need, 2);
1400     tx_cu(pll_locked); tx_cu(pfc_reason); tx_cu(relay_waiting); tx_c('\n');
1401 
1402     tx_s("$A"); tx_cu(ahf_active); tx_cf(m->apk, 4); tx_cf(ahf_ref_pk_f, 4);
1403     tx_cf(ahf_duty_pk, 3); tx_cf(ahf_resid_pct, 1); tx_cf(m->thd_c, 2); tx_c('\n');
1404 
1405     tx_s("$K"); tx_cu(mask); tx_cu(lst); tx_cu(cmd_ok); tx_cu(cmd_bad); tx_cu(rx_bytes);
1406     tx_c('\n');
1407 
1408     tx_s("$R");
1409     static const uint32_t order[N_CH] = { SLOT_V, SLOT_I, SLOT_A };
1410     for (uint32_t j = 0; j < N_CH; j++) {
1411         const uint32_t c = order[j];
1412         tx_cf(m->off[c], 1); tx_cu(m->mn[c]); tx_cu(m->mx[c]); tx_cu(m->clip[c]);
1413     }
1414     tx_cu(adc_ovr_cnt); tx_cu(win_dropped); tx_cu(tx_skip); tx_cf(m->i_noise, 4);
1415     tx_c('\n');
1416 
1417     tx_s("$E\n");
1418     uart_tx_dma((uint32_t)(tx_p - tx_buf));
1419 }
```

- L1376: if the previous frame is still being transmitted, skip this one and count it (the dashboard shows “tx-skip”).
- L1379–1383: five waveforms (voltage with 1 decimal, currents with 3).
- L1385–1390: the measurement line `$M`.
- L1392–1394: three harmonic lines.
- L1396–1400: capacitor bank line `$P`.
- L1402–1403: AHF line `$A`.
- L1405–1406: command / lock line `$K`.

- L1408–1415: ADC diagnostics `$R` for V, I, A (offset, min, max, clip), then overruns, dropped windows, skipped frames, current-sensor noise.
- L1417: end-of-frame marker.
- L1418: start the DMA transfer of everything written (≈4 KB). The function returns immediately.

26.3 nvic_init (L1421–1436)

```
1421 /* ========================================================================== */
1422 /*  Main                                                                      */
1423 /* ========================================================================== */
1424 static void nvic_init(void)
1425 {
1426     NVIC_SetPriority(TIM2_IRQn,         0);
1427     NVIC_SetPriority(DMA2_Stream0_IRQn, 1);
1428     NVIC_SetPriority(ADC_IRQn,          1);
1429     NVIC_SetPriority(USART2_IRQn,       2);
1430     NVIC_SetPriority(DMA1_Stream6_IRQn, 3);
1431     NVIC_EnableIRQ(TIM2_IRQn);
1432     NVIC_EnableIRQ(DMA2_Stream0_IRQn);
1433     NVIC_EnableIRQ(ADC_IRQn);
1434     NVIC_EnableIRQ(USART2_IRQn);
1435     NVIC_EnableIRQ(DMA1_Stream6_IRQn);
1436 }
```

Sets the priorities of Table 10.1 and enables the five interrupts in the NVIC. Priorities must be set before enabling, so no interrupt ever runs at a default priority.

26.4 on_window (L1438–1458)

```
1438 static void on_window(bool clean)
1439 {
1440     process_window(snap, &mw);
1441     pll_update(av[1], bv[1], mw.vrms > 0.0f);
1442     grid_v_present = mw.vrms > 0.0f;
1443     pll_apply();
1444     ahf_step(clean);
1445 
1446     if (settle_win) settle_win--;
1447     else            acc_add(&mw);
1448 
1449     if (++win_count >= TELEM_WIN) {
1450         win_count = 0;
1451         avg_t a;
1452         make_avg(&a, &mw);
1453         if (acc.n >= 3u) apfc_step(&a);
1454         telemetry_send(&mw, &a);
1455         acc_reset();
1456         GPIOA->ODR ^= GPIO_ODR_OD5;
1457     }
1458 }
```

- L1440: all measurements of this window.
- L1441–1443: update the grid lock from the voltage fundamental, record whether mains is present, and apply the new sample period.
- L1444: AHF control step.

- L1446–1447: add this window to the 0.5 s average, unless it is one of the settling windows after a relay change.
- L1449–1457: every 5th window: average; run the APFC if at least 3 clean windows were averaged; send telemetry; reset the sums; toggle the LED (blinks once per second).

26.5 `main` (L1460–1501)

```
1460 int main(void)
1461 {
1462     SCB->CPACR |= (0xFu << 20);
1463     __DSB(); __ISB();
1464 
1465     clock_init();
1466     gpio_init();                /* relays forced OFF before anything else */
1467 #if RELAY_SELFTEST
1468     relay_selftest();
1469 #endif
1470     tables_init();
1471     uart_init();
1472     tim2_init();
1473     adc_dma_init();
1474     tim4_init();
1475     nvic_init();
1476     sampling_sync_start();
1477 
1478     for (;;) {
1479         if (resync_req) {
1480             resync_req = false;
1481             sampling_sync_start();
1482         }
1483 
1484         pfc_cmd_pending = false;
1485         relay_service(ms_ticks);       /* applies manual locks immediately */
1486 
1487         __disable_irq();
1488         const int32_t half = win_ready;
1489         win_ready = -1;
1490         __enable_irq();
1491 
1492         if (half >= 0) {
1493             const bool clean = win_clean[half] != 0u;
1494             memcpy(snap, (const void *)&adc_buf[(uint32_t)half * N_WIN * N_CH], sizeof snap);
1495             on_window(clean);
1496         } else {
1497             __WFI();
1498         }
1499     }
1500 }
```

Line	Explanation
L1462	Enable the FPU: full access for coprocessors CP10 and CP11 (`CPACR` bits 20–23).
L1463	Barriers so the FPU is enabled before the next instruction executes.
L1465–1475	Initialisation in a deliberate order: clocks first (everything depends on them), GPIO second (relays forced OFF as early as possible), tables, UART, TIM2, ADC/DMA, TIM4, then the NVIC last, so no interrupt can fire before its peripheral is configured.
L1476	Start sampling.
L1478–1499	The endless main loop.

L1479–1482	If an ISR reported an ADC overrun or DMA error, restart acquisition cleanly.
L1484	Clear the legacy command flag.
L1485	Service the relays on every pass (fast reaction to commands and decisions).
L1487–1490	Take the ready-window number and clear it atomically, so a DMA interrupt between the read and the clear cannot be lost.
L1492–1495	If a window is ready: read its clean flag; copy the 3000 samples of that half into `snap` (6 KB, about 25 μs), so the processing works on a stable copy; process it.
L1497	Otherwise sleep until the next interrupt.

Key idea

The whole program is **interrupt-driven with a cooperative main loop**. Hard real-time work (sampling, PWM, relay instants) is done by hardware or in a very short top-priority ISR. Heavy computation (DFT, control) runs in the main loop, which has a full 100 ms per window. There is no operating system.

Part III

The PC dashboard

27 Dashboard architecture

27.1 What the dashboard is for

The STM32 does all measurement and control on its own. The dashboard on the PC (`dashboard.py`, 1312 lines of Python) only *shows* what the firmware measured and *sends* two kinds of command back: capacitor overrides and AHF on/off. If the PC is disconnected, the board keeps correcting the grid.

Division of labour

Firmware: sampling, DFT, PF, THD, PLL, capacitor decisions, AHF loop, PWM. **Dashboard**: parsing text frames, drawing plots, a few display-only numbers (tracking correlation ρ, spectrum match), and turning button clicks into `$C`/`$A` commands.

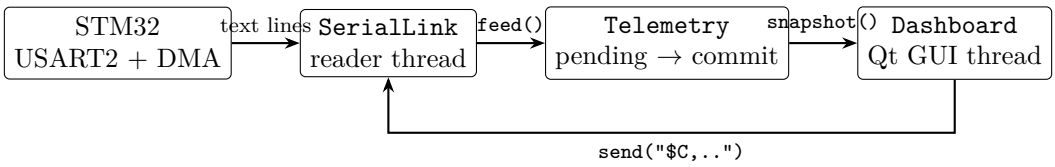
27.2 Libraries

Library	Use
`PySide6`	Qt 6 GUI toolkit: windows, tabs, buttons, layouts, timers
`pyqtgraph`	fast real-time plotting on top of Qt
`numpy`	arrays, FFT for the fallback spectrum, statistics
`pyserial`	the USB virtual COM port of the ST-LINK
`threading` (stdlib)	background reader thread, locks

Install and run:

```
python -m pip install numpy pyserial pyqtgraph PySide6
python dashboard.py --port COM6      # live board
python dashboard.py --demo           # simulator, no hardware
```

27.3 Threads and data flow



1. The **reader thread** (`SerialLink.run`) blocks on the serial port, splits the byte stream at `\n` and passes each line to `Telemetry.feed`.
2. `Telemetry` collects the lines of one frame in a *pending* dictionary. Only when `$E` arrives does it copy everything into the visible store, under a lock. The GUI therefore never shows the waveform of one frame with the numbers of the next.
3. The **GUI thread** runs a `QTimer` every 33 ms (about 30 fps). Each tick takes a `snapshot()` (a copy made under the lock) and redraws only if a new frame has arrived.
4. Button clicks call `link.send()`, which writes under a separate write lock so the reader and the GUI never touch the port at the same moment.

Why a thread?

Reading a serial port blocks. If the GUI thread did it, the window would freeze between frames. The reader thread blocks instead; Qt keeps painting smoothly. Python's GIL is released while waiting on the port, so the two threads do not slow each other down.

28 Dashboard code walkthrough

28.1 Protocol constants (L46–60)

- `CAP_UF = (2.5, 10.0, 20.0)`: the three capacitor steps, the same as `cap_uf[]` in the firmware.
- `CAP_RECLOSE_S`, `RELAY_MIN_ON_S` = 5 s, `PF_TARGET` = 0.99: copies of firmware constants, used only for text on screen.
- `N_SHOW = 2`: the firmware sends one averaged cycle; the dashboard draws it twice so two cycles are visible.
- `I1_MIN = 0.02` A: below this fundamental current a THD value is meaningless and is shown as "–".
- `M_KEYS`, `P_KEYS`, `A_KEYS`, `K_KEYS`, `R_KEYS`: the field names, *in the order the firmware sends them.* Field n of `$M` is `M_KEYS[n]`. If you add a field in `telemetry_send()` you must add the name here at the same position.

28.2 The frame format, side by side

Tag	Firmware (`telemetry_send`, L1374)	Fields
`$W,V`	`tx_wave('V', w_v, 1)`	100 voltage samples (every 2nd of 200), 1 decimal
`$W,I`	`tx_wave('I', w_i, 3)`	grid current, 3 decimals
`$W,A`	`tx_wave('A', w_a, 3)`	AHF current from sensor A
`$W,T`	`tx_wave('T', w_t, 3)`	target reference $-(I - I_1)$
`$W,C`	`tx_wave('C', w_c, 3)`	after compensation, $I + A$
`$M`	L1385–1390	vrms, irms, p, s, q, pf, thd_v, thd_i, crest, vpk, ipk, dpf, dva, freq
`$H,I/A/C`	L1392–1394	rms of harmonics 1–15 of I, A and I+A
`$P`	L1398–1400	state, cap, pf, q, hold, target, c_need, lock, why, wlock
`$A`	L1402–1403	active, peak, ref_peak, duty, resid, thd_c
`$K`	L1405	lock mask, lock state, cmd ok, cmd bad, rx bytes
`$R`	L1408–1414	per channel: offset, min, max, clip; then adc_ovr, win_drop, tx_skip, noise_i
`$E`	L1417	end of frame: commit

Link budget

A frame is about 4 kB (five waveforms of 100 numbers are most of it). At 115 200 baud with 10 bits per byte the link carries 11 520 bytes/s, so one frame takes about 0.36 s of the 0.5 s between frames: roughly 70 % utilisation. That is why the waveforms are decimated by 2 (`WAVE_DECIM`) and why the firmware counts `tx_skip` if the previous DMA transfer has not finished. The buffer `TX_BUF_SZ` = 5632 bytes bounds the worst case (0.49 s).

28.3 `Telemetry` (L64–151)

- `__init__`: the visible store (`wave`, `h`, `m`, `pfc`, `ahf`, `k`, `r`), counters, and a `threading.Lock`.
- `_kv(keys, fields)`: zips names with the comma-separated values and converts each to float.
- `feed(line)`: strips the line, ignores anything not starting with `$`, splits on commas, and dispatches on the tag. `$W` and `$H` carry a sub-tag (V, I, A, T, C) in field 1. A malformed number raises `ValueError`, which is counted in `bad_lines` instead of crashing.
- `_commit()`: swaps in a fresh pending frame, then under the lock merges the finished one into the store, updates the frame rate (exponential smoothing 0.8 old + 0.2 new) and increments `frames`.
- `snapshot()`: returns copies of all dictionaries under the lock, plus `age`, the time since the last frame. The GUI shows "NO DATA" when `age` > 2.5 s.
- `optimistic()`: when a button is pressed, the GUI shows the new state immediately; the next real frame (at most 0.5 s later) overwrites it with the firmware's truth.

28.4 `find_port` and `SerialLink` (L157–222)

- `find_port()`: looks for USB vendor ID `0x0483` (STMicroelectronics) or "STLink" in the description, then for COM35, then takes the first port.
- `run()`: an endless reconnect loop. It opens the port with a 0.2 s timeout, flushes stale input, and reads whatever is waiting (`ser.in_waiting or 1`). `*lines, buf = buf.split(b"\n")` keeps the incomplete last line in `buf` for the next read. If the buffer exceeds 16 kB without a newline (garbage), it is dropped. On any serial error the status text changes and the loop retries after 1 s, so unplugging and replugging the board just works.
- `send(cmd)`: writes ASCII bytes under `wlock`; returns `False` if the port is not open, and the GUI says "NOT SENT".

28.5 `DemoLink`: the built-in simulator (L228–394)

Runs the dashboard without hardware and emits *exactly* the firmware's line protocol every 0.5 s, so the same parser and GUI code are exercised.

- The grid: 230 V with 2 % fifth harmonic and noise; a lagging load whose angle wanders between 28° and 52° over 45 s; a rectifier-like pulse current; and the current of the switched capacitors $\omega C V \sqrt{2} \cos\theta$.
- `_dft()`: the same a_h, b_h DFT as the firmware, as a matrix product with precomputed cosine/sine tables.
- P, Q_1, DPF, D are computed with the firmware's formulas. The AHF is modelled as a converged loop that removes 97 % of h2–h15.
- The capacitor logic mirrors `apfc_step`: all combinations allowed by the locks, choose the closest to C_{need} with fewest relay changes, 0.75 µF deadband, and the 5 s min-ON / min-OFF timers.
- `send()` parses `$C` and `$A` with the same validation as the firmware's `cmd_exec`.

28.6 Helper functions (L397–425)

- `_corr(a, b)`: Pearson correlation of two waveforms. Used for AHF tracking ρ: +1 means the measured AHF current has exactly the shape of the target.
- `_spectrum(y)`: rms of harmonics 1–15 from an FFT of one cycle; used only if the firmware did not send `$H,C`.
- `cyc(y)`: repeats the cycle `N_SHOW` times.
- `_thd(h)`: $100\sqrt{\sum_{h\geq 2} I_h^2}/I_1$, or NaN when $I_1 < 20\,\text{mA}$.

28.7 Widgets (L430–775)

- **Theme and `QSS`**: colour constants and a Qt style sheet (light background, white cards). The colours are consistent: voltage blue, current orange, AHF green, target purple.
- `Card`: a white rounded frame with a title and a sub-title line.
- `Stat`: a label, a large value with unit, and a small sub-text; `set()` can colour the value (green/amber/red).
- `Pill`: the coloured status badge (LIVE / NO DATA).
- `PFGauge`: a semicircle drawn with `QPainter`. Red zone below 0.7, amber 0.7–0.9, green above 0.9; a dotted tick at the 0.99 target; a purple triangle for DPF. `step()` moves the needle 18 % of the remaining distance each 33 ms tick, so it glides instead of jumping.
- `_Grid`: draws grid lines only at major ticks. pyqtgraph's own grid replayed a large picture on every repaint and took about 85 % of paint time; this was the main cause of the earlier lag.
- `Plot`: a non-interactive `PlotWidget` (no zoom/pan, no menu) with the custom grid. `hover()` adds a vertical cursor, dots on each curve and a tooltip with the nearest sample's value (`np.searchsorted` on the x data).
- `_grid`, `_row`: small layout helpers.

28.8 `Dashboard` window (L777–1284)

28.8.1 Construction

- `__init__`: header, the two **common** plots (grid voltage and grid current, full width, one above the other), then a `QTabWidget` with the two tabs. Everything sits in a scroll area. A `QTimer` at 33 ms calls `_tick`.
- `hist`: a `deque` of up to 4000 rows (t, PF, DPF, C_{on}, C_{need}, THD before, THD after): 33 minutes at 2 frames/s; the trend plots show the last 120 s.
- `_build_pfc()`: PF gauge; eight stats (P, Q_1, S, D, V, I, C on, C required); capacitor bank card with the AUTO button and one button per capacitor; PF trend and capacitance trend plots.
- `_build_hm()`: eight stats (THD before/after, THD-V, crest factor, tracking ρ, spectrum match, H1 leak, I_1); the AHF card with ACTIVE/STANDBY buttons and a diagnostics text box; AHF-vs-target plot; before/after current plot; the grid spectrum (grey before, green after); the AHF spectrum with purple diamonds for what is required.

28.8.2 Commands

- `_on_auto()`: if all steps are in AUTO, it sends `$C,j,1,s` for each step j with s = its present state, which freezes the bank where it is (MANUAL). Otherwise it sends `$C,j,0,0` for all three, releasing them to AUTO.
- `_cap_cb(j)`: clicking a capacitor cycles **AUTO → MAN (toggled) → MAN OFF → AUTO**. The firmware still applies the 5 s timers and point-on-wave switching to manual commands, which is the protection against rapid on/off.
- `_ahf_cmd(en)`: sends `$A,1` or `$A,0`.

28.8.3 Refresh

1. `_tick()` advances the gauge animation and calls `_refresh()`.
2. `_refresh()` updates the status pill four times a second, then returns early unless a new frame has arrived (cheap when idle).
3. It reads the grid frequency from `$M`; the time axis is $1000/f$ ms per cycle, so the x-axis is correct even at 49.8 Hz.
4. The after-compensation spectrum comes from `$H,C` (exact, computed on the MCU).
5. Common plots are updated; then *only the visible tab* is updated, which halves the drawing work.
6. `_yscale()`: the y-range jumps up immediately when the signal grows, and relaxes 10 % per frame when it shrinks, so plots do not flicker.

`_update_pfc()` fills the gauge and stats, colours each capacitor button (green when on, amber border when manual, "→ ON (wait)" while a timer or lock delays a change), and explains a HOLD with the firmware's `why` code: 1 = no load, 2 = need less than half the smallest step, 3 = all manual.

`_update_hm()` computes ρ between `$W,A` and `$W,T`, the THD reduction percentage, the spectrum shape match (cosine similarity of the AHF spectrum to the required spectrum, as a percentage), the H1 leak (how much fundamental the AHF injects; it should be small), and the diagnostics block (ADC offsets, min/max, clipping, overruns, dropped windows, UART counters, removed current-sensor noise).

Which numbers are computed on the PC?

Only display aids: ρ, spectrum match, H1 leak, the percentage reduction, and the colours. Every electrical quantity (V, I, P, Q, S, D, PF, DPF, THD, harmonics, C_{need}, frequency) is computed by the STM32 and sent as text. If asked "is the PF calculated in Python?", the answer is no.

Part IV

Panel preparation

29 Numbers to remember

Quantity	Value	Where it comes from
Clocks		
HSI oscillator	16 MHz	internal RC, no crystal needed
PLL	M=16, N=256, P=4	$16/16 \times 256/4 = 64$ MHz
SYSCLK / AHB	64 MHz	flash 2 wait states, VOS scale 2
APB1 / APB1 timers	32 / 64 MHz	timers run at 2× when APB prescaler $\neq 1$
APB2 / ADC clock	64 / 16 MHz	ADC prescaler ÷4 (ADC max 36 MHz)
Sampling		
Sample rate f_s	10 kHz	TIM2: 64 MHz/6400
Samples per cycle	200	10 000/50, held exact by the PLL
Window	1000 samples = 100 ms = 5 cycles	one DMA half-buffer
Nyquist frequency	5 kHz	$f_s/2$; highest harmonic used is h15 = 750 Hz
ADC conversion	492 cycles = 30.75 µs	480 sampling + 12 conversion at 16 MHz
3-channel scan	92.25 µs	must fit in 100 µs (95.2 µs at 52.5 Hz)
Channel skew V–I	30.75 µs = 0.55° at 50 Hz	corrected by phasor rotation
DMA buffer	6000 halfwords = 12 kB	2 halves × 1000 × 3 channels
PWM		
PWM frequency	80 kHz	TIM4: 64 MHz/800
PWM periods per sample	8	80 kHz / 10 kHz
Duty resolution	800 steps, 0.125 %	ARR = 799
Max modulation	0.92	anti-windup limit `AHF_MOD_DEPTH`
Sensors		
ADC LSB	0.806 mV	3.3/4095
ACS712-05B	185 mV/A	zero at $V_{cc}/2$ (about 2068 counts measured)
K_I (grid, with divider)	6.534 mA/count	divider ratio 0.6667 (10 k series, 20 k to GND)
K_A (AHF, direct)	4.356 mA/count	no divider
K_V (ZMPT101B)	0.817 V/count	1014 V per ADC volt (calibrated)
Grid current range	about −13.5 to +13.2 A	set by the 3.3 V ADC range after the divider
Grid lock (PLL)		
Frequency range	47.5–52.5 Hz	`PLL_F_MIN/MAX`
Proportional gain	0.4 Hz/rad	`PLL_KP_HZ`
Lock threshold	$\lvert\varphi\rvert < 0.35$ rad (20°)	
Integer ARR resolution	0.0078 Hz	one tick of 6400
With 16-bit fraction	$\approx 1.2 \times 10^{-7}$ Hz	dithered ARR
APFC		
Steps	2.5 / 10 / 20 µF, 8 combinations	total 32.5 µF
Var per µF at 230 V, 50 Hz	16.62 var	$2\pi f V^2 \cdot 10^{-6}$

Quantity	Value	Where it comes from
Target PF / $\tan\varphi$	0.99 / 0.1425	
Deadband	0.75 µF	
Min ON / min OFF	5 s / 5 s	capacitor discharge, relay protection
Relay timing	operate 7 ms, release 5 ms, lead 2 ms	typical relay data
Point-on-wave indices	$K_{ON} = 130$, $K_{ON0} = 80$, $K_{OFF} = 130$	sample in the cycle when the coil is switched
Decision rate	every 0.5 s	5 windows averaged
AHF		
Harmonics cancelled	h2–h15	mask `0xFFFC`
Loop gain μ / probe / leak	0.5 / 0.15 / 0.002	
Identification rate	0.25	first measurement replaces the model
Update rate	10 per second	once per 100 ms window
Minimum target	0.02 A peak	below this the bridge is not driven
Communication and memory		
UART	115 200 baud (BRR 278, −0.08 %)	USART2 via ST-LINK virtual COM
Telemetry	2 frames/s, about 4 kB each	
Flash / RAM used	about 12 kB / 36 kB	of 512 kB / 96 kB

29.1 Measured results

Load	THD-I before	THD-I after (estimated)	Note
Laptop adapter	35.9 %	10.5 %	PF already about 0.9: small Q_1
0.69 A load	15.9 %	3.2 %	
0.204 A load	33.7 %	14.4 %	low current, close to sensor resolution
Phone charger	–	–	at the noise floor of the ACS712

APFC example from the dashboard (laptop)

$V = 246$ V, $P_1 = 97.5$ W, $Q_1 = 15.3$ var, no capacitor on. $k = 2\pi \cdot 50 \cdot 246^2 \cdot 10^{-6} = 19.01$ var/µF. $Q_{\text{tgt}} = 97.5 \times 0.1425 = 13.9$ var. $C_{\text{need}} = (15.3 - 13.9)/19.01 = 0.07$ µF, far below half the 2.5 µF step, so the controller correctly holds (reason 2). Switching 2.5 µF in would add 47.5 var leading and make the PF *worse*.

30 Likely panel questions with answers

30.1 System and concept

Q: What problem does your project solve?

A: Two power-quality problems of non-linear and inductive loads: low displacement power factor (reactive current) and harmonic distortion. A switched capacitor bank corrects the first; an active harmonic filter injects opposite harmonic currents to reduce the second. One STM32 measures, decides and drives both.

Q: Why two stages instead of only an active filter?

A: Reactive power at 50 Hz is large and steady; a capacitor supplies it passively at almost no loss. Harmonics are smaller but change with the load and cannot be cancelled by a capacitor. Using the active stage only for harmonics keeps its current, and the H-bridge rating, small.

Q: What is the difference between PF and DPF?

A: DPF $= \cos\varphi_1$ uses only the fundamental: $P_1/\sqrt{P_1^2 + Q_1^2}$. True PF $= P/S$ also includes harmonics: PF $\approx$ DPF$/\sqrt{1 + \mathrm{THD}^2}$. Capacitors improve DPF; only the AHF improves the distortion part.

Q: Why did the phone charger show PF 0.5 but no capacitor switched in?

A: Its low PF is mostly *distortion*, not displacement. Q_1 was only about 3 var, which needs about 0.2 µF. The smallest step is 2.5 µF (about 41 var), so switching it in overshoots to a leading PF (you saw −38 var). The controller holds when the need is below half the smallest step.

Q: What is distortion power D?

A: $D = \sqrt{S^2 - P^2 - Q_1^2}$: the part of apparent power caused by harmonic currents. It is not reactive power and a capacitor cannot cancel it.

30.2 Sampling and timing

Q: Why 10 kHz?

A: 200 samples per 50 Hz cycle: an integer count gives leakage-free DFTs. Nyquist is 5 kHz, well above h15 (750 Hz), and three ADC conversions (92.25 µs) fit inside the 100 µs sample period.

Q: How is the sample time triggered? Is it software?

A: No. TIM2's update event is routed as TRGO to the ADC's external trigger (EXTSEL = 6, rising edge). The ADC starts a scan of three channels by hardware and DMA moves the results. The CPU does nothing per sample except a short TIM2 ISR for the AHF output and relays.

Q: Why 480-cycle sampling time?

A: The ACS712 and ZMPT outputs have a few kΩ of source impedance. A long sampling time lets the ADC's sample capacitor charge fully, which reduces noise and crosstalk between channels. 480 cycles is the longest that still fits three channels in one sample period.

Q: The three channels are not sampled at the same instant. Does that matter?

A: The V sample is 30.75 µs after the I sample: 0.55° at 50 Hz. The firmware rotates the voltage

phasor by that angle for each harmonic before computing P and Q, so the error is removed.

Q: What is the purpose of the ping-pong (double) buffer?

A: The DMA fills one half while the CPU processes the other. Half-transfer and transfer-complete interrupts mark which half is ready. The CPU has 100 ms to process 1000 samples per channel, which takes a few milliseconds.

Q: How do you know the processing keeps up?

A: `win_dropped` counts windows that arrived before the previous one was processed, and the ADC overrun counter (`adc_ovr`) counts lost conversions. Both are shown on the dashboard diagnostics and stay at zero.

30.3 Timers and PWM

Q: How is 80 kHz PWM generated?

A: TIM4 at 64 MHz, prescaler 0, ARR 799: 64 MHz/800 = 80 kHz. Channels 1 and 3 in PWM mode 1 drive TB6612 IN1/IN2. CCR sets the duty; preload makes changes take effect at the next period so there are no glitches.

Q: Why 80 kHz?

A: It is above hearing, gives eight PWM periods per sample (smooth output), and the 10 mH inductor filters it well. The TB6612 is specified up to 100 kHz; higher would increase switching loss.

Q: Why is TIM4 reset by TIM2?

A: TIM4 is slave of TIM2 in reset mode (ITR1). Every sample, TIM4's counter restarts, so the PWM periods are phase-aligned to the sample clock. The PWM therefore never beats against the sampling, and the ADC always samples at the same point of the PWM period.

Q: What is sign-magnitude drive?

A: For a positive output one input is held high (CCR = `0xFFFF`, larger than ARR) and the other is pulsed; for negative, the roles swap. The bridge alternates between drive and brake (both low-side on), which is the TB6612's "short brake" mode: the inductor current recirculates instead of flowing back to the supply.

Q: What is the duty resolution?

A: 800 steps = 0.125 %. The duty is computed in Q15 fixed point, $d = (|v| \cdot p) \gg 15$, where p is the period in ticks.

Q: How does the sample rate change with the PLL without disturbing the PWM?

A: TIM2's ARR changes, and TIM4's period is set from TIM2's period divided by 8. The PWM frequency follows the grid, for example 79.6 kHz at 49.8 Hz.

30.4 ADC, DMA and interrupts

Q: Why DMA?

A: 30 000 conversions per second would cost 30 000 interrupts. DMA writes them to RAM with no CPU involvement; the CPU sees only two interrupts per 100 ms.

Q: What are your interrupt priorities and why?

A: TIM2 = 0 (highest): it writes the PWM duty and times the relays to one sample. DMA2 and ADC = 1: window hand-over and overrun recovery. USART2 RX = 2: command bytes. DMA1 TX = 3: end of telemetry transmission. The heavy DSP runs in `main`, so no ISR blocks the time-critical ones.

Q: How do you share data between ISRs and main safely?

A: Shared variables are `volatile`. Multi-word updates are made with interrupts briefly disabled (critical section). The AHF table is double-buffered: main writes the inactive table and requests a swap; the TIM2 ISR swaps it at sample 0 of a cycle, so the output never mixes two tables. `__DMB()` ensures memory writes are complete before the flag is set.

Q: What does WFI do?

A: Wait For Interrupt: the core stops until an interrupt arrives. It saves power and has no effect on timing, because all real-time work is in interrupts and DMA.

30.5 DSP

Q: Why DFT instead of FFT?

A: Only 15 harmonics are needed. A direct DFT on 200 points costs 15×200 multiply-adds per channel, with no restriction to power-of-two lengths. An FFT of 200 is not radix-2, and would compute 100 bins we do not need.

Q: What is coherent folding?

A: The window contains exactly 5 cycles. Adding sample k of each cycle gives an averaged cycle of 200 points. Noise and non-harmonic content average down, and the DFT then runs on 200 samples instead of 1000.

Q: How did you remove the current sensor noise?

A: The noise is random from cycle to cycle while the real current repeats. Comparing the mean square of the averaged cycle with the mean square of the raw samples gives the noise power: $S^2 = (K\,\overline{x}^2 - \overline{x^2})/(K-1)$. That power is subtracted from I^2_{rms}. Below 0.02 A rms the current is shown as zero.

Q: Why Butterworth filters, and why not on everything?

A: Butterworth is maximally flat in the passband, so harmonics up to 750 Hz are not distorted in amplitude. The AHF sensor uses a causal 1 kHz filter and the phase it adds is removed per harmonic using $H(e^{j\omega})$. V and I use a zero-phase (forward-backward) 1.5 kHz filter only for the displayed waveform. The measurements (P, Q, THD) use the DFT, which already rejects out-of-band noise; filtering them would only add error.

Q: How are the filter coefficients obtained?

A: Bilinear transform of the analog 2nd-order Butterworth with frequency pre-warping, $K = \tan(\pi f_c/f_s)$. At 1 kHz: $b = [0.0675, 0.1349, 0.0675]$, $a = [1, -1.1430, 0.4128]$. Direct form II transposed is used because it needs only two state variables and is numerically good in float.

30.6 PLL and grid lock

Q: Why do you need a PLL? Why not a fixed 10 kHz?

A: The grid is not exactly 50.000 Hz. At 50.1 Hz with fixed sampling, the 15th harmonic phase would drift 54° every 100 ms window, and the AHF would chase a moving target. The relays also need to know where the voltage peak is.

Q: How does it work?

A: The phase of the voltage fundamental at the window start, $\varphi = \text{atan2}(-b_1, a_1)$, is measured. Its change between windows gives the frequency; the phase itself is driven to zero with a proportional term. The result sets TIM2's ARR with a 16-bit fraction (dithered), so the sample rate is exactly 200 times the grid frequency.

Q: What happens without mains?

A: Below 20 V rms the voltage is treated as absent: values are zeroed, the PLL returns to 50 Hz, the APFC holds and relays wait up to 2 s for lock before switching anyway.

30.7 APFC

Q: How do you decide which capacitors to switch?

A: $C_{\text{need}} = (Q_{\text{load}} - |P_1| \tan \varphi_{\text{tgt}})/(2\pi f V^2)$, where Q_{load} adds back the capacitors already on. All 8 combinations allowed by manual locks are tried; the closest to C_{need} wins, ties broken by fewest relay changes. Nothing changes if the present choice is within 0.75 µF.

Q: Why measure Q_{load} by adding back the capacitors that are on?

A: The measured Q_1 already includes their leading vars. Adding them back gives the load's own demand, so the decision does not oscillate.

Q: What is point-on-wave switching?

A: The coil command is timed in the TIM2 ISR so the contacts close when the voltage across them is near zero (at the peak, where a capacitor left charged at $+V_{\text{pk}}$ matches the line) and open just before the peak, where the capacitor current is near zero. Timing: closing coil at sample 130, 7 ms operate time ⇒ contacts close at sample 200 = the next peak.

Q: Why did the 10 µF relay weld?

A: Closing a capacitor at a random instant can draw hundreds of amps for microseconds; the 10 A relay contacts weld. Rapid manual on/off made it worse, because the capacitor could still be charged with the opposite polarity (up to 650 V across the contacts). The fixes: 5 s minimum ON/OFF, point-on-wave timing, an NTC inrush limiter, a bleeder resistor, or a zero-cross SSR (used on PB2).

30.8 AHF

Q: How does the AHF generate the opposite current?

A: For each harmonic h the required current is $R_h = -I_h$. The measured AHF current M_h is compared to it, $E_h = R_h - M_h$, and the command updates as $U_h \mathrel{+}= \mu E_h / G_h$, where G_h is the measured gain from duty to current. The 14 harmonic commands are synthesised into one 200-sample table and output by PWM.

Q: Why measure the plant gain G_h?

A: The inductor, bridge and wiring change amplitude and phase at each frequency. Dividing by the right complex gain makes each harmonic converge in a few steps. G_h is identified from the change in M_h when U_h changes; the first measurement replaces the initial model, later ones are blended at 25 %.

Q: Is the "after" waveform real?

A: It is computed: grid current (sensor I) plus the measured AHF current (sensor A). Both are real measurements, but their sum is the expected grid current at the point of common coupling, not a third sensor. Measuring the grid side directly is the recommended next validation (it also confirms `A_SIGN`).

Q: Why only up to the 15th harmonic?

A: Most energy in rectifier currents is in h3, h5, h7, h9. The TB6612 and inductor limit the slew rate, and 15 harmonics keep the loop within the time budget. h15 = 750 Hz is well below the 5 kHz Nyquist limit and 80 kHz PWM.

Q: What prevents the AHF from saturating?

A: If the synthesised peak exceeds 0.92, the whole table is scaled down (anti-windup), and the stored commands are scaled too, so the integrator does not wind up. A small leak (0.002) slowly forgets harmonics that are no longer needed.

30.9 Communication

Q: Why 115 200 baud and what is the error?

A: It is the fastest standard rate the ST-LINK bridge handles reliably. BRR = 32 MHz/115 200 = 277.8 → 278, giving 115 108 baud, −0.08 % error; UART tolerates about ±2 %.

Q: Why text, not binary?

A: Readable in any serial terminal for debugging, robust resynchronisation at each newline, and the bandwidth is sufficient at 2 frames/s.

Q: How do commands work?

A: The USART2 RX interrupt collects bytes until a newline, parses `$C,i,mode,on` or `$A,e` with strict range checks, and applies them. Valid, invalid and total bytes are counted and returned in `$K`.

31 Limitations and honest answers

- **After-compensation THD is estimated** from sensor I plus sensor A; a third sensor on the grid side would measure it directly.
- **AHF power**: the TB6612 is a 1.2 A continuous (3.2 A peak) motor driver at 12 V on a low-voltage coupling; it demonstrates the control method. A grid-connected product needs an inverter with a DC bus above the grid peak and an isolation/coupling design.
- **Harmonic range**: h2–h15 only.
- **Sensor resolution**: 6.5 mA per count on the grid current. Very small loads (phone charger) are close to the noise floor.
- **Capacitor steps**: 2.5 µF minimum (about 41 var at 230 V); small reactive loads cannot be corrected without overshooting.
- **Relay inrush**: mechanical relays need NTC limiting and bleeder resistors; a zero-cross SSR is preferred.
- **Single phase only**.
- **Calibration**: K_V and the ACS712 sensitivities are nominal or single-point calibrated.

31.1 Future scope

Three-phase extension; a grid-side current sensor; an IGBT/MOSFET inverter with a 400 V DC bus; more harmonics with a faster ADC; thyristor-switched capacitors; data logging; IEC 61000-4-7 compliant measurement.

32 Glossary

Term	Meaning
AHF	Active Harmonic Filter: injects currents that cancel load harmonics
APFC	Automatic Power Factor Correction: switched capacitor bank
ARR	Auto-Reload Register: timer period minus one
CCR	Capture/Compare Register: sets PWM duty
CMSIS	Cortex Microcontroller Software Interface Standard: register definitions
DFT	Discrete Fourier Transform
DMA	Direct Memory Access: hardware copies data without the CPU
DPF	Displacement Power Factor, $\cos\varphi_1$
DF2T	Direct Form II Transposed: IIR filter structure
ISR	Interrupt Service Routine
NVIC	Nested Vectored Interrupt Controller
PCC	Point of Common Coupling: where load, AHF and grid meet
PLL	Phase-Locked Loop (hardware clock PLL, and the software grid PLL)
Q15	Fixed-point format: value $\times 2^{15}$ stored in 16 bits
RMS	Root Mean Square
SSR	Solid-State Relay
THD	Total Harmonic Distortion: $\sqrt{\sum_{h\geq 2} I_h^2}/I_1$
TRGO	Trigger Output: a timer event routed to another peripheral
VOS	Voltage scaling of the internal regulator
WFI	Wait For Interrupt instruction